

SDA

MADE BY: “Mujtaba Chohan”

ROLL NO: 24k-0531

Contents

SDA	1
Class diagram + SDLC models.....	1
Robustness Analysis Diagram	10
Activity Diagram.....	31
SEQUENCE DIAGRAM	57
EXAMPLE	64
EXAMPLE	76
EXAMPLE	77
4+1 ARCHITECTURAL MODELLING	79
LOGICAL VIEW:	80
PROCESS VIEW:	82
DEVELOPMENT VIEW:	83
PHYSICAL VIEW:	84
HOMOGENIZATION	87
TIMING DIAGRAM.....	91
COMPONENT DIAGRAM.....	94
DEPLOYMENT DIAGRAM	114
State Chart Diagram.....	130
DESIGN SOLUTIONS	144

Class diagram + SDLC models

1. Association

Symbol: Solid line →

Meaning: A generic link between two classes showing they are related in some way.

Usage in Class Diagram: Shows that one class uses or communicates with another.

Example:

Customer → Order (a customer can place orders)

Notes: Multiplicity can be added (1..*, 0..1) to indicate how many instances are involved.

2. Inheritance (Generalization)

Symbol: Solid line with hollow triangle →

Meaning: "Is-a" relationship, subclass inherits from superclass.

Usage in Class Diagram: Shows class hierarchy.

Example:

SavingsAccount → Account (SavingsAccount is a type of Account)

Notes: Only one parent class per inheritance arrow, but multiple inheritance is possible in some languages.

3. Realization / Implementation

Symbol: Dashed line with hollow triangle →

Meaning: A class implements an interface.

Usage in Class Diagram: Shows that the class provides the behavior specified by the interface.

Example:

CreditCardPayment --|> PaymentMethod (class implements the interface)

4. Dependency

Symbol: Dashed line with open arrow →

Meaning: One class depends on another temporarily (uses it in a method, parameter, or local variable).

Usage in Class Diagram: Indicates that a change in one class may affect another.

Example:

Invoice - - -> Printer (Invoice uses Printer to print but is not permanently linked)

5. Aggregation

Symbol: Solid line with hollow diamond →

Meaning: Whole-part relationship, but parts can exist independently.

Usage in Class Diagram: Shows that a class contains or groups other classes, but the parts are not strictly owned.

Example:

Library <|-- Book (books can exist without the library)

6. Composition

Symbol: Solid line with filled diamond →

Meaning: Strong whole-part relationship; parts cannot exist without the whole.

Usage in Class Diagram: Indicates ownership and lifecycle dependency.

Example:

House ◆-- Room (if House is destroyed, Rooms are destroyed too)

SDLC Models — Exam-Ready Definitions & Use Cases

1. Waterfall Model

Definition: The Waterfall Model is a linear, sequential software development model in which each phase must be fully completed before the next phase begins. The phases flow in one direction — like a waterfall — from Requirements → Design → Implementation → Testing → Deployment → Maintenance. There is no going back to a previous phase once it is completed.

Key Characteristics:

- Rigid and structured
- Heavy documentation at each phase
- No overlapping of phases

- Testing occurs only after coding is complete

When to Use:

- Requirements are clearly defined, fixed, and unlikely to change
 - The project is small to medium in size with a well-understood domain
 - Government or defense projects where documentation is mandatory
 - Example: Developing a payroll system for a government department where all rules and calculations are already defined by law
-

2. Spiral Model

Definition: The Spiral Model is a risk-driven, iterative software development model that combines elements of the Waterfall model and prototyping. Development proceeds in repeated cycles called "spirals," and each spiral passes through four quadrants: Planning, Risk Analysis, Engineering, and Evaluation. The focus is on identifying and resolving risks at every stage before moving forward.

Key Characteristics:

- Risk assessment is performed at every iteration
- Prototypes are built progressively
- Suitable for large, complex, and high-risk projects
- Each spiral produces a more refined version of the system

When to Use:

- Projects with high risk and uncertainty (e.g., new technology, unclear requirements)
 - Large-scale, long-term projects where risks must be managed early
 - Projects where user feedback is needed at regular intervals
 - Example: Developing a satellite control system or a banking core engine where failure could be catastrophic
-

3. Iterative Incremental Model

Definition: The Iterative Incremental Model divides the system into smaller, manageable modules called increments. Each increment goes through the full cycle of design, implementation, and testing independently. After each increment is built and verified, it is integrated with the previous increments. This continues until the complete system is delivered. Unlike Waterfall, the system is delivered in parts, not all at once.

Key Characteristics:

- System is broken into increments (functional modules)

- Each increment delivers a working piece of the software
- Users can begin using parts of the system early
- Feedback from earlier increments improves later ones

When to Use:

- When core functionality needs to be delivered quickly while additional features are still in development
 - When requirements are partially known at the start but expected to evolve
 - Large systems that can be logically divided into independent modules
 - Example: An e-commerce platform where login and catalog are delivered first, then cart, then payment gateway
-

4. Agile Model

Definition: The Agile Model is a flexible, iterative, and collaborative software development approach based on the Agile Manifesto (2001). It prioritizes individuals and interactions, working software, customer collaboration, and responding to change over rigid processes and documentation. Development is done in short, fixed-length cycles called sprints (typically 1–4 weeks), and working software is delivered at the end of each sprint. Scrum is the most popular framework for implementing Agile.

Key Characteristics:

- Short development cycles (sprints)
- Continuous customer involvement and feedback
- Requirements can change at any point
- Daily stand-up meetings, sprint reviews, and retrospectives
- Self-organizing, cross-functional teams
- Working software is the primary measure of progress

When to Use:

- Requirements are frequently changing or not fully known upfront
 - The client wants to be actively involved throughout development
 - Startups or product-based companies building user-facing applications
 - Web or mobile app development where market feedback shapes features
 - Example: A startup building a food delivery app where features are added, removed, or changed based on user response after each sprint
-

Quick Comparison Table

Factor	Waterfall	Spiral	Iterative Incremental	Agile
Flexibility	Very Low	Medium	Medium	Very High
Risk Handling	Poor	Excellent	Good	Good
Customer Involvement	Low	Medium	Medium	Very High
Documentation	Heavy	Heavy	Moderate	Minimal
Delivery	Single, at end	Per spiral	Per increment	Per sprint
Best for	Fixed, small projects	High-risk, large projects	Modular, large projects	Dynamic, fast-paced projects

Exam Tip: When a question asks you to *choose a model for a given scenario*, justify your answer based on: (1) how well-defined the requirements are, (2) the risk level, (3) whether incremental delivery is needed, and (4) how much customer involvement is expected.

Exam Question:

"A software house has been hired to develop an online banking system for a large national bank. The requirements are not fully defined yet, the budget is large, failure could have serious financial consequences, and the bank's management wants regular progress updates with the ability to change requirements as regulations evolve. Which SDLC model would you recommend and why?"

How to Approach It — Use the 4 Checkpoints:

Step 1 — How well-defined are the requirements? The question says requirements are *not fully defined* and may change due to evolving regulations. This rules out Waterfall immediately, because Waterfall requires requirements to be fixed from the start.

Step 2 — What is the risk level? An online banking system handles real money and sensitive data. Failure = financial loss and loss of public trust. Risk is very high. This points strongly toward the **Spiral Model**, which is built specifically around risk analysis at every cycle.

Step 3 — Is incremental delivery needed? The bank wants regular progress updates, which suggests they want to see the system being built progressively. Both Spiral and Iterative Incremental support this.

Step 4 — How much customer involvement is expected? Management wants to stay involved and change requirements. This favors Spiral or Agile. However, since the system is large-scale, high-risk, and not a fast-paced startup environment, pure Agile may be too informal.

Your Answer:

Recommended Model: Spiral Model

The Spiral Model is the most suitable choice here because the project is large and complex, the requirements are not fully clear upfront and will evolve with regulations, and the risk of failure is extremely high given that it is a financial system. The Spiral Model's core strength is its dedicated risk analysis phase in every iteration, which ensures that potential threats — technical, financial, or operational — are identified and resolved before development proceeds further. Additionally, each spiral produces a working prototype that management can review, allowing requirements to be refined progressively. This satisfies the bank's need for regular visibility into progress while also accommodating changing requirements without destabilizing the entire project.

Waterfall is ruled out because requirements are unstable. Pure Agile is ruled out because the system is too large and high-stakes for informal short sprints without thorough risk management. Iterative Incremental is a close alternative but lacks the formal risk assessment mechanism that a banking system demands.

Imagine a university hires you to build a complete student portal. Instead of waiting 2 years for everything, they want something usable as soon as possible.

So you deliver it in pieces:

- **Month 2** → Student login and profile system is live. Students can already use it.
- **Month 4** → Course registration module added. Now students can register for courses too.
- **Month 6** → Fee payment module added.
- **Month 8** → Result and transcript system added.
- **Month 10** → Full system complete.

Step 1 — How well-defined are the requirements? Mostly known and stable. Rules out Spiral (Spiral is for unclear, risky requirements).

Step 2 — What is the risk level? Low. A student portal is not life-critical or financially dangerous. Rules out Spiral again.

Step 3 — Is incremental delivery needed? Yes! The university explicitly wants to start using parts early — login first, then registration, then fees, etc. This is the biggest clue. Points directly to **Iterative Incremental Model**.

Step 4 — How much customer involvement is expected? Moderate. They review each delivered module but aren't changing requirements every week. Rules out Agile (Agile needs very high, constant involvement).

Your Answer:

Recommended Model: Iterative Incremental

Because requirements are clear, risk is low, and the client wants working modules delivered progressively rather than waiting for the complete system. Each increment (login → registration → fees → results) is independently built, tested, and handed over, allowing the university to benefit from the system while development is still ongoing.

"A small startup wants to build a food delivery mobile app. They have a rough idea but no clear requirements. They want to launch something basic quickly and keep improving it based on user feedback every few weeks."

Step 1 — How well-defined are the requirements? Only a rough idea exists, requirements are unclear and will keep changing based on user feedback. Rules out Waterfall completely.

Step 2 — What is the risk level? Low to moderate. It's a startup app, not a banking or defense system. No catastrophic consequences if something goes wrong. Rules out Spiral.

Step 3 — Is incremental delivery needed? Yes, they want to launch something basic quickly. But more importantly, they want to **keep changing and improving every few weeks** based on feedback — this is more than just incremental delivery, this is continuous adaptation. Points strongly to **Agile**.

Step 4 — How much customer involvement is expected? Very high. User feedback after every few weeks directly shapes the next version. This is the defining trait of Agile.

Answer:

Recommended Model: Agile

Because requirements are unclear and constantly evolving based on real user feedback, the team needs to work in short sprints (1–4 weeks), deliver something working quickly, collect feedback, and immediately adapt the next sprint accordingly. Agile is built exactly for this — fast-paced, uncertain, user-driven environments like startups.

Iterative Incremental is ruled out because although it also delivers in parts, it doesn't handle **constantly changing requirements** as smoothly as Agile does. The startup doesn't just want pieces delivered — they want to **pivot and reshape** the product continuously, which is Agile's core strength.

Key Trick to Remember:

Clue in Question	Model
"Requirements keep changing based on feedback"	Agile
"Want to use parts early, requirements are clear"	Iterative Incremental
"High risk, large, unclear requirements"	Spiral
"Everything is fixed and documented"	Waterfall

Robustness Analysis Diagram

◆ STEP 1: Why Do We Need This?

- Use Case → tells **WHAT** the system should do (user view)
- But it **doesn't** tell:
 - Which objects are involved
 - How they interact

👉 So there's a gap between:

- Requirements (**WHAT**)
- Design (**HOW**)

✓ Use Case Realization fills this gap

◆ STEP 2: What is Use Case Realization?

It answers:

“How is this use case actually executed inside the system?”

Example:

- Use Case: *User logs in*
- Realization:
 - Login Page (takes input)
 - Controller (processes logic)
 - User Entity (checks data)

👉 Output:

- **Interaction diagrams** → flow between objects
- **Class diagrams** → involved classes

◆ STEP 3: What Are Analysis Classes?

- Early **rough identification of system objects**
- Not final — just a **starting point**

👉 Important:

They can change, split, or disappear later

✓ Purpose:

- Help you move from **idea** → **structure**

🧠 One-line summary (remember this):

Use Case = WHAT

Use Case Realization = HOW

Analysis Classes = WHO (objects involved)

STEP 4: The ECB Pattern — The Heart of This Topic

To identify analysis classes, we use the **ECB Pattern** (Entity, Control, Boundary). This is a refined version of the famous **MVC Pattern** (Model-View-Controller).

Think of it as: every system has exactly **3 types of things happening**:

Type	Simple Question to Ask	Real World Analogy
Boundary	What does the user SEE and interact with?	The restaurant menu you read
Control	What LOGIC processes the request?	The waiter who takes your order and coordinates
Entity	What DATA is stored?	The kitchen's recipe/ingredient record

◆ STEP 5: Boundary Class

- Interface between **actor** ↔ **system**
- **Actors ONLY talk to Boundary**

✓ Examples:

- LoginPage
- RegistrationForm

✓ Types:

1. UI (screens/forms)
2. System interface (external APIs, payment gateway)
3. Device interface (hardware like scanner)

👉 Rule:

Usually **1 Boundary per use case (or screen)**

Think of it like a bank's reception desk. You (the customer/actor) never walk directly into the vault (entity) or the manager's office (control). You always go through the front desk (boundary) first.

Rule: Actors can **ONLY** talk to Boundary classes. Never directly to Control or Entity.

◆ STEP 6: Control Class

- Handles **logic + flow of use case**
- Connects **Boundary ↔ Entity**

✓ Does:

- Validation
- Decisions
- Process handling

✓ Example:

- LoginController → checks credentials, decides access

👉 Rule:

Usually **1 Control per use case**

Example: In a login use case — the LoginController checks if the username/password match, decides whether to grant access, and tells the boundary what to display.

◆ STEP 7: Entity Class

- Represents **stored/persistent data**
- Lives beyond one use case

✓ Examples:

- Student
- Course
- Account

👉 Key idea:

Entity = **data inside system**

Actor = **external user**

Important distinction from your slides: A Student entity class is NOT the same as the Student actor. The actor is an external person using the system. The entity is the data about that student stored INSIDE the system. They can coexist.

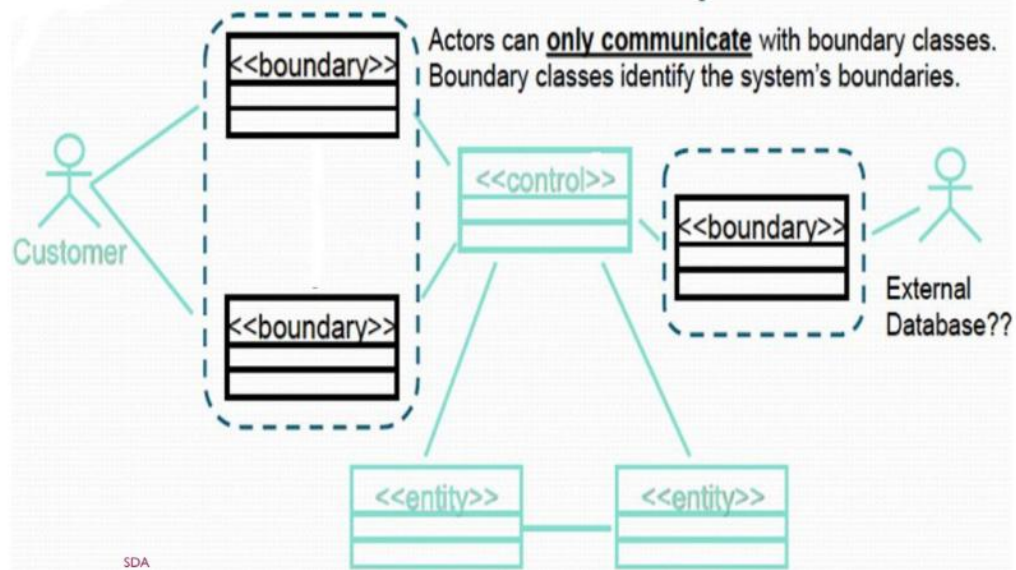
STEP 8: ECB Communication Rules — Very Important for Exams

This is where most students lose marks. Not everything can talk to everything.

✅ ALLOWED Communication:

From	Can Talk To
Actor	Boundary only
Boundary	Control only
Control	Control, Entity, Boundary
Entity	Entity only

THE ROLE OF BOUNDARY CLASS



STEP 9: Reading the Login Diagram From Your Slides

Your slides show this scenario:

- Step 1: System displays Login Page to User
- Step 2: User enters username and password
- Step 3: System validates User details against the Account

Let's trace how this becomes a Robustness Diagram:

Identifying the classes:

Boundary: LoginPage — this is what the User sees. The User (actor) connects to LoginPage.

Control: The controller in the middle — handles "User enters username and password" and triggers the validation logic.

Entity: Account — this is the stored data that gets checked during validation.

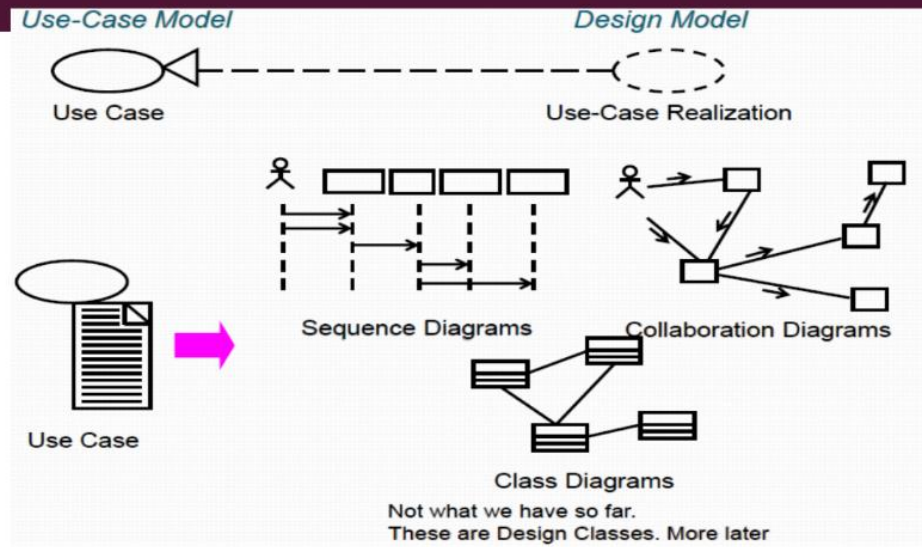
Flow: User (actor) → LoginPage (boundary) → Controller (control) → Account (entity)

This matches exactly the diagram in your slides. The actor connects only to the boundary, the boundary connects to control, and control connects to the entity.

The Golden Rule to memorize:

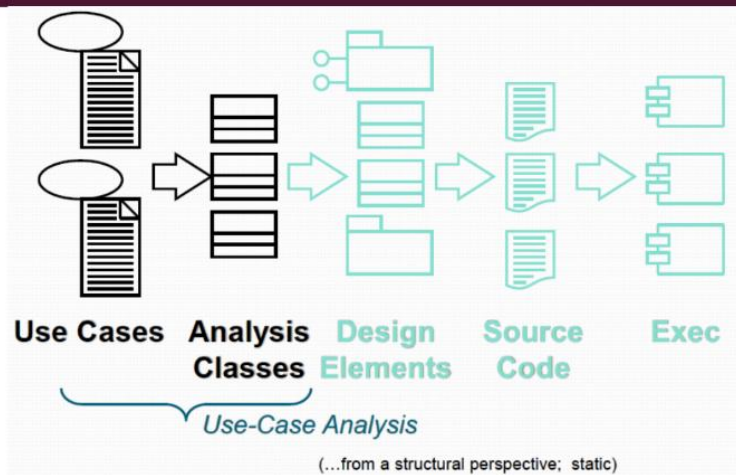
- Actor → Boundary → Control → Entity
- Everything must pass through this chain. No shortcuts.

USE CASE REALIZATION



CPA

ANALYSIS CLASSES: A FIRST STEP TOWARDS EXECUTABLES



CPA

ENTITY, CONTROL, AND BOUNDARY DESIGN PATTERN

Stereotype	UML Element	Element in analysis class diagram	Icon in the Rational Unified Process
«boundary»	Class	Class with stereotype «boundary»	
«control»	Class	Class with stereotype «control»	
«entity»	Class	Class with stereotype «entity»	

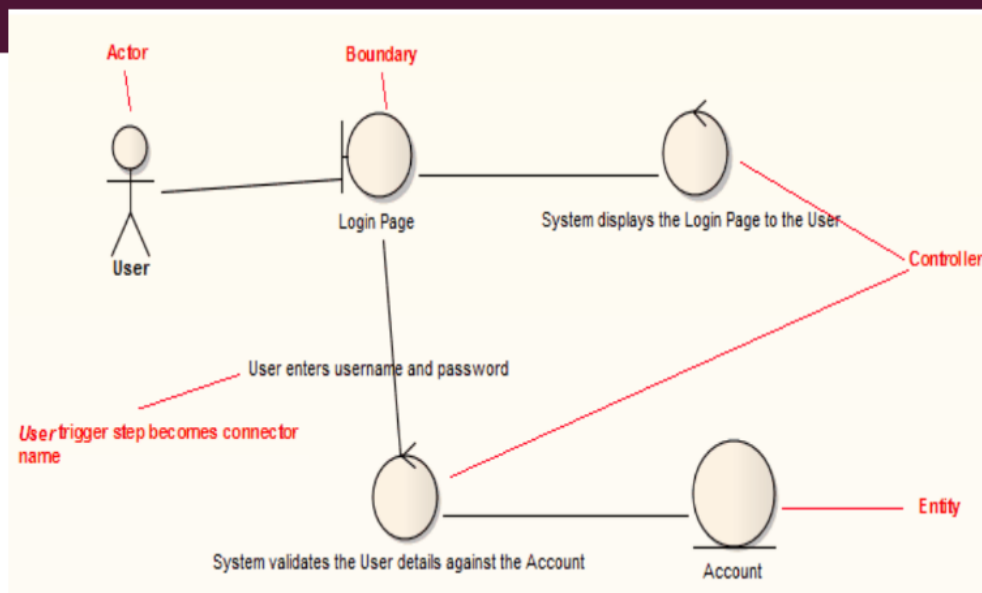
ALLOWED COMMUNICATION WITHIN THE ECB DESIGN PATTERN

Actor or Object Initiating Communication	Flow of Communication	Target Object
		
		
		
		

First one is boundary , second one is control ,third is entity

ROBUSTNESS DIAGRAM ACTIVITY

Step	Action
 1	System displays the <u>Login Page</u> to the <u>User</u>
 2	<u>User</u> enters username and password
 3	System validates the <u>User</u> details against the <u>Account</u>



Now Every Connection — One by One

Connection 1: User → Login Page

Line: Straight horizontal line from User (actor) to Login Page (boundary)

What it means: The User interacts with the Login Page. This is the entry point. The actor touches the system for the first time here.

Rule being followed: Actor can ONLY communicate with Boundary. So the User cannot jump to Controller or Account directly. The Login Page is the door.

Real life: You open your browser and the login screen appears in front of you. That screen IS the boundary.

Connection 2: Login Page ↔ Controller

Line: Line from Login Page (boundary) to Controller (control), labeled *"System displays the Login Page to the User"*

What it means: When the system needs to display the Login Page, the Controller is responsible for triggering that display. The controller tells the boundary what to show.

Direction reading: The label says "System displays Login Page to the User" — meaning the Controller is initiating this. The Controller instructs the Boundary to render/show itself to the user.

Connection 3: Login Page → Controller

Line: Diagonal line going DOWN from Login Page to Controller, labeled *"User enters username and password"*

What it means: When the User types their username and password into the Login Page and hits submit, that input is passed FROM the boundary TO the controller for processing.

Important note from the diagram: It says *"User trigger step becomes connector name."* This means whenever the USER does something (enters data, clicks a button), that action becomes the label on the line going from Boundary to Control.

Rule being followed: Boundary sends input to Control. Allowed.

Connection 4: Controller ↔ Account

Line: Horizontal line from Controller (control) to Account (entity), labeled *"System validates the User details against the Account"*

What it means: The Controller takes the username and password it received, then goes to the Account entity (the stored data in the database) to check if they match.

Rule being followed: Control communicates with Entity. This is allowed.

The Full Flow — Reading It As A Story

```
User
|
| (opens the system)
↓
Login Page (Boundary)
|
| Controller says "display yourself"
↔ Controller
|
| User types credentials → sent to Controller
↓
Controller (Control)
|
| "let me check if this is valid"
↔ Account (Entity)
|
| match found / not found → result goes back up
```


In plain English:

1. User arrives → sees Login Page (boundary shows itself because controller told it to)
2. User types username + password → Login Page sends this to Controller
3. Controller takes credentials → checks against Account entity in database
4. Account confirms or denies → Controller processes the result
5. Controller tells Boundary what to display next (welcome screen or error)

The One Red Note — "User trigger step becomes connector name"

This is a diagramming rule your slide highlights in red. It means:

Whenever you write a step like *"User enters username and password"* — because the USER is doing the action, this step's text becomes the **label on the arrow** from Boundary to Control.

Contrast this with *"System displays the Login Page"* — because the SYSTEM is doing it, the label goes on the Boundary ↔ Controller connection on the system's side.

So the rule is:

- **User does something** → label goes on Boundary → Controller arrow
- **System does something** → label goes on the system-side connections

Summary Table of All Connections

Connection	Label	Meaning
User → LoginPage	(none)	Actor enters system through boundary
LoginPage ↔ Controller	"System displays Login Page to User"	Controller triggers the display
LoginPage → Controller	"User enters username and password"	User input travels to controller
Controller ↔ Account	"System validates details against Account"	Controller queries the entity/database

- From the student detail page, the teacher clicks on the "Add courses" button and the system displays the list of courses on the course list page. The teacher selects the name of a course and presses the "Register" button. The system registers the student for the course. And displays the confirmation message on Confirmation screen.

When given a scenario in an exam, follow this process:

Step 1: Read the scenario and underline all nouns (they become entity candidates) and all screens/pages/windows (they become boundary candidates) and all verbs/actions (they become control candidates).

Step 2: Identify your actors from the text.

Step 3: For each screen or page mentioned → create a Boundary class.

Step 4: For each piece of data that needs to be stored or retrieved → create an Entity class.

Step 5: For each action or process that happens (validate, register, approve, verify) → create a Control class.

Step 6: Connect them following the ECB communication rules strictly.

Nouns – entity candidates + boundary + actor

Screens / page / windows - boundary candidates

Verbs – control candidates

Identify actors from text

And for each screen or page , create a boundary class

In the context of robustness diagrams specifically, nouns are useful because they point to **things the system needs to remember or manage** — which makes them entity candidates.

Types of Nouns to Look For

People/Roles — Teacher, Student, Advisor, Professor These often become actors OR entities depending on whether they use the system or are just stored data in it.

Objects/Things — Course, Schedule, Payment, Order, Receipt These are the core data the system works with. Almost always become entities.

Screens/Pages/Forms — Login Page, Course List Page, Confirmation Screen These are nouns too — but they become boundaries, not entities, because they are interfaces not data.

Documents/Records — Transcript, Invoice, Report, Form Usually become entities because they represent stored information.

Example From Your Scenario

"From the **student detail page**, the **teacher** clicks Add Courses and the system displays the list of courses on the **course list page**. The **teacher** selects the name of a **course** and presses Register. The system registers the **student** for the **course** and displays the confirmation message on **confirmation screen**."

Noun	Type
Student detail page	Boundary
Teacher	Actor
Courses	Entity
Course list page	Boundary
Course	Entity
Student	Entity
Confirmation screen	Boundary

One actor , pages one boundary , rest of nouns , entity

Types of Verbs to Look For

User actions — clicks, selects, presses, submits, enters These tell you what the actor does — they become the **labels on arrows** from boundary to control.

System processing actions — validates, registers, approves, verifies, calculates, checks These are the core logic the system performs. Almost always become **control classes**.

Display actions — displays, shows, renders, presents These represent the system telling a boundary what to show. Usually become a **display controller**.

Display controller – when there are display actions , like show , render .

It represent that system is telling a boundary what to show.

Validates , register , approves – all are system actions and become control classes.

While clicks , selects – tell actors what to do . so they become labels on arrows from boundary to control

When System Does It — Becomes a Control Circle

"displays", "registers" — all system actions — appear as **actual circles with the curved arrow** (the control symbol) in the diagram.

Look at your diagram:

- "displays" → became the **Display Controller circle**
- "registers" → became the **Register Controller circle**

Example From Your Scenario

"From the student detail page, the teacher **clicks** Add Courses and the system **displays** the list of courses on the course list page. The teacher **selects** the name of a course and **presses** Register. The system **registers** the student for the course and **displays** the confirmation message on confirmation screen."

Verb	Who Does It	Becomes
clicks	Teacher (user)	Label on arrow: boundary → control
displays (course list)	System	DisplayController
selects	Teacher (user)	Label on arrow: boundary → control
presses	Teacher (user)	Label on arrow: boundary → control
registers	System	RegisterController
displays (confirmation)	System	DisplayController

The Simple Rule to Remember

Word Type	Done By	Becomes
Verb — user does it	Actor	Arrow label (boundary → control)
Verb — system does it	System	A Control class

Step 2 — Identify Actors

- Teacher — the only one interacting with the system

Step 3 — Boundary Classes (every screen mentioned)

- StudentDetailScreen
- CourseListPage
- ConfirmationScreen

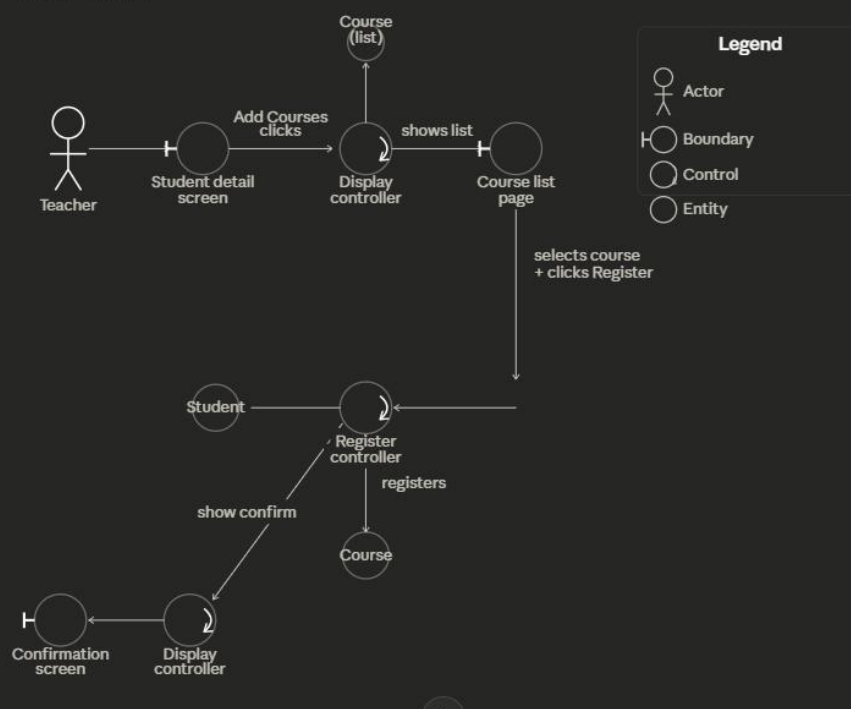
Step 4 — Entity Classes (data stored/retrieved)

- Course — list of courses fetched and stored
- Student — the student being registered

Step 5 — Control Classes (every action/process)

- DisplayController — handles displaying the course list
- RegisterController — handles the registration logic

The Diagram



ok when do you decide i have to connect ot entity class like you have done above regiter controller

Great question. Here is the exact thinking process:

The Question to Ask

Whenever you have a control class, ask yourself:

 "Does this control need to READ or WRITE any data to do its job?"

If yes → connect it to an entity.

If no → no entity connection needed.

Applied to RegisterController

The scenario says: *"The system registers the student for the course."*

Now ask — to register a student, what data does the system need?

- It needs to know **which student** → Student entity
- It needs to know **which course** → Course entity

Without accessing these two, registration is impossible. So RegisterController **must** connect to both.

to both.

Applied to DisplayController

The scenario says: *"System displays the list of courses."*

Now ask — to display courses, what data does the system need?

- It needs to fetch **the course list** → Course entity

So DisplayController connects to the Course entity to fetch that data.

The Simple 3 Questions to Ask Every Time

STEP 13: Key Exam Points Summary

What is a Robustness Diagram? A diagram that bridges use cases and design by identifying Boundary, Control, and Entity classes and showing how they collaborate to realize a use case.

What is an Analysis Class? An early, rough-draft class that represents initial data and behavior requirements, without implementation details. It comes in three stereotypes: Boundary, Control, Entity.

What is Use Case Realization? The process of describing how a use case is carried out in terms of collaborating objects, shown through interaction diagrams and class diagrams.

The one rule that earns marks every time: Actors → Boundary → Control → Entity. No direct actor-to-entity or boundary-to-entity communication is ever allowed.

Yes, exactly. You can see this directly in the diagram.

When Teacher Does It — Becomes an Arrow Label

"clicks", "selects", "presses" — all Teacher actions — appear as **text written ON the connecting lines** in the diagram.

Look at your diagram:

- "clicks Add Courses" → written on the arrow between Student Detail Screen and Display Controller
- "selects course + clicks Register" → written on the arrow between Course List Page and Register Controller

The action itself is not a shape — it is just a label sitting on a line.

When System Does It — Becomes a Control Circle

"displays", "registers" — all system actions — appear as **actual circles with the curved arrow** (the control symbol) in the diagram.

Look at your diagram:

- "displays" → became the **Display Controller circle**
- "registers" → became the **Register Controller circle**

The action becomes a physical shape — a control class with its own circle.



Paragraph 1

"The process begins by professors deciding on which courses they will teach for the semester. The Registrar's office then enters the information into the computer system, allocating times, room, and student population restrictions to each course. A batch report is then printed for the professors to indicate which courses they will teach. A course catalogue is also printed for distribution to students."

Actors Found

- **Professor** — decides courses
 - **Registrar** — enters info into system
 - **Student** — receives catalogue (passive here, but identified as actor)
-

Boundaries Found

- **RegistrarUI** — screen through which Registrar enters course info
-

Entities Found

- **Course** — times, room, population restrictions stored
 - **BatchReport** — printed for professors
 - **CourseCatalogue** — printed for students
-

Entities Found

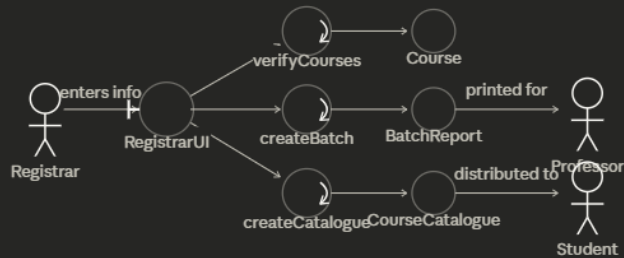
- **Course** — times, room, population restrictions stored
 - **BatchReport** — printed for professors
 - **CourseCatalogue** — printed for students
-

Controls Found

- **createBatch** — system creates the batch report
 - **verifyCourses** — system verifies course info entered
 - **createCatalogue** — system creates the course catalogue
-

Mini Diagram — Paragraph 1 Only

...



Paragraph 2

"Students then select what courses they desire to take and indicate these by completing paper-based course advising forms. They then meet with an academic advisor who verifies their eligibility to take the selected courses, that the sections of the courses selected are still open, and that the schedule of selected courses does not clash."

Actors Found

- **Student** — selects courses, meets advisor
- **Advisor** — verifies eligibility, checks clashes

Boundaries Found

- **StudentUI** — student interacts here to select courses
- **AdvisorUI** — advisor interacts here to verify

Entities Found

- **Course** — checked for open sections
- **StudentSchedule** — checked for clashes

Controls Found

- **verifyEligibility** — checks if student is eligible
- **verifyClash** — checks schedule does not clash
- **checkOpenSections** — checks if course sections still open



The Sentence

"They then meet with an academic advisor"

Applying the Verb Test

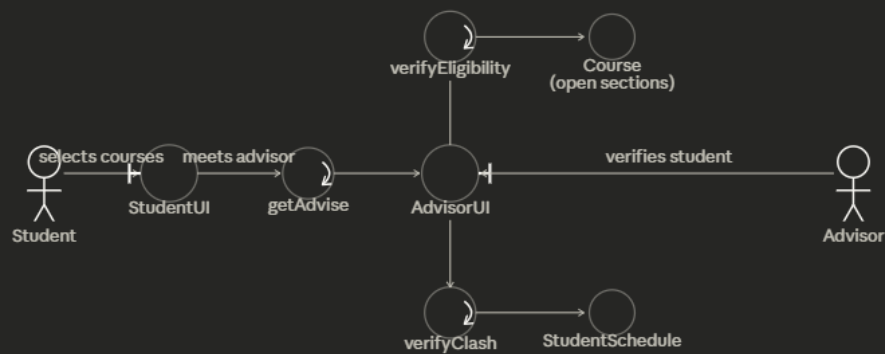
The verb here is "meet with" — now ask the two questions:

Who does it? — The Student does it. It is a student-triggered action.

But wait — the student meeting the advisor is not just a label on an arrow. Something has to **coordinate and manage** that meeting process inside the system. The system needs to facilitate the connection between the student and the advisor.

So the system action behind "meet with" is essentially "get advising" — the system processes the request and routes the student to the advisor.

Mini Diagram — Paragraph 2 Only



Paragraph 2 — Student selects courses, advisor verifies eligibility and clashes

Paragraph 3

"The advisor then approves the courses and completed the course registration forms for the student. These are then sent to the registrar who keys them into the registration system — thereby formally registering a student. If courses selected are not approved, the student has to select other courses and complete the course advising forms afresh."

Actors Found

- **Advisor** — approves courses
- **Registrar** — keys registration into system
- **Student** — reselects if not approved

Boundaries Found

- **AdvisorUI** — advisor approves here
- **RegistrarUI** — registrar keys in forms here
- **StudentUI** — student reselects here if rejected

Entities Found

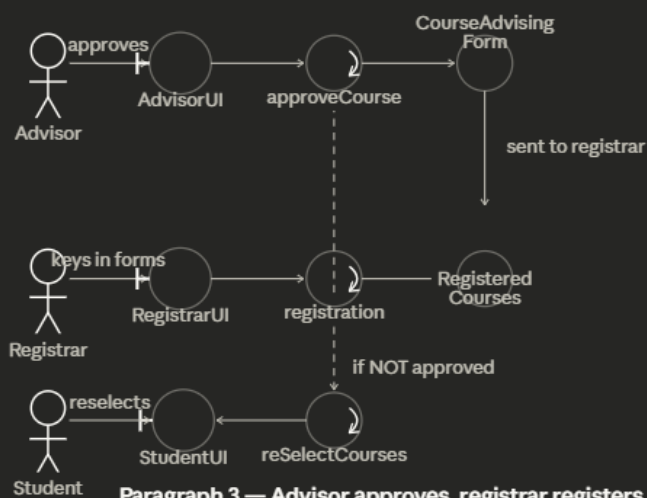
- **RegisteredCourses** — formally stored registration record
- **CourseAdvisingForm** — form completed by advisor

Controls Found

- **approveCourse** — advisor approves or rejects
- **registration** — formally registers student in system
- **reSelectCourses** — triggered when courses not approved



Mini Diagram — Paragraph 3 Only



Paragraph 3 — Advisor approves, registrar registers, rejection path shown



Applying the Test

Who does it? — The Advisor does it. It is an advisor-triggered action.

Is it a human doing it or the system? — The HUMAN (Advisor) is doing it by clicking or confirming something on their screen.

So By The Rule

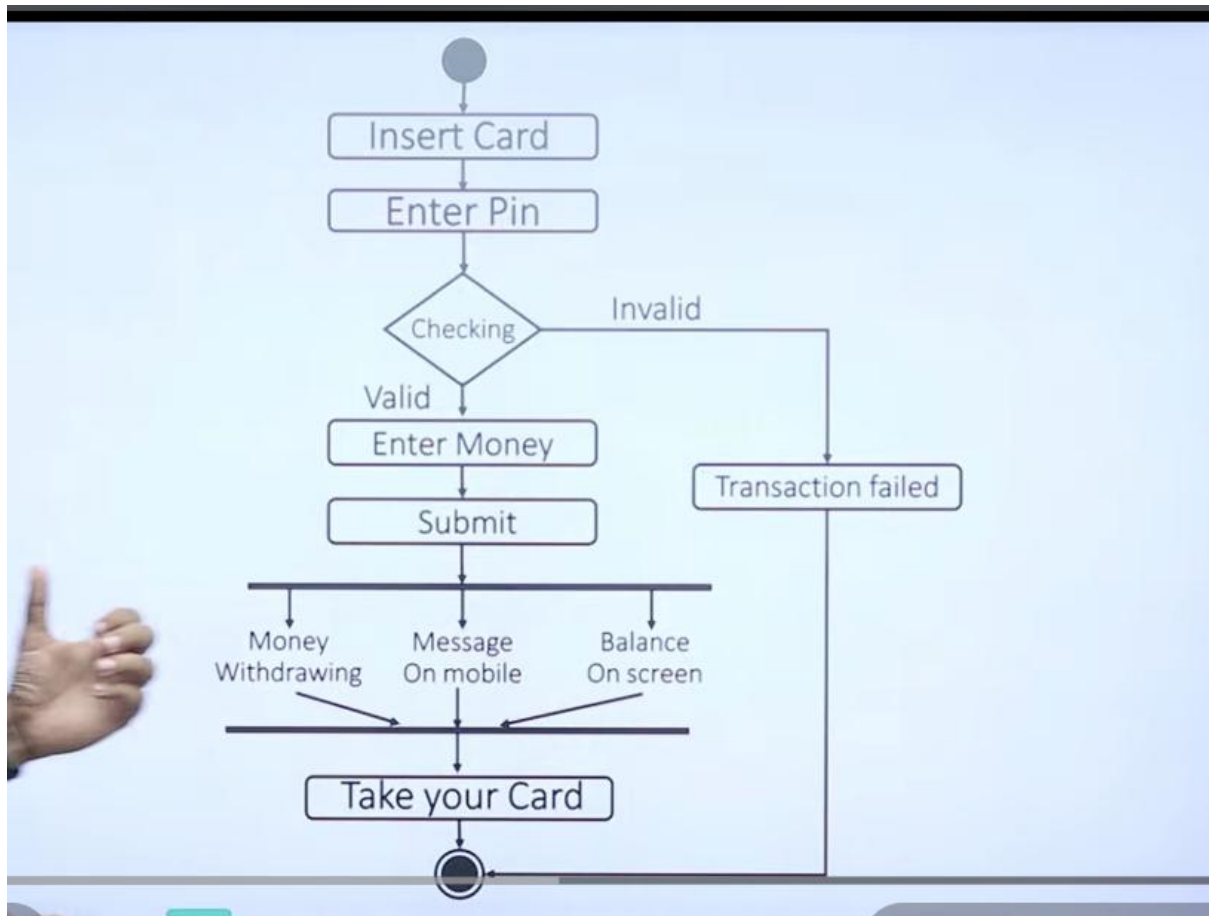
Since the **human (Advisor) is doing it** → it becomes an **arrow label** from boundary to control.

```
Advisor → AdvisorUI --- (approves) ---> approveCourse (control)
```

The word "approves" is what the advisor physically does on the AdvisorUI screen — so it becomes the label on the arrow going from AdvisorUI to the approveCourse control class.

Approves come on top of approve courses line.

Activity Diagram



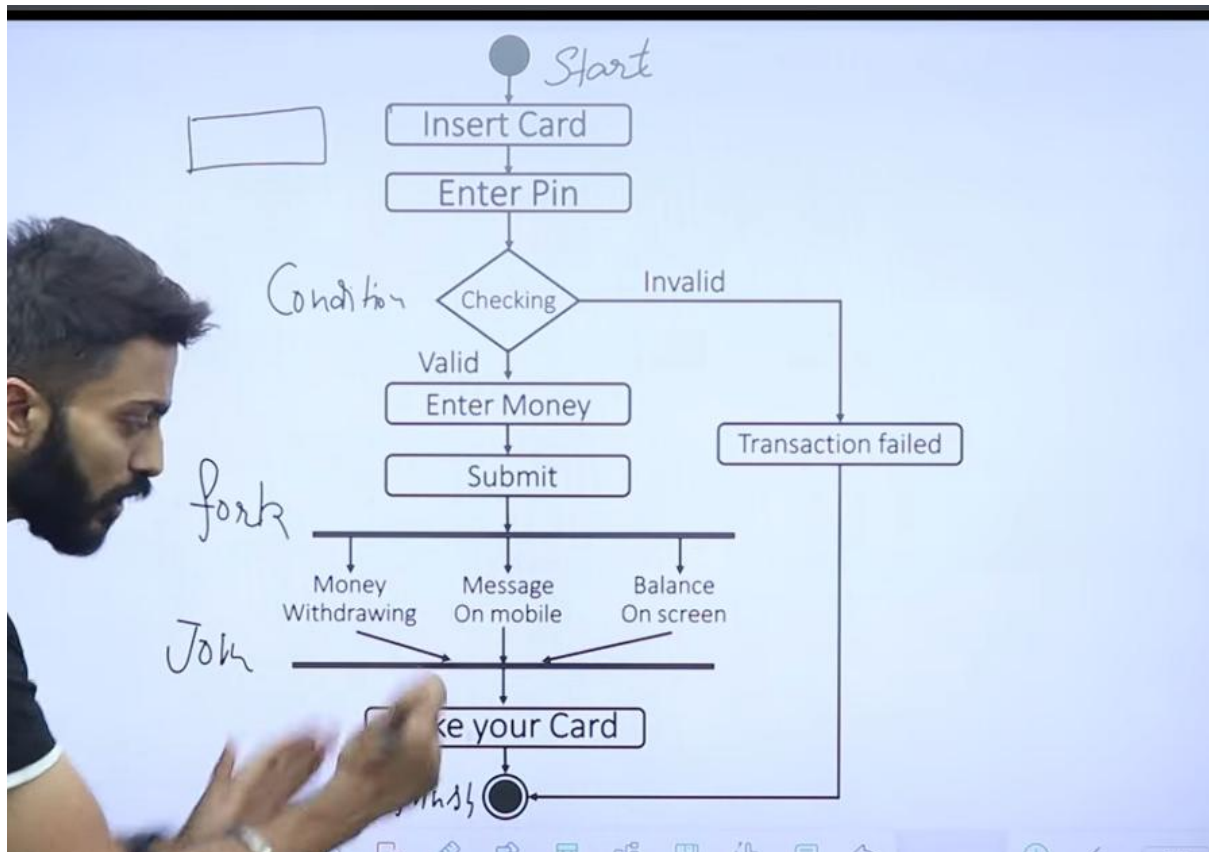
Start – black dot

Activity = rectangle – corners curved.

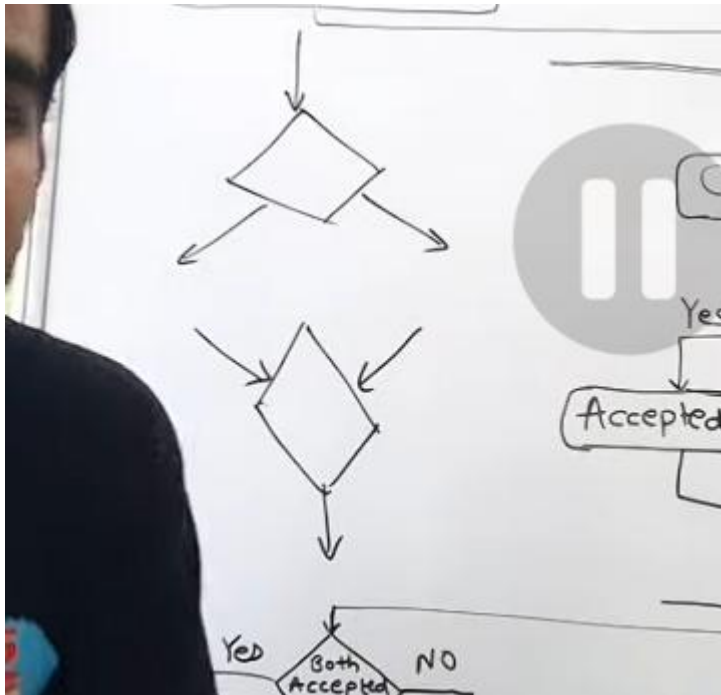
Condition ie pin checking – diamond

Finish – circle and black dot

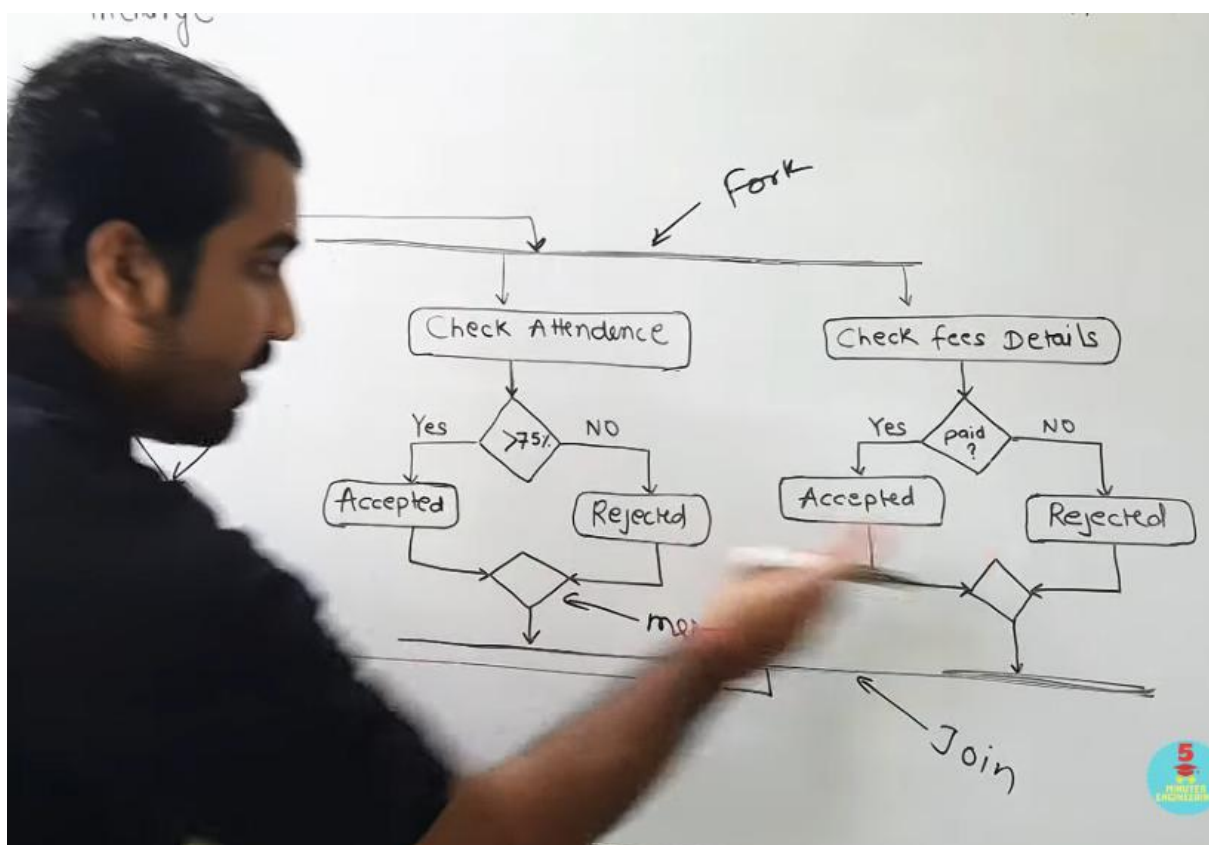
Fork = parallel processes – like after submitting the amount of money I want money withdrawing , message on mobile , all these two processes are parallel.



The output of fork – join –

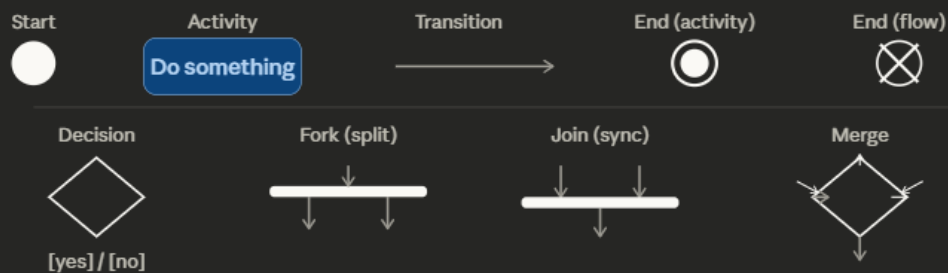


First one is output , second one is merge . merge is done when process is not in cocurrent



Here are comprehensive notes on UML Activity Diagrams, drawn from the lecture slides.

Activity diagram — notation reference



Swimlanes — partition activities by role/department

2. Transitions are the arrows. When an activity finishes, control immediately passes along the transition to the next activity.

3. Start marker — a solid filled circle. Every diagram has exactly one.

4. End markers — two types: the activity final node (filled circle inside a ring, ends the whole activity) and the flow final node (circle with an X inside, ends only that branch).

5. Decision diamond — one incoming flow, two or more outgoing flows each labelled with a guard condition like `[x>0]` or `[materials ready]`. Represents a choice based on a boolean expression.

6. Fork (splitting bar) — a thick horizontal/vertical bar with one flow in and multiple flows out. Marks the start of parallel (concurrent) activities.

7. Join (synchronisation bar) — a thick bar with multiple flows in and one flow out. All incoming flows must arrive before execution continues.

8. Merge — a diamond with multiple flows in and one out. Unlike a join, it does not wait — whichever flow arrives first passes straight through.

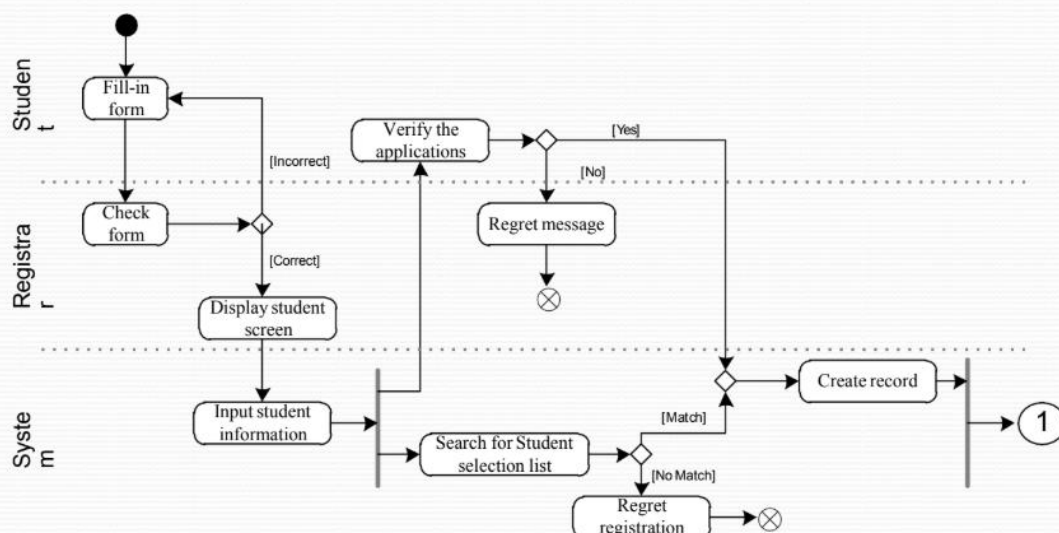
9. Swimlanes — vertical (or horizontal) partitions separated by dashed lines, each labelled with a role (Student, Registrar, System). Every activity lives in exactly one lane, but transitions can cross lanes. They make it easy to see who is responsible for what.

SEIIT System

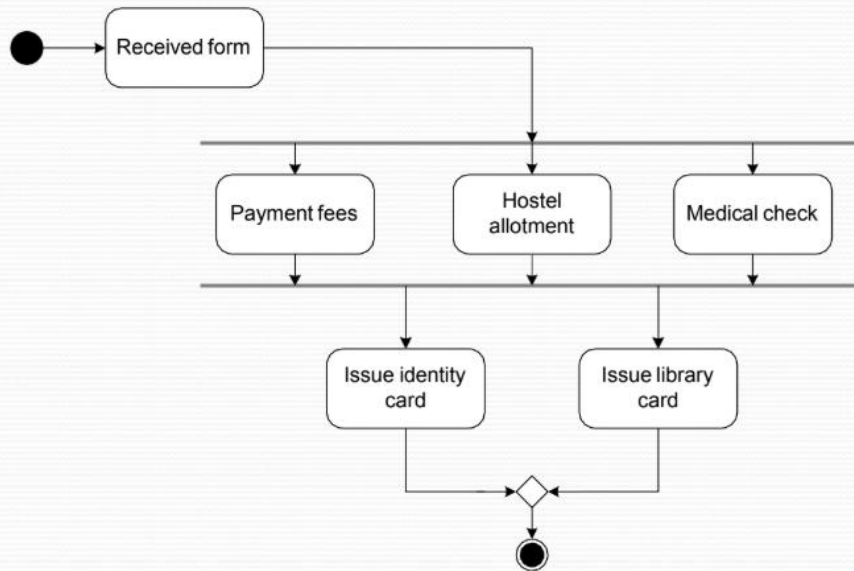
□ Here different activities are:

- Received enrollment form filled by the student
 - Registrar checks the form
 - Input data to the system
 - System authenticates the environment
- Pay fees by the student
 - Registrar checks the amount to be remitted and prepares a bill
 - System acknowledges fee receipts and prints receipt
- Hostel allotment
 - Allot hostel
 - Receive hostel charge
 - Allot room
- Medical check up
 - Create hostel record
 - Conduct medical bill
 - Enter record
- Issue library card
- Issue identity card

Activity Diagram of SEIIT with Swim Lane



Activity Diagram for SEIT



Basic Components in an Activity Diagram

Initial node

- The filled circle is the starting point of the diagram

Final node

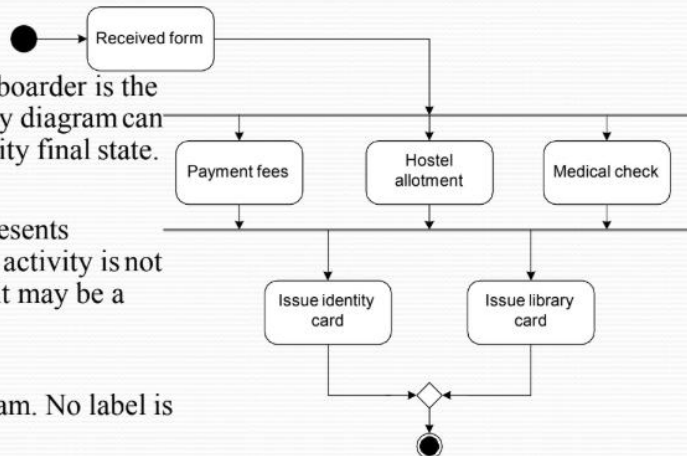
- The filled circle with a boarder is the ending point. An activity diagram can have zero or more activity final state.

Activity

- The rounded circle represents activities that occur. An activity is not necessarily a program, it may be a manual thing also

Flow/ edge

- The arrows in the diagram. No label is necessary



Basic Components in an Activity Diagram

Fork

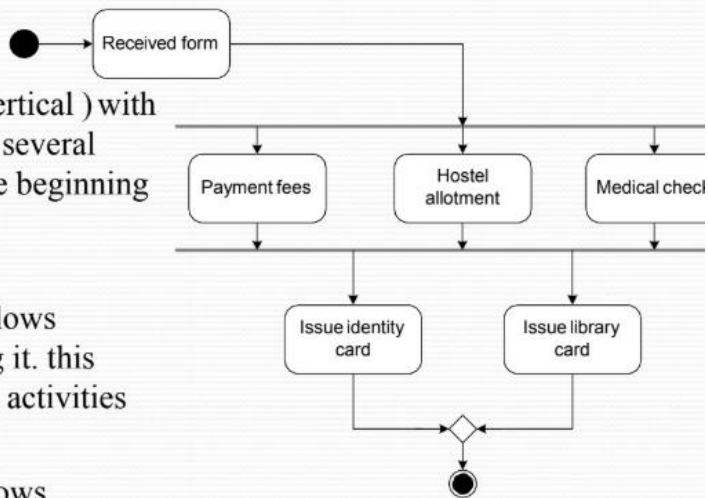
- A black bar (horizontal/vertical) with one flow going into it and several leaving it. This denotes the beginning of parallel activities

Join

- A block bar with several flows entering it and one leaving it. this denotes the end of parallel activities

Merge

- A diamond with several flows entering and one leaving. The implication is that all incoming flow to reach this point until processing continues



Decision

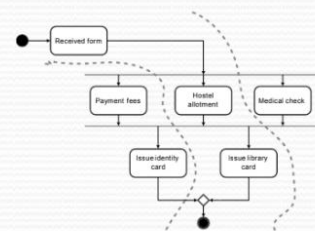
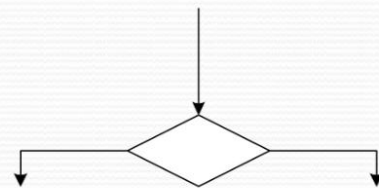
- A diamond with one flow entering and several leaving. The flow leaving includes conditions as yes/ no state

Flow final

- The circle with X through it. This indicates that Process stop at this point

Swim lane

- A partition in activity diagram by means of dashed line, called swim lane. This swim lane may be horizontal or vertical



◆ Purpose of Swimlanes in Activity Diagram (SDA)

Swimlanes are used in Unified Modeling Language activity diagrams to:

👉 Show WHO is responsible for each action

Without swimlanes:

- You only see the flow of steps

With swimlanes:

- You see **which actor/system part performs each step**

🔥 Why they matter (don't skip this)

Swimlanes help you:

- Separate responsibilities (User vs System vs Admin)
- Avoid confusion in complex processes
- Clearly map actions to roles or components

👉 In short:

Swimlanes = **clarity of responsibility**

◆ Simple Example: Login System

Swimlanes:

- User
- System

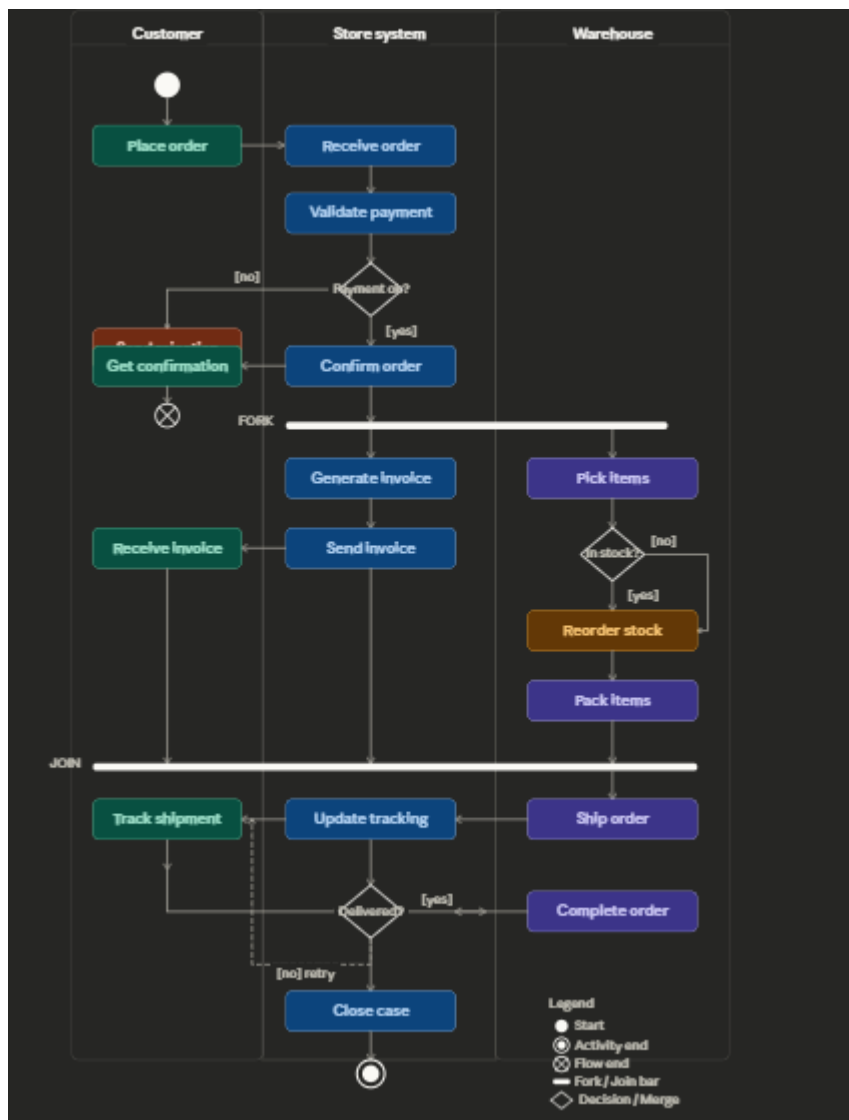
Flow:

User lane:

- Enter username & password
- Click login

System lane:

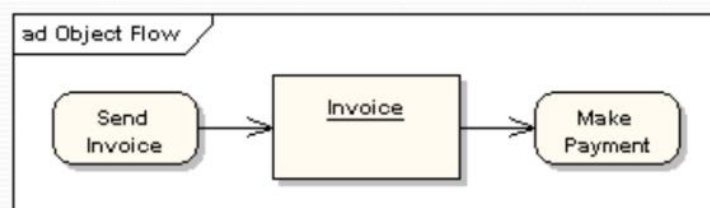
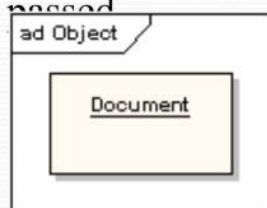
- Validate input
- Check credentials
- Show result (success/error)



Object & Object Flow — activities don't just pass control, they pass *things* (objects). An object is drawn as a plain rectangle on the arrow between two activities showing what data moves between them.

Object is shown as a rectangle

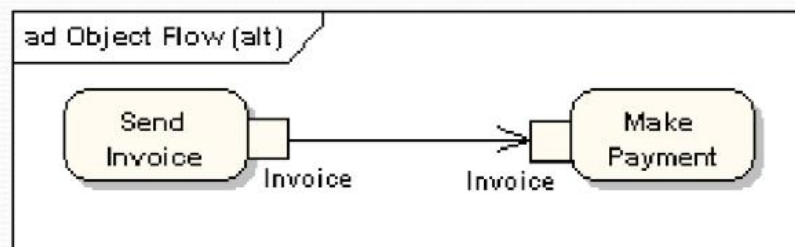
- An object flow is shown as a connector with an arrowhead denoting the direction the object is being passed



Input/Output Pins — a shorthand for object flow. Instead of a floating rectangle between activities, small squares attach directly to the edge of the activity box — output pin on the sender, input pin on the receiver.

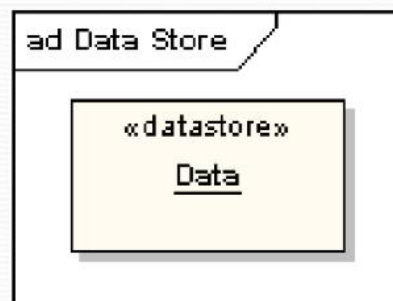
Input and Output Pin

- An object flow must have an object on at least one of its ends. A shorthand notation for the above diagram would be to use input and output pins



Data Store

- A data store is shown as an object with the «datastore» keyword



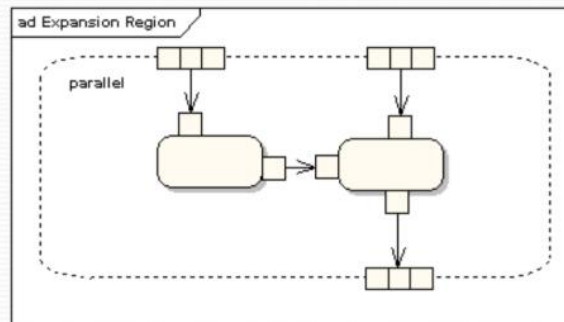
sender, input pin on the receiver.

Data Store — a persistent object tagged **“datastore”**. Unlike a regular object that just passes through, a data store *persists* — it's read from or written to (like a database).

Expansion Region — a dashed-border region that loops over a *collection* of items. Think of

Expansion Region

- Expansion regions and structured activities provide mechanisms to show the strategy for managing the synchronization issues in a real-time system.
- An expansion region is a structured activity region that executes multiple times for collection items.
- Input and output expansion nodes are drawn as a group of three / four boxes representing a multiple selection of items. The keyword *iterative*, *parallel* or *stream* is shown in the top left corner of the region



Expansion Region — a dashed-border region that loops over a *collection* of items. Think of it as a for-each loop. Modes: **parallel** (all items at once), **iterative** (one at a time), **stream** (in order).

🔥 When do we use it?

Use Expansion Region when:

- You have **multiple inputs (array/list/collection)**
- Same steps repeat for each item
- You want to show **looping clearly in UML**

◆ Real Examples (these are exam gold)

✅ 1. Processing Online Orders (Best Example)

- Input: List of products in cart
- For each product:
 - Check availability
 - Calculate price
 - Add to bill

👉 Expansion Region = "for each product"

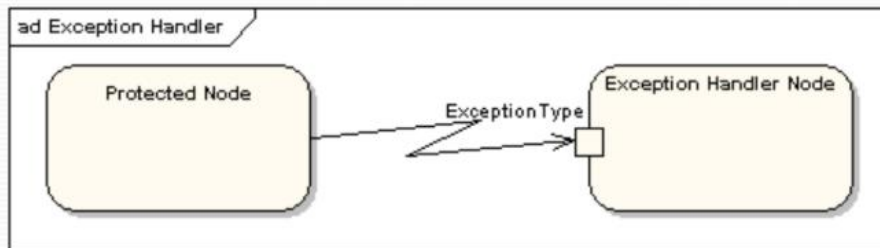
✅ 2. Grading Students

- Input: List of students
- For each student:
 - Check marks
 - Assign grade
 - Store result

Expansion Region is used in an activity diagram to represent repeated execution of actions over a collection of elements.

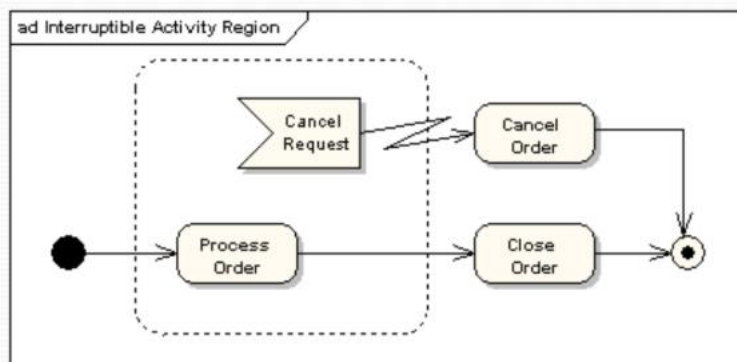
Exception Handling

- Exception Handlers can be modeled on activity diagrams as in the example below



Interruptible Activity Region

- An interruptible activity region surrounds a group of actions that can be interrupted. In the very simple example below, the Process Order action will execute until completion, when it will pass control to the Close Order action, unless a Cancel Request interrupt is received which will pass control to the Cancel Order action



Expansion Region: application-processing

The organization must accept an electronic application, review the eligibility of the application, store the application, and then confirm receipt of a valid application. The top section on the diagram shows an expansion region, a type of structured activity that handles the input and output of collections with multiple executions.

- ❑ The collection as two sets of four boxes on the top and one at the bottom showing the application, the personal information, and the verified applications.

- ❑ The italicized word in the upper-left corner shows that this expansion region allows multiple executions to the actions to occur in parallel, or concurrently.

- ❑ The action executions do not have to follow any ordering on the entering

- ❑ collections. You can also use *iterative* to show the region only allows one set of actions to execute at a time or *streaming* to show execution in the order of the collection.

- ❑ The system also relies on information from a database about the person.

- ❑ When combined with the eligibility rules for the application, shown on the diagram as a <<decisionInput>>, the organization filters out ineligible applications and sends the application on for storage.

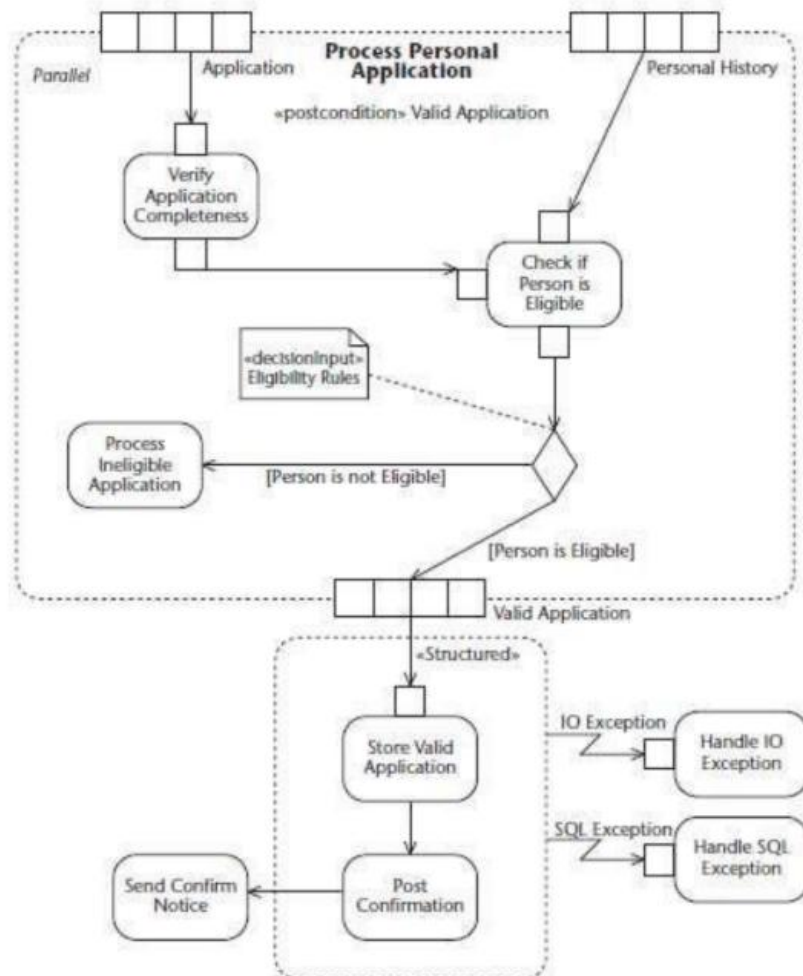


Figure 6.8 Managing applications for monetary grants.

Simple way to understand

Think:

- Decision = **if statement**
- Decision input = **data used in condition**
- Outgoing arrows = **if/else branches**

 Real code equivalent:

```
</> C
if (checkEligibility(person, rules)) {
    // Person is Eligible
} else {
    // Person is not Eligible
}
```

◆ Why dotted arrow?

👉 Dotted arrow = **input to decision (not main flow)**

- It supplies **data**
- It does NOT represent execution flow

◆ 2. Main Flow (Understand this first)

● Start flow:

- **Application + Personal History** come in (top)

⚙ Processing:

1. **Verify Application Completeness**
2. **Check if Person is Eligible**

◆ 3. Decision (IMPORTANT)

Diamond = **decision point**

- If **NOT eligible** →
👉 *Process Ineligible Application (ends there)*
- If **eligible** →
👉 *Continue forward (Valid Application)*

◆ 4. Structured Part (Lower Box)

This dashed box = **structured activity** (group of steps)

Inside it:

- Store Valid Application
- Post Confirmation
- Send Confirmation Notice

👉 This is like a **mini workflow** grouped together

◆ 5. Exception Handling (Right Side)

- IO Exception → Handle IO Exception
- SQL Exception → Handle SQL Exception

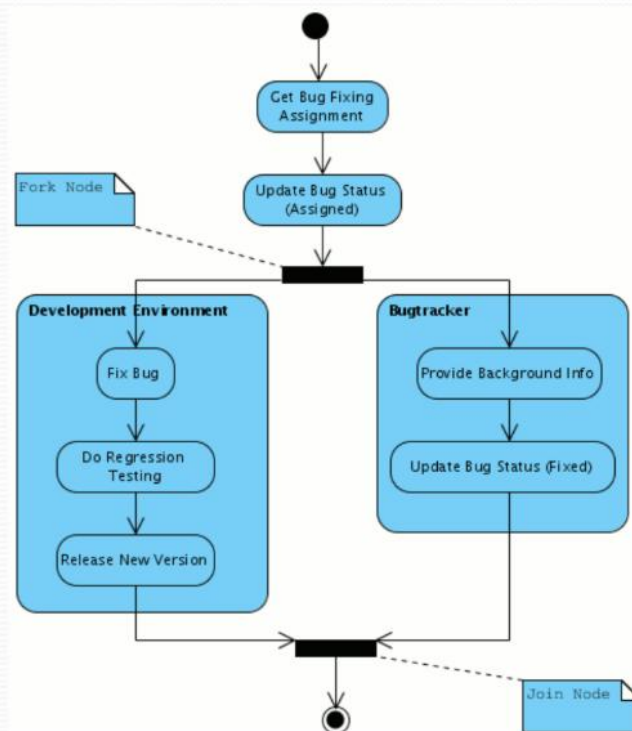
👉 Shows:

“If something goes wrong during storage → handle errors”

◆ 6. Parallel Input (Top Left “Parallel”)

- Application and Personal History come **together**
👉 This is similar to **parallel/fork** concept

Parallel Activities: Forking and Joining



Passing Objects Between Actions

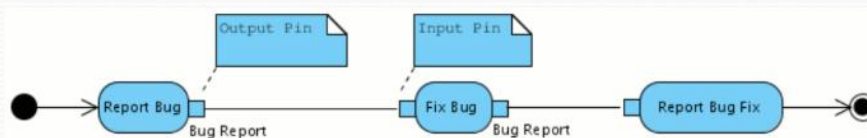
- Objects in the UML language hold information - state - that is passed between actions. Objects may represent class instances:



- Objects usually change state between actions:

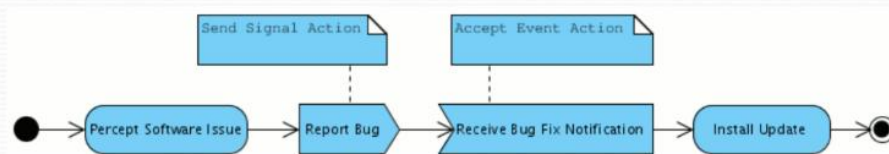


- An alternative notation shows action **input pins** and **output pins**. These emphasize that the corresponding object is *required* while **object nodes** rather emphasize the *existence* of that object:



Interacting with External Participants

- External participants may be external processes, systems, or people, interacting with an activity.
- A *receive signal* "wakes up" an action that is "sleeping".
- A *send signal* is sent to an external participant.
- In the following diagram, both a *send* and *receive signal action* are used.



- Note that the activity flow gets interrupted - gets into a wait state - until the bug fix notification is received. (If there was no *receive signal action*, however, the flow would just continue after executing the *send signal action*.)

🔥 Now YOUR diagram (step-by-step)

● Flow:

1. Start
2. Percept Software Issue
(User/system notices a bug)
3. Report Bug
 - 📄 This is a **Send Signal**
 - System sends bug report to external team

🛑 IMPORTANT PART (your question)

4. Receive Bug Fix Notification

📄 This is the **Receive Signal**

👉 What happens here?

- The system **STOPS and WAITS**
- Nothing moves forward until:
 - External system sends "bug fixed" notification

▶ After receiving signal:

5. Install Update

6. End



Simple Real-Life Analogy

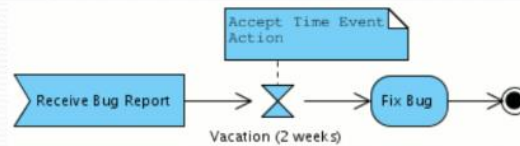
Interacting with External Participants

- An activity may also start with a *receive signal node*:

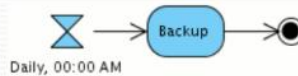


Time Events

- A *time event* models a wait period:

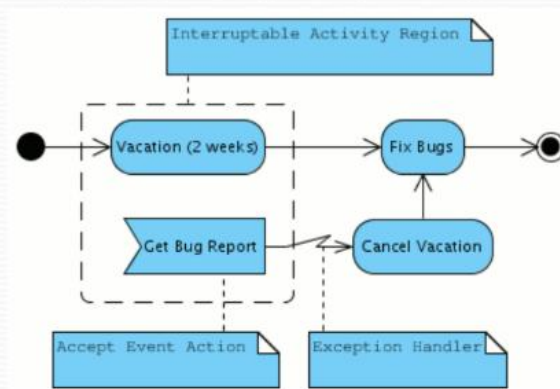


- An activity starting with a time event is **launched** periodically:

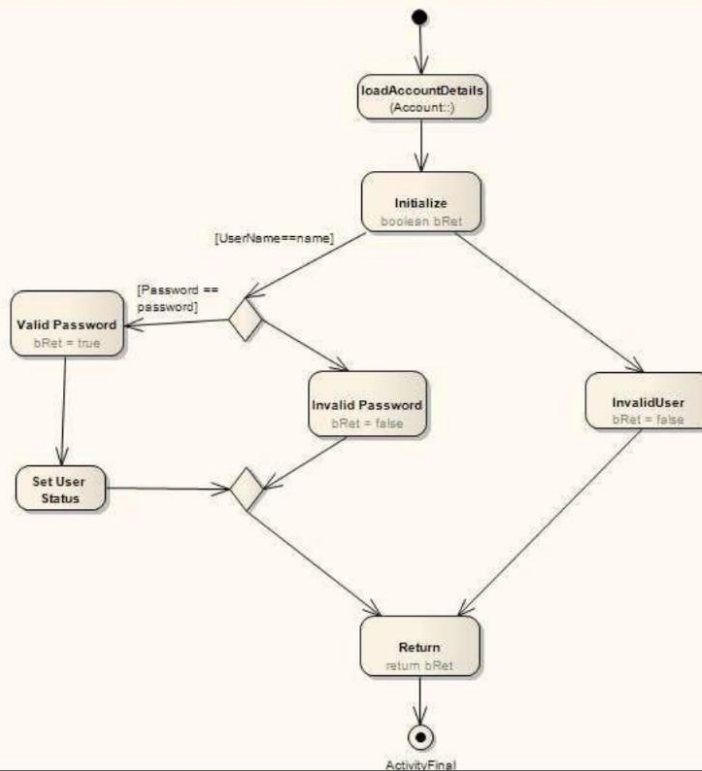


Interrupting an Activity

- Longer running processes can be interrupted by an event - an *accept event action* within an *interruptable activity region*:



Conditional Statements



Conditional Statements

```

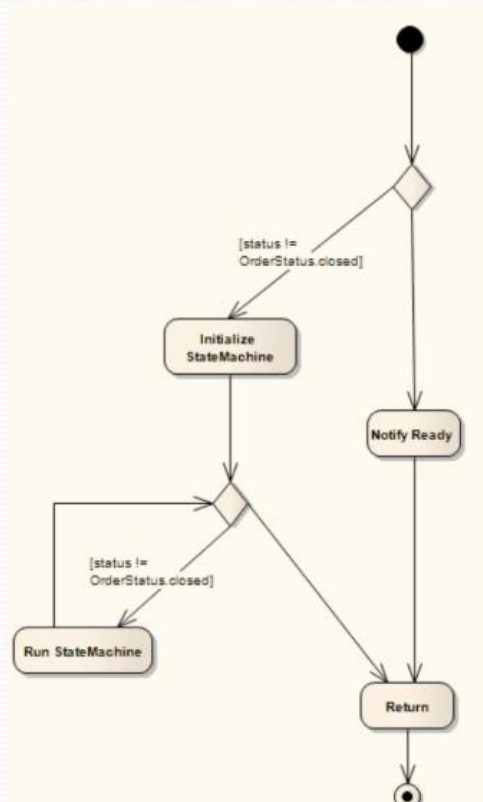
public boolean doValidateUser(String Password,String UserName)
{
    loadAccountDetails();
    boolean bRet;
    if (Username==name)
    {
        if (Password == password)
        {
            bRet = true;
            bValidUser = true;
        }
        else
        {
            bRet = false;
        }
    }
    else
    {
        bRet = false;
    }

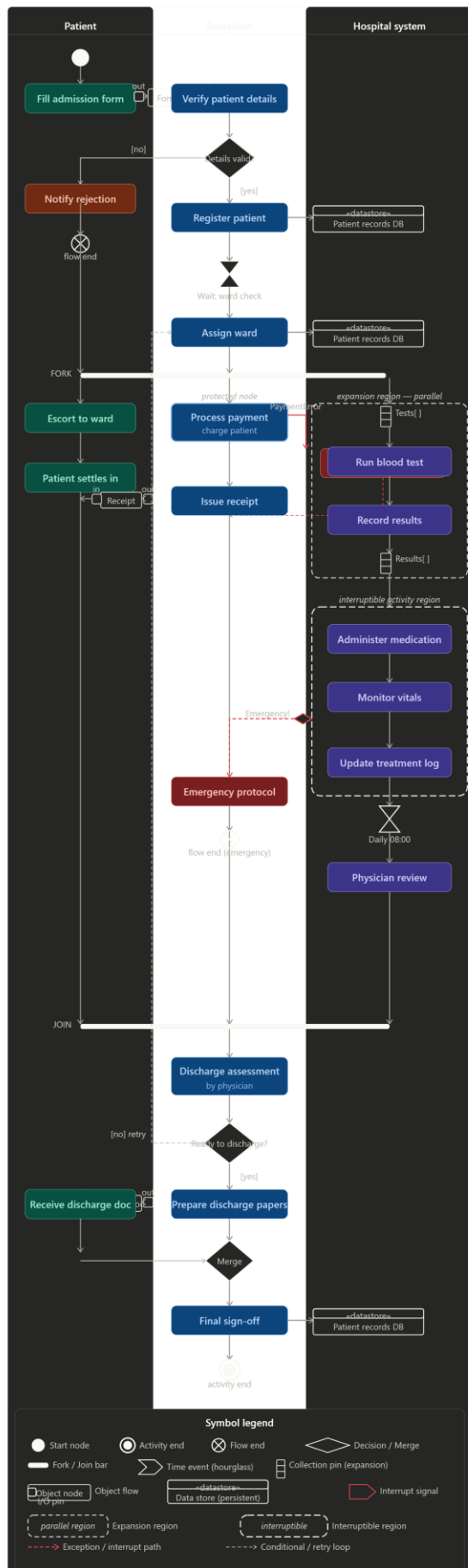
    return bRet;
}
  
```


Loops

```
public void doCheckForOutstandingOrders()
{
    if (status != closed)
    {
        initializeStateMachine();
        while (status != closed)
        {
            runStateMachine();
        }
    }
    else
    {
        //No Outstanding orders;
    }
    return;
}
```

Loops





Start node (solid black circle) — the filled dot at the very top of the Patient lane. The whole process begins when a patient arrives.

Object flow + I/O pins — when the patient fills the admission form, you see a small square on the right edge of the box (output pin labelled "out"), a floating rectangle labelled `Form` `[filled]`, and another small square on the left edge of Verify Patient Details (input pin labelled "in"). This shows the actual form object being handed from Patient to Reception — not just control, but data.

Decision diamond (Valid?) — one arrow in, two arrows out labelled `[yes]` and `[no]`. The `[no]` branch routes left, triggers Notify Rejection, and hits a **flow-end node** (circle with an X). That branch is permanently terminated — the rest of the diagram keeps going.

Data store — three times you see a double-topped rectangle labelled `"datastore" Patient records DB`. Each time an activity writes to or reads from it — Register Patient writes a new record, Assign Ward updates it, Final Sign-off closes it. This is a persistent database, not a one-time object.

Time event — wait period (hourglass after Assign Ward) — the hourglass shape between Assign Ward and Assign Ward confirms means "pause here until a ward becomes available." Control is held at this point until the condition is met, then continues.

Fork bar — the thick horizontal bar after Assign Ward splits into three simultaneous threads: Patient is escorted to the ward, Reception processes payment, and the Hospital System runs tests. All three happen in parallel.

Exception handling (Process Payment) — the payment activity has a thicker border marking it as a protected node. A red sideways arrow labelled `PaymentError` exits it. If the payment fails, flow redirects to Alert Billing Dept. The dashed red line shows the handler rejoining the normal flow below — so billing is notified but the admission process still continues.

Expansion region (parallel, Run Blood Test) — the dashed box in the Hospital System lane with stacked collection pins at the top and bottom. This represents running the same pair of activities (Run Blood Test → Record Results) for every test in a collection, all in parallel. Think of it as a for-each loop.

Interruptible activity region (Administer Medication → Monitor Vitals) — the heavier dashed rounded box. Inside it, a red pentagon labelled "Emergency!" is the interrupt signal. At any moment while these activities are running, if an emergency occurs, the entire region is abandoned and flow jumps to Emergency Protocol, which ends in another flow-end node. If no emergency occurs, the region exits normally to Update Treatment Log.

Time event — periodic trigger (hourglass, Daily 08:00) — after treatment logging, a time event fires every day at 8 AM to trigger Physician Review. Unlike the wait-period hourglass, this one has no incoming arrow — it fires on a schedule, not as a consequence of something.

Join bar — the thick bar that waits for all three threads (Patient, Reception, System) to finish before continuing. Discharge Assessment cannot start until the patient is settled, payment is done, and all tests are reviewed.

Merge diamond — after Prepare Discharge Papers, both the Reception path and the Patient path (Receive Discharge Doc) flow into a merge diamond. Unlike a join, the merge does not wait — it just passes whichever path arrives, converging the two into one flow for Final Sign-off.

Activity-final node (bull's-eye) — the filled circle inside a ring at the very bottom. The entire process ends here after the final sign-off is recorded.

□ □ □

SEQUENCE DIAGRAM

Behavioural –

How sequence diagrams differ from class and activity diagrams

These three UML diagram types serve very different purposes:

A **class diagram** is structural — it shows what *exists* in a system: the classes, their attributes, methods, and the static relationships between them (inheritance, association, composition). It's a blueprint of the system's architecture, like a map of rooms in a building. It does not show time or behaviour.

An **activity diagram** shows *workflow* — the flow of control or data through a process, including branching, parallel paths, and loops. It's more like a flowchart showing what steps happen and in what order, but it doesn't show *which object* is responsible for each step.

A **sequence diagram** is about *dynamic interaction over time*. It shows which specific objects talk to each other, what messages they send, and in what order. It brings a use case to life by showing the actual conversation between objects. Think of it as a script of a scene — you see who speaks, what they say, and when.

In short: the class diagram tells you **what** the system is made of, the activity diagram tells you **what** happens in a process, and the sequence diagram tells you **who** does what and **when**.

What is a sequence diagram?

Interaction diagram

Dynamic behaviour

A sequence diagram shows which objects interact, what messages they exchange, and in what time order. You read it top-to-bottom — earlier events are higher up.

Also called an *event diagram*. Can model sequential flow, branching, iteration, recursion, and concurrency.

Two axes

↑ Vertical = time (top = earlier)

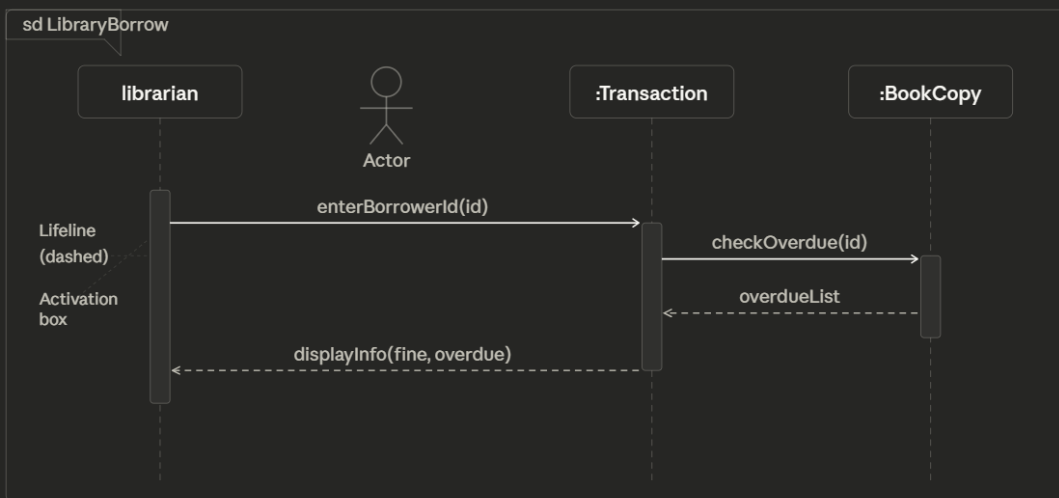
↔ Horizontal = participants (objects/actors)

Two forms

Instance form — one specific scenario, no branches or loops

Generic form — all alternatives, includes conditions & loops

Core notation at a glance



The frame (rectangle) is labeled `sd DiagramName`. Objects appear along the top. Each has a dashed vertical *lifeline*. The narrow rectangles on lifelines are *activation boxes* — they show when that object is busy. Solid arrows = sync messages. Dashed arrows = return messages.

Lifelines — types of participants

Object / class role

Shown as a rectangle with the name underlined. Format:

`instanceName : ClassName`. If only the class matters: `: ClassName`.

Example: `freshman : Student` — the Student class, instance named freshman

Actor

A stick figure. Represents a human or external system that interacts with your system but is outside it. Used at the start of a use-case-driven diagram.

Boundary

Circle with a vertical bar on the left. Represents a UI or interface object — the layer between an actor and the system. E.g. a form screen or API endpoint.

Control

Circle with an arrow on top. Represents a controller or coordinator object — manages the flow of a use case. E.g. a TransactionController.

Entity

Circle with a line underneath. Represents a persistent data object — usually maps to a database entity. E.g. a BorrowerRecord or BookCopy.

All five types share the same dashed lifeline below them. The shape at the top is just a visual hint about the participant's role in the system.



Activation box (execution occurrence)

A thin rectangle on a lifeline. Shows the period when an object is actively executing — either running its own code or waiting for another object to reply. When the box ends, control returns to the caller.

Practical rule: if object A sends a message to B and waits for a reply, A's activation box stays open while B's box is active. Once B replies, B's box closes.

Types of messages

Synchronous

Solid arrowhead (filled triangle). The sender *waits* for a reply before continuing. This is the most common message type in object-oriented systems. An activation box opens on the receiver.

Example: `ok = mayBorrow(member)` — the caller blocks until it gets a result back.

An asynchronous message requires a response before the interaction can continue. It is usually drawn using a line with a solid arrowhead pointing from one object to another.



Simple, also used for asynchronous

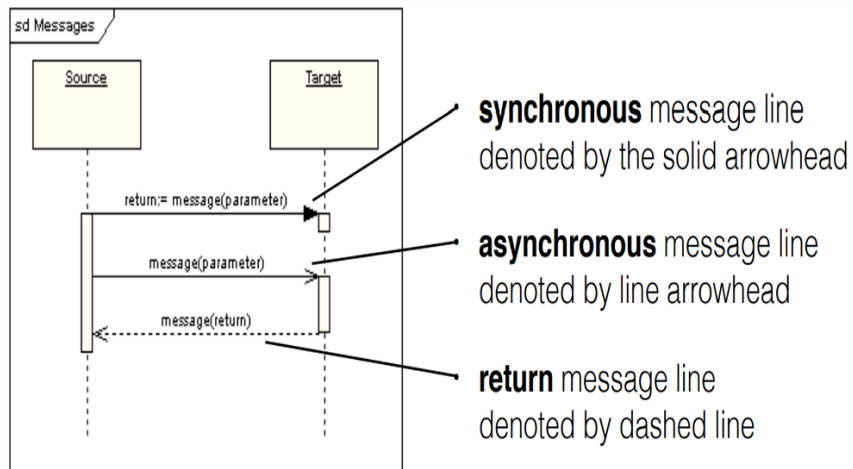


Asynchronous

■ Asynchronous message

- An asynchronous message reveals that the sending object does not wait, but continues to execute immediately after having sent the message (any result is typically sent back as an asynchronous message as well).

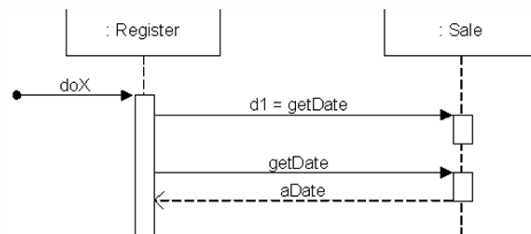
EXAMPLE



REPLY OR RETURN

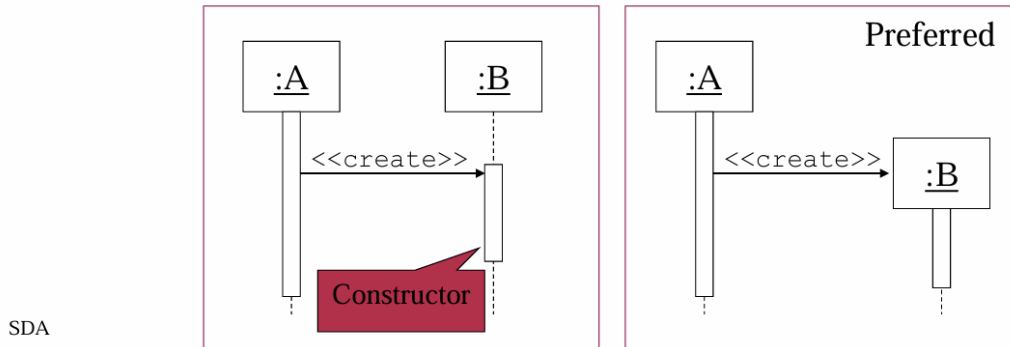
There are two ways to show return result from a message:

- Using the message syntax `returnVar=message(parameter)`
- Using a reply(or return) message line at the end of an activation bar.



TYPES OF MESSAGES

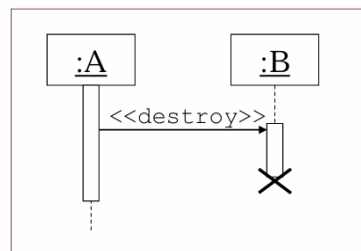
- Create Message
- An object may create another object via a **<<create>>** message.



TYPES OF MESSAGES

- An object may destroy another object via a **<<destroy>>** message.
- An object may destroy itself.
- Avoid modeling object destruction unless memory management is critical.

Objects can be terminated early using an arrow labeled "**<< destroy >>**" that points to an X. This object is removed from memory. When that object's lifeline ends, you can place an X at the end of its lifeline to denote a destruction occurrence



SDA

23

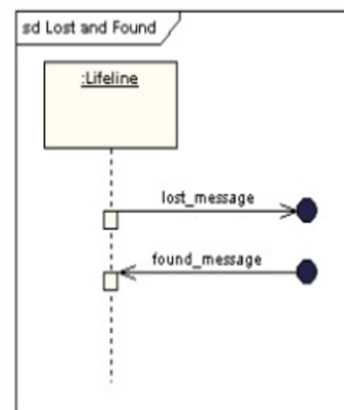
TYPES OF MESSAGES

Found Message

A message sent from an unknown recipient or from a sender not shown on the current diagram, shown by an arrow from an endpoint to a lifeline.

Lost Message

A message sent to an unknown recipient that is not shown on the diagram. It's shown by an arrow going from a lifeline to an endpoint, a filled circle.



Reply / return

Dashed line with an open arrowhead pointing back to the original caller. Shows a return value. Two ways to show return: (1) use the syntax `result = message(param)` on the call arrow itself, or (2) draw an explicit dashed return arrow at the end of the activation box.

Both styles are valid UML. Style 1 is more compact. Style 2 is more explicit about *when* the return happens.

Self / reflexive

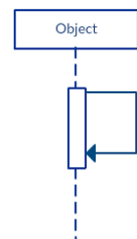
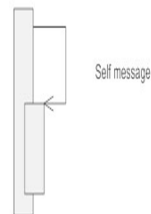
A U-shaped arrow that loops from a lifeline back to itself. Represents an object calling one of its own methods. A second, nested activation box appears on the same lifeline.

Example: a `:Register` object calling its own `calculateTotal()` method internally before returning to the UI.

TYPES OF MESSAGES

■ Self Message/Reflexive message

- A message an object sends to itself, usually shown as a U shaped arrow pointing back to itself



Create

A `<<create>>` stereotyped message arrow pointing to a new participant box. The created object's box appears mid-diagram (not at the top) — it only starts existing at the moment of creation. The preferred style points directly to the new object's header box.

Example: a `TransactionController` creates a new `Fine` object: arrow labeled `<<create>>` points to a new `:Fine` box appearing partway down the diagram.

Destroy

A `<<destroy>>` message to an object. The receiving object's lifeline ends with an X mark. An object can also destroy itself. Only model this when memory management is critical to the scenario.

Use sparingly — most high-level design diagrams don't model destruction. It becomes important in embedded systems or when modelling resource cleanup.

Found message

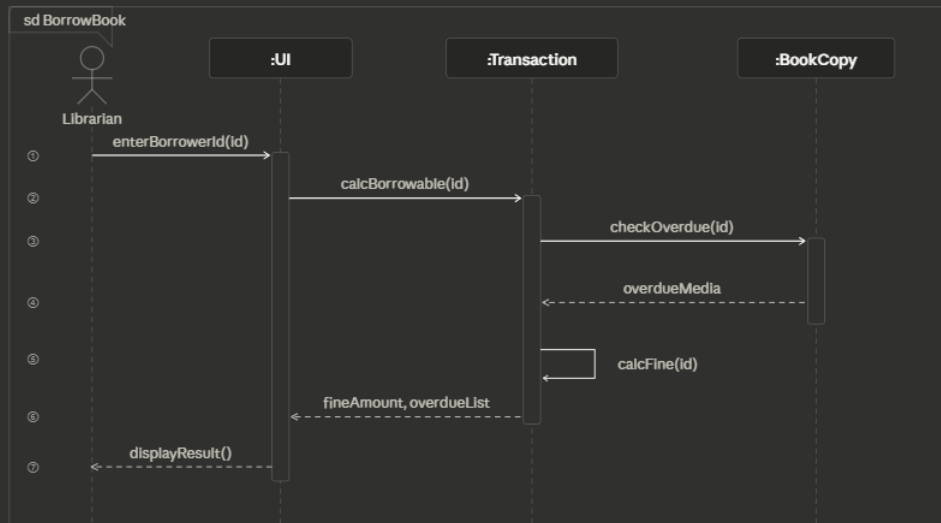
Arrow arriving at a lifeline from a filled circle endpoint. The sender is unknown or outside the diagram's scope. Used when a message enters your system from an unspecified external source.

Lost message

Arrow going from a lifeline to a filled circle endpoint. The recipient is unknown or outside scope. Used when your system sends something to an unspecified destination.

Practical example — Borrow Book use case

Here is how the "Borrow Book" scenario maps to a sequence diagram. Four participants: the librarian (actor), the UI (boundary), a Transaction object (control), and a BookCopy (entity).



① Librarian enters borrower ID into UI · ② UI calls Transaction to calculate what can be borrowed · ③ Transaction asks BookCopy for overdue media · ④ BookCopy returns the overdue list · ⑤ Transaction calls itself (self-message) to calculate fine · ⑥ Transaction returns fine amount and overdue list to UI · ⑦ UI displays everything to the librarian

Why use sequence diagrams?

Model object interactions

See exactly who calls whom, in what order. Much clearer than reading code for understanding the flow of a use case.

Verify class design

If you can't draw the sequence without adding new methods to existing classes, that's a signal your class design needs revisiting.

Assign responsibilities

Each message in the diagram becomes a method that must exist on the receiving object. Sequence diagrams directly drive what operations you add to classes.

Understand use case realisation

A use case says *what* the system does. A sequence diagram shows *how* objects collaborate to make it happen.



Participants. Along the top you place your objects. A plain object is a rectangle with its name underlined in the format *instanceName* : *ClassName*. If only the class matters, you write : *ClassName*. Beyond plain objects, UML gives you specialised shapes: the stick figure *actor* for humans or external systems, the circle-with-bar *boundary* for UI/interfaces, the circle-with-arrow *control* for coordinators, and the circle-with-underline *entity* for persistent data objects. All of them get a dashed vertical lifeline below.

Activation boxes. When an object is actually doing something — running code or waiting for a response — you show a thin narrow rectangle on its lifeline. It opens when the object receives a message and closes when it finishes and returns. Nested activation boxes occur when object A calls B while A is already active.

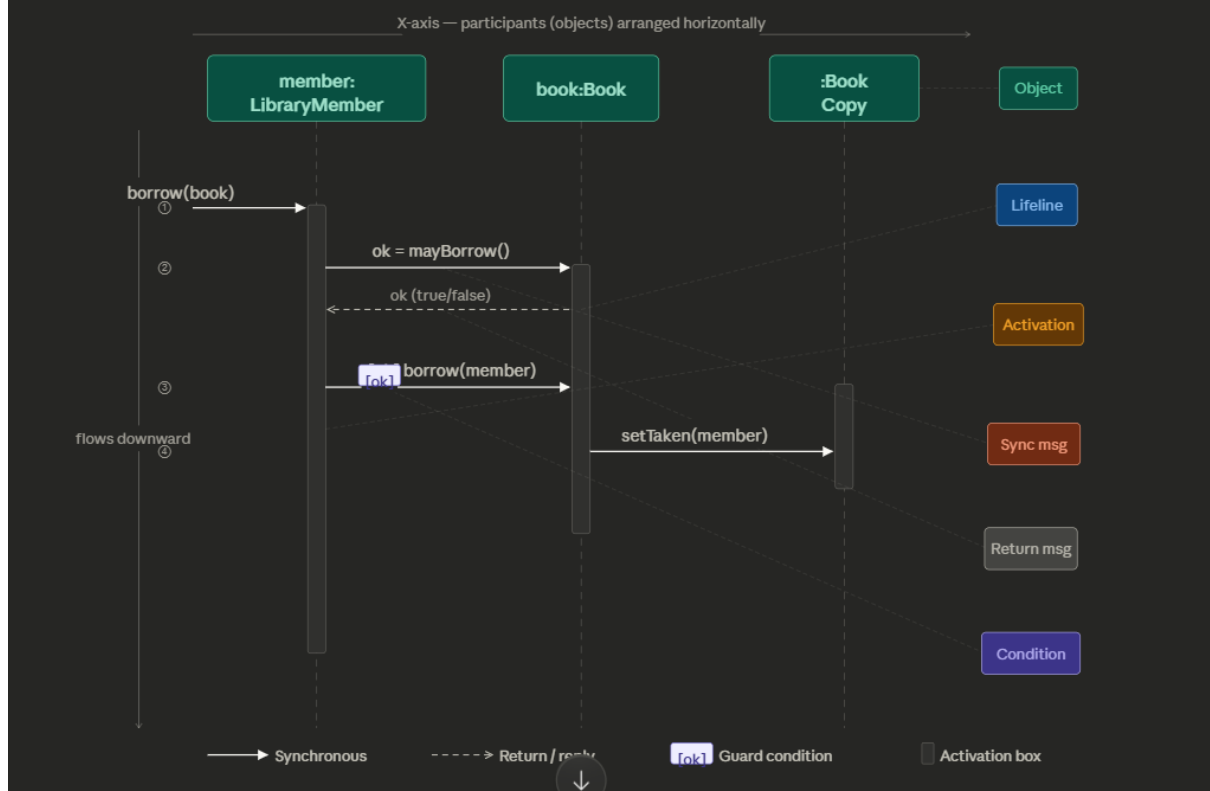
Messages are the heart of the diagram. A solid filled-arrowhead line is a synchronous message — the sender blocks and waits. An open or line arrowhead is asynchronous — sender fires and continues immediately. A dashed line with an open arrowhead going back is a return message. A U-shaped arrow looping back to the same lifeline is a self-message (an object calling its own method). A <<create>> arrow pointing to a new participant box that appears mid-diagram shows object creation. A <<destroy>> arrow ending in an X shows an object being removed from memory. A *found message* arrives from a filled-circle endpoint (unknown sender); a *lost message* departs toward one (unknown receiver).

Instance vs generic form. The instance form documents one specific, successful run-through — no branching, no loops. The generic form can include conditions like *[ok]* on arrows and loops to model all possible scenarios.

Practical implication. Every arrow in your diagram implies a method that must exist on the receiving object. If you draw *checkOverdue(id)* going to :*BookCopy*, then the *BookCopy* class must have a *checkOverdue* method. This  how sequence diagrams feed back into your class design

EXAMPLE

Let me walk you through every single element of this diagram clearly.



The three boxes at the top — the participants

Each rectangle at the top is an *object* (a participant in this scenario). They are written as `instanceName : ClassName`, both underlined — that's the UML convention to distinguish an object from a class.

- `member : LibraryMember` — a specific library member object. "member" is the variable name, "LibraryMember" is the class it belongs to.
- `book : Book` — a specific book object.
- `: BookCopy` — notice there is no instance name before the colon. This means we don't care about a specific instance — just that *some* BookCopy object is involved.

The dashed vertical lines — lifelines

The dashed line hanging down from each box is the *lifeline*. It represents that object existing over time. The further down you go, the later in time you are. The lifeline itself (when it's just dashed with no rectangle on it) means the object exists but is idle — not currently processing anything.

The arrows — messages

There are three types visible here:

A solid arrow with a filled triangle arrowhead is a *synchronous message* — the sender stops and waits for a reply before doing anything else. Every plain method call in OOP works this way.

A dashed arrow with an open arrowhead going back is a *return message* — it carries the result back to the caller. In this diagram, `mayBorrow()` returns a value `ok` (true or false) back to `member:LibraryMember`.

The message text — what each call means

Step ①: `borrow(book)` — something outside (perhaps a librarian UI, shown coming from the left edge) tells the `member` object to borrow a specific book. This kicks off the entire sequence.

Step ②: `ok = mayBorrow()` — the `member` object asks the `book` object: "am I allowed to borrow you?" The return value is stored in `ok`. This is a query — the book checks if it's available, if the member has no overdue fines, etc. The dashed arrow returns the answer.

Step ③: `[ok] borrow(member)` — this is the most important line to understand. The square brackets `[ok]` are a *guard condition*. This message is ONLY sent if the condition `ok` is true. If `ok` is false (borrowing is not allowed), this entire step is skipped. When `ok` is true, `member` tells the `book` object to actually register the borrow, passing itself (`member`) as the argument so the book knows who borrowed it.

Step ④: `setTaken(member)` — `book:Book` now calls `setTaken(member)` on `:BookCopy`. This makes sense because a Book can have multiple physical copies. The `book` object delegates to the actual physical copy (`BookCopy`) to mark itself as taken by this member.

delegates to the actual physical copy (`BookCopy`) to mark itself as taken by this member.

The X-axis and Y-axis labels

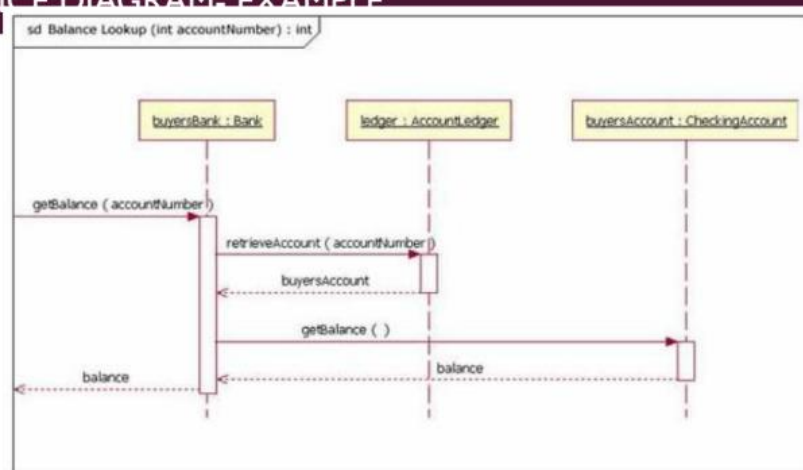
The diagram literally labels these to remind you: horizontal space = who (the different objects), vertical space = when (time flows downward). The earlier something appears in the diagram, the earlier it happens in real execution.

Reading the whole story in one sentence: A borrow request comes in → the system checks if borrowing is allowed → if yes, the book is registered to the member → and the physical copy is marked as taken. The guard condition `[ok]` is the safety check that prevents the last two steps from running if the member is not eligible.

EXAMPLE: BALANCE LOOKUP USE CASE

- Customer asks the bank for his account balance.
- Bank verifies his account number from the account ledger.
- Bank then sends message to checkaccount for getting the balance.
- The bank informs the customer about balance.

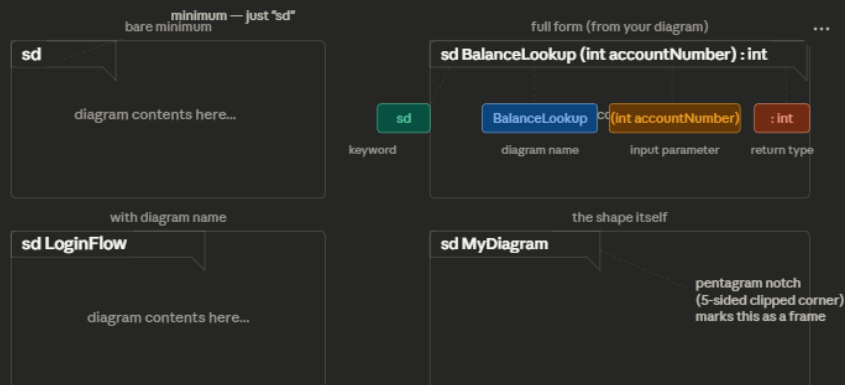
SEQUENCE DIAGRAM- EXAMPLE



The full story in sequence

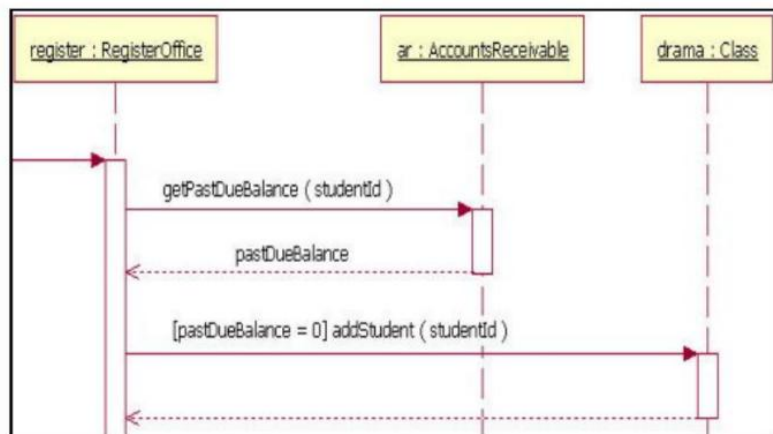
- ① Customer sends `getBalance(accountNumber)` to the Bank — the request enters the system.
- ② Bank asks the Ledger to `retrieveAccount(accountNumber)` — the Ledger is a directory that maps account numbers to actual account objects.
- ③ Ledger returns `buyersAccount` — this is not a balance, it's a reference to the `CheckingAccount` object itself. The Ledger's job is done here; its activation box closes.
- ④ Bank now calls `getBalance()` directly on `buyersAccount:CheckingAccount`, completely skipping the Ledger. The Bank uses the object reference it just got from the Ledger.
- ⑤ `CheckingAccount` returns the actual `balance` number to the Bank.
- ⑥ Bank sends `balance` back to the Customer. The use case is complete.

The **sd** label in the top-left corner of the frame is the **diagram identifier**. Let me break it down completely.

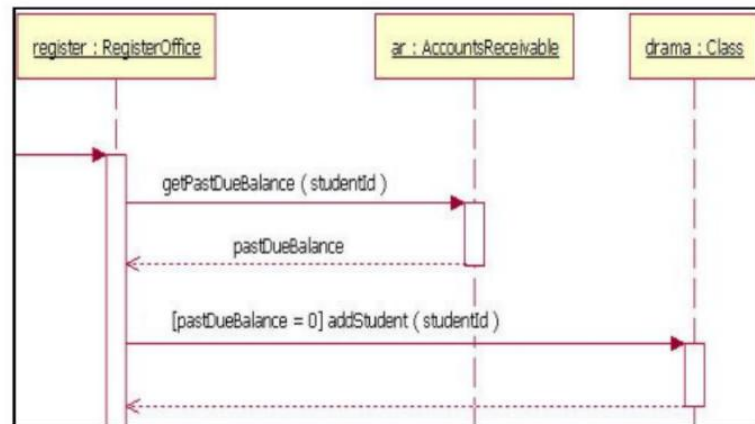


sd stands for "sequence diagram" — it's a UML keyword, nothing more. It's just UML's way of saying "this frame contains"

GUARDS

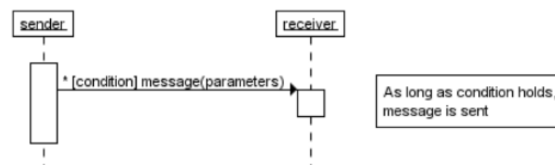


GUARDS



REPEATED INTERACTION

- When a message is prefixed with an asterisk (the '*'-symbol), it means that the message is sent repeatedly. A guard indicates the condition that determines whether or not the message should be sent (again). As long as the condition holds, the message is repeated.



SDA

10

- Condition means as long as this condition holds , implement the thing
- Think of it exactly like a while loop in code — the * is the loop, the [condition] is the while condition.

Scenario: ATM withdrawing cash

Imagine a user is withdrawing money, and the ATM checks if the entered PIN is correct — it keeps asking until the condition is met or attempts run out.

```
* [attempts < 3] validatePIN(enteredPIN)
```

In a sequence diagram it would look like:

```
sequenceDiagram
    participant User
    participant ATM
    Note over User, ATM: * [attempts < 3]
    User->>ATM: validatePIN(pin)
    ATM-->>User: "Wrong PIN"
    Note over User, ATM: * [attempts < 3] ← repeats again
    User->>ATM: validatePIN(pin)
```

Breaking it down:

- ***** → this message is sent repeatedly
- **[attempts < 3]** → this is the guard/condition — keep going as long as this is true
- **validatePIN(pin)** → the message with its parameter
- Once attempts hit 3 (condition fails), the loop stops

Another quick example — loading items from a cart:

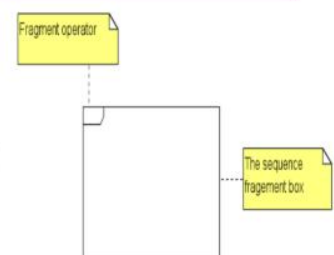
```
* [items remaining] processItem(item)
```

The system keeps calling **processItem()** for each item until no items remain.

Think of it exactly like a **while loop** in code — the ***** is the loop, the **[condition]** is the while condition.

FRAGMENTS

- Sequence diagrams can be broken up into chunks called fragments or combined fragments.
- **Manage complex interactions with sequence fragments**
- It is used to show complex interactions such as alternative flows and loops in a more structured way. On the top left corner of the fragment sits an operator. This — the fragment operator — specifies what sort of a fragment it is.
 - Fragment types: ref, loop, break, alt, opt, parallel
- Sequence fragments make it easier to create and maintain accurate sequence diagrams



LOOPS

- A repetition or loop within a sequence diagram is depicted as a frame.
- In the frame's name box the text "loop" is placed.
- Loops are designated by placing the interaction operator keyword loop in the pentagram. Textual syntax of the loop operand is "loop ['(' <minint> [, <maxint>] ')']".
- Inside the frame's content area the loop's guard is placed towards the top left corner, on top of a lifeline.

A SIMPLE EXAMPLE OF TWO OPERATIONS BEING REPEATED FIVE TIMES.

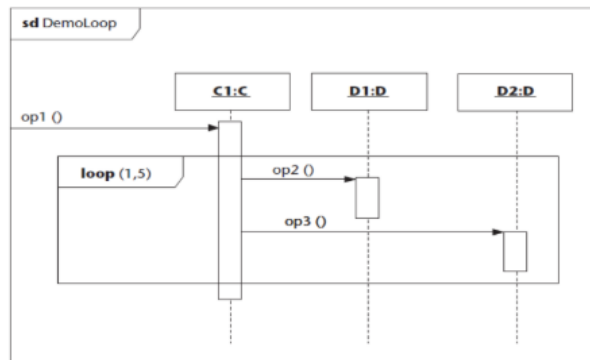
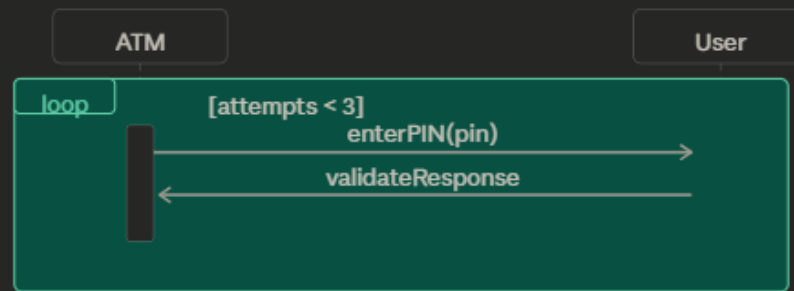
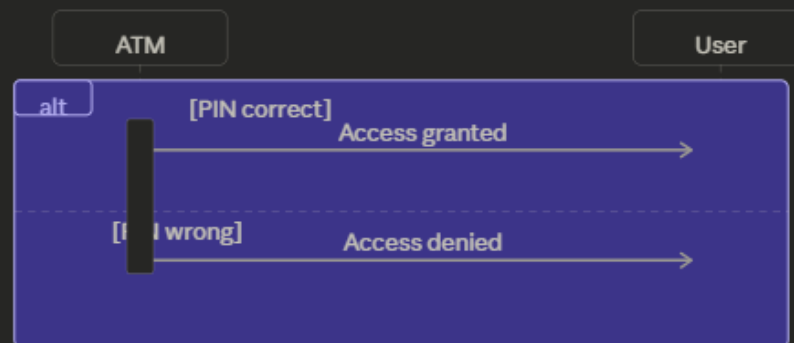


Figure 5.31 Iteration expressed with a loop operand.

1. loop — repeat while condition holds



2. alt — if / else (alternatives)



Opt

indicate looping n times.

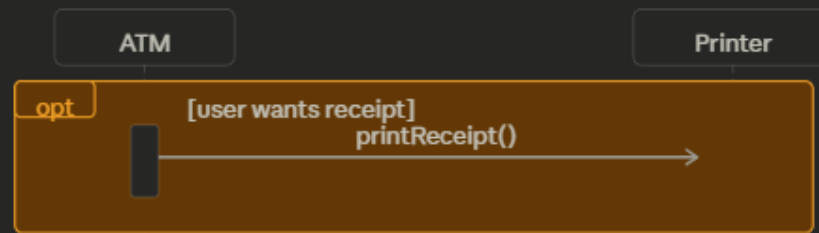
Optional fragment that execute if the guard is true.

par

Parallel fragments that execute in parallel.

- If -> (opt) [condition]
- if/else -> (alt) [condition],
separated by horizontal dashed line
- loop -> (loop) [condition or
items to loop over]

3. opt — optional block (if, no else)



The key practical difference:

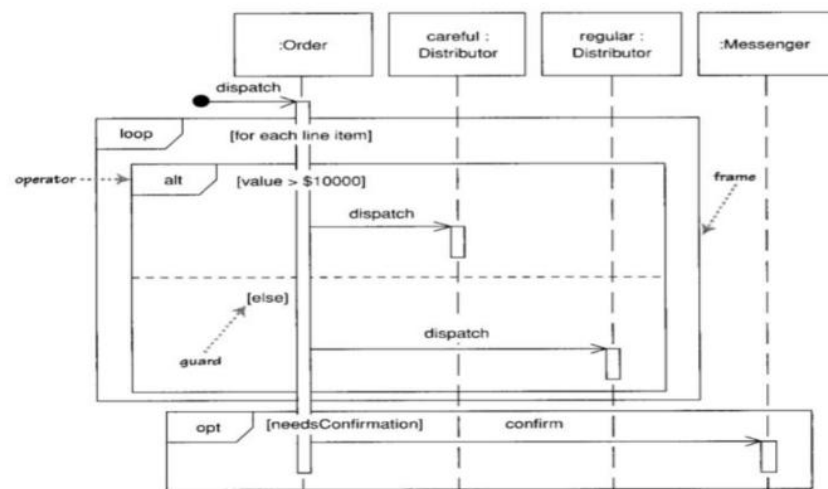
If only one message repeats → * is fine and cleaner.

If multiple messages repeat together as a group → you need loop, because * can only sit on a single arrow and can't group several messages into one repeating block.

Think of it like:

	*	loop
Scope	One arrow	Multiple messages
Visual	Label on arrow	Box around a region
Use	Quick/simple	Complex/multi-step

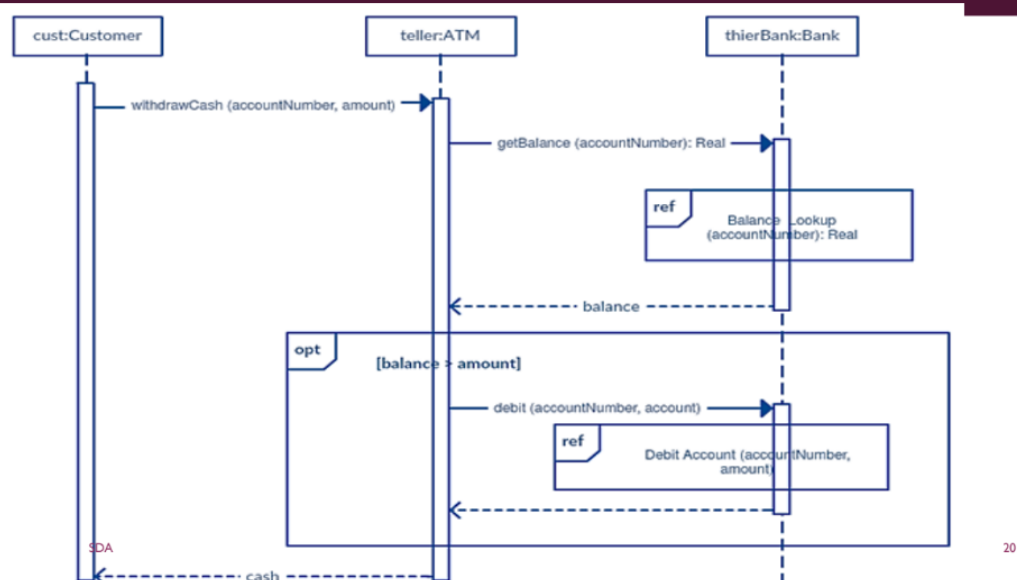
- When an order is received for dispatching, each item in the order is checked for its value. If the value is greater than 10000, it is sent to express delivery dispatcher other wise it is sent to regular dispatcher.
- If the customer has asked for confirmation, then notification department is asked to send confirmation.



REFERENCE FRAGMENT

- You can use the **ref** fragment to manage the size of large sequence diagrams. It allows you to reuse part of one sequence diagram in another, or in other words, you can reference part of a diagram in another diagram using the ref fragment.
- To specify the reference fragment, you have to mention 'ref' in the name box of the frame and the name of the sequence diagram that is being referred to inside the frame.

REFERENCE FRAGMENT- EXAMPLE



The diagram has two **ref** fragments:

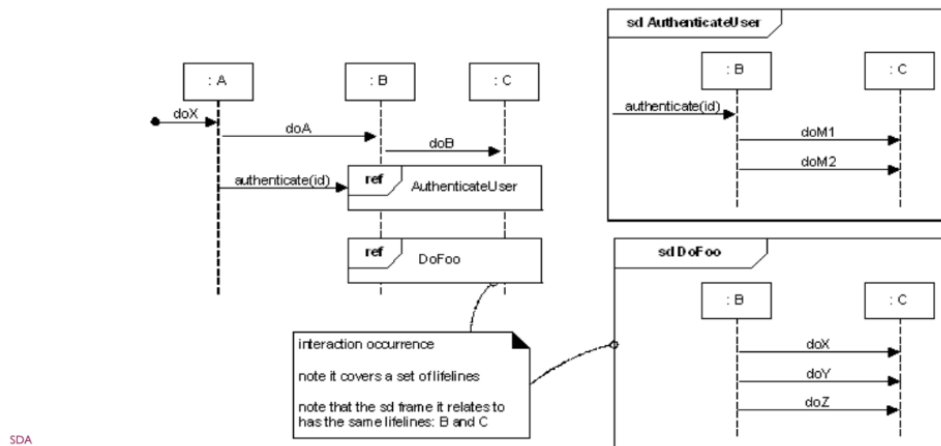
ref 1 — "Balance Lookup (accountNumber): Real"

- Triggered when ATM calls **getBalance(accountNumber)** on the Bank
- Instead of drawing all the internal steps of how the bank looks up a balance (query DB, check account, return value etc.), they just write **ref: Balance Lookup** — meaning "go look at that separate diagram for these details"
- The Bank then returns **balance** back to ATM

ref 2 — "Debit Account (accountNumber, amount)"

- Triggered inside the **opt** fragment (only runs if **balance > amount**)
- When ATM calls **debit(accountNumber, account)** on the Bank, again the internal debit steps are hidden behind a **ref**
- Meaning "the full debit process is documented in a separate diagram"

REFERENCE FRAGMENT- EXAMPLE

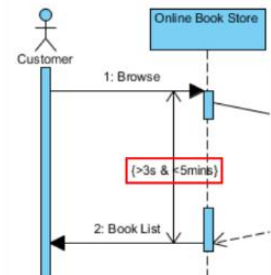


SDA

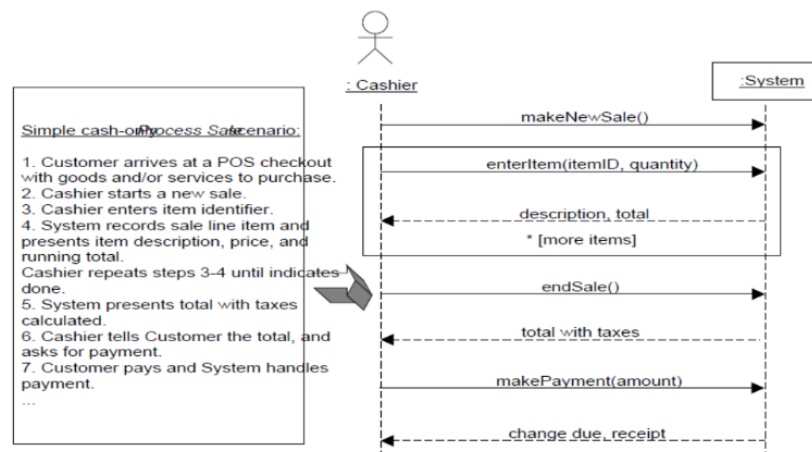
21

SPECIFYING TIMING REQUIREMENTS

- When modeling a real-time system, or even a time-bound business process, it can be important to consider the length of time it takes to perform actions.
 - Assume you need to specify the time limit between Browse message and Book List message, you therefore, have to add duration constraint between them.
 - For example, it should take more than 3 seconds but less than 5 minutes.
- You can enter $> 3s$ & $< 5mins$.



SYSTEM SEQUENCE DIAGRAM (SSD)



SDA

29

SSDs are derived from use cases.

Get Plus x

◆ 3. How does it ensure BOTH step 3 and 4 repeat?

Because in sequence diagrams:

- A **request + response pair** is treated as one interaction cycle
- The ***** applies to the **entire exchange**, not just one arrow

So internally it means:

```
LOOP:  
  Cashier → System: enterItem()  
  System → Cashier: description, total
```



◆ 4. Why is there no loop frame here?

You're talking about something like:

```
loop [more items]  
  enterItem()  
  return total
```



✓ Answer: It's optional in SSDs

There are **two styles**:

✓ Style 1 (Used here — simple SSD style)

- Uses ***** and **[condition]**
- Cleaner and faster
- Common in **System Sequence Diagrams**



✓ Style 2 (More formal UML)

- Uses a **loop frame box**

◆ 5. Why prefer ***** instead of loop frame here?

Because SSDs focus on:

- **System-level interactions**
- Not detailed control structures

👉 So they keep it **simple and readable**

🔥 Final Understanding

- ***** = repetition (loop)
- It applies to the **whole interaction (step 3 + 4)**
- Loop frame is **not required**, just a more formal alternative

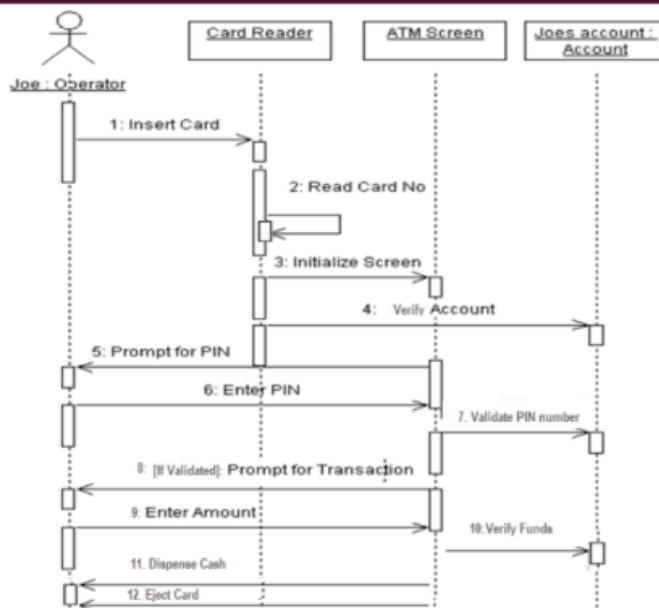
EXAMPLE

YOUR TURN: EXAMPLE FOR WITHDRAW

- Joe withdraws \$20 from the ATM (flow of events)
 - The process begins when Joe inserts his card into the card reader. The card reader reads the number on Joe's card, then tells the ATM screen to initialize itself
 - The ATM verifies the card against account and prompts Joe for his PIN.
 - Joe enters PIN.
 - Joe's PIN is validated and the ATM prompts him for a transaction
 - Joe selects Withdraw Money
 - The ATM prompts Joe for an amount.
 - Joe enters \$ 20.
 - The ATM verifies that Joe's account has sufficient funds and subtracts \$ 20 from his account.
 - The ATM dispenses \$ 20 and ejects Joe's card

SDA

34



35

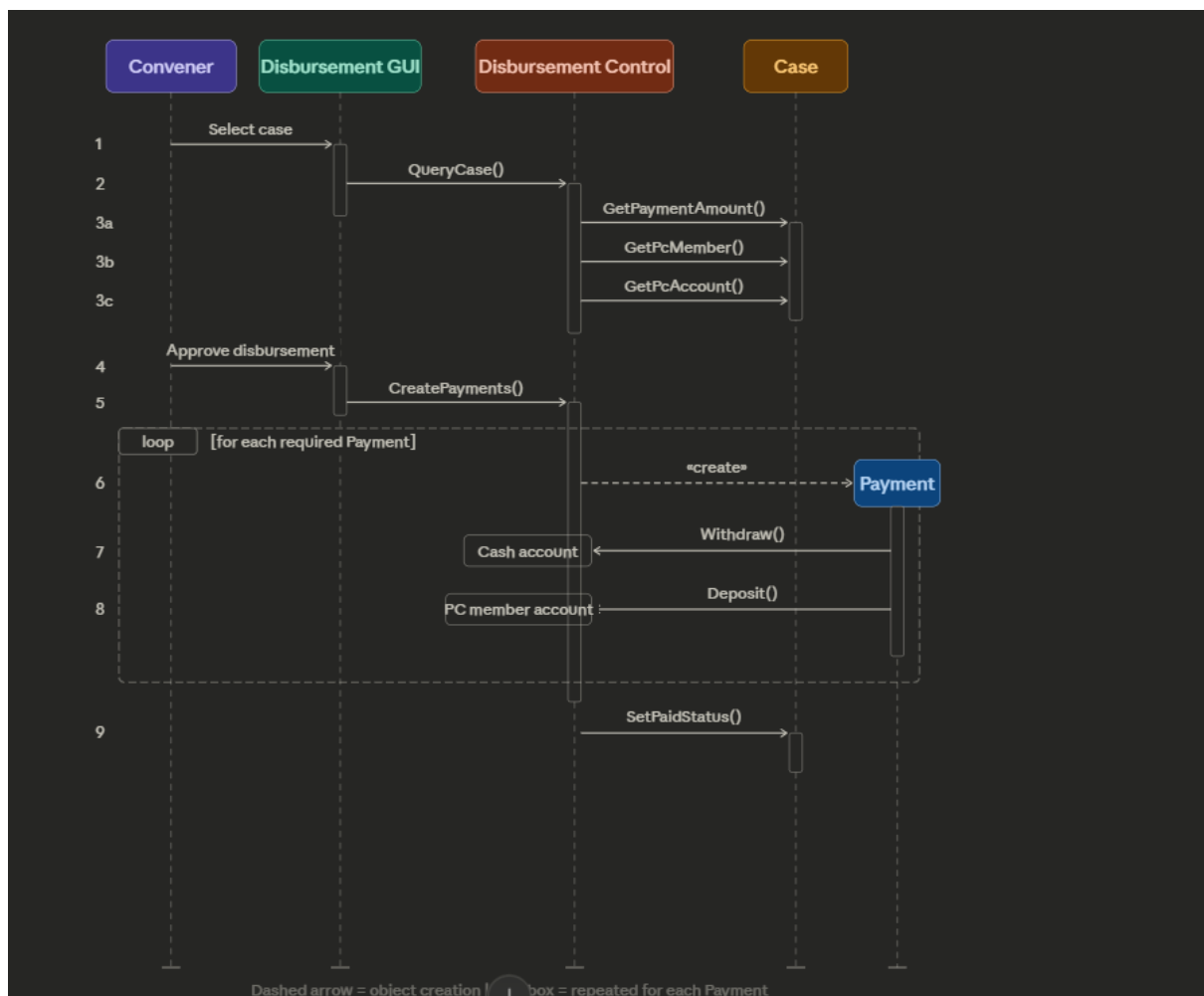
EXAMPLE

Your Turn

- The convener selects a case on the Disbursement GUI (graphical user interface) screen.
- The Disbursement GUI sends the message `QueryCase()` to the Disbursement Control object, requesting it to query payment-related details about the case.
- The Disbursement Control object services this request by passing a number of messages to the Case object. These include `GetPaymentAmount()`, `GetPcMember()`, and `GetPcAccount()`. These are requests to retrieve payment and Peace Committee member information relevant to the case.
- The convener approves the disbursement for the case.
- The GUI responds to the approval by sending the message `CreatePayments()` to the Disbursement Control object.
- The Disbursement Control object responds by sending a `Create()` message to each required Payment object.
- The Payment object sends a `Withdraw()` message to the cash account and a `Deposit()` message to the Peace Committee member account.
- The Disbursement Control object finishes the process by sending the message `SetPaidStatus()` to the Case object to indicate that payments have been made.

36

SDA



4+1 ARCHITECTURAL MODELLING

4+1 Architectural View Model

① Logical view :-

- Static Modeling view
- deals with functionalities
- Sequence, Class and Communication diagrams

② Development view :-

- Component design view
- developer's view
- Component & Package diagram
- Subsystem and system development.

③ Process view :-

- Task view
- deals with runtime aspects of system
- Scalability & Performance
- Run time description of system structure.
- Activity diagram

④ Physical view :-

- deployment of system
- Tools & environment of system
- System engineer's point of view
- Deployment diagram

④+1 Scenario :-

- Use cases

PART 2: WHAT is 4+1 View Model?

Developed by: Philippe Kruchten

◆ Core Idea

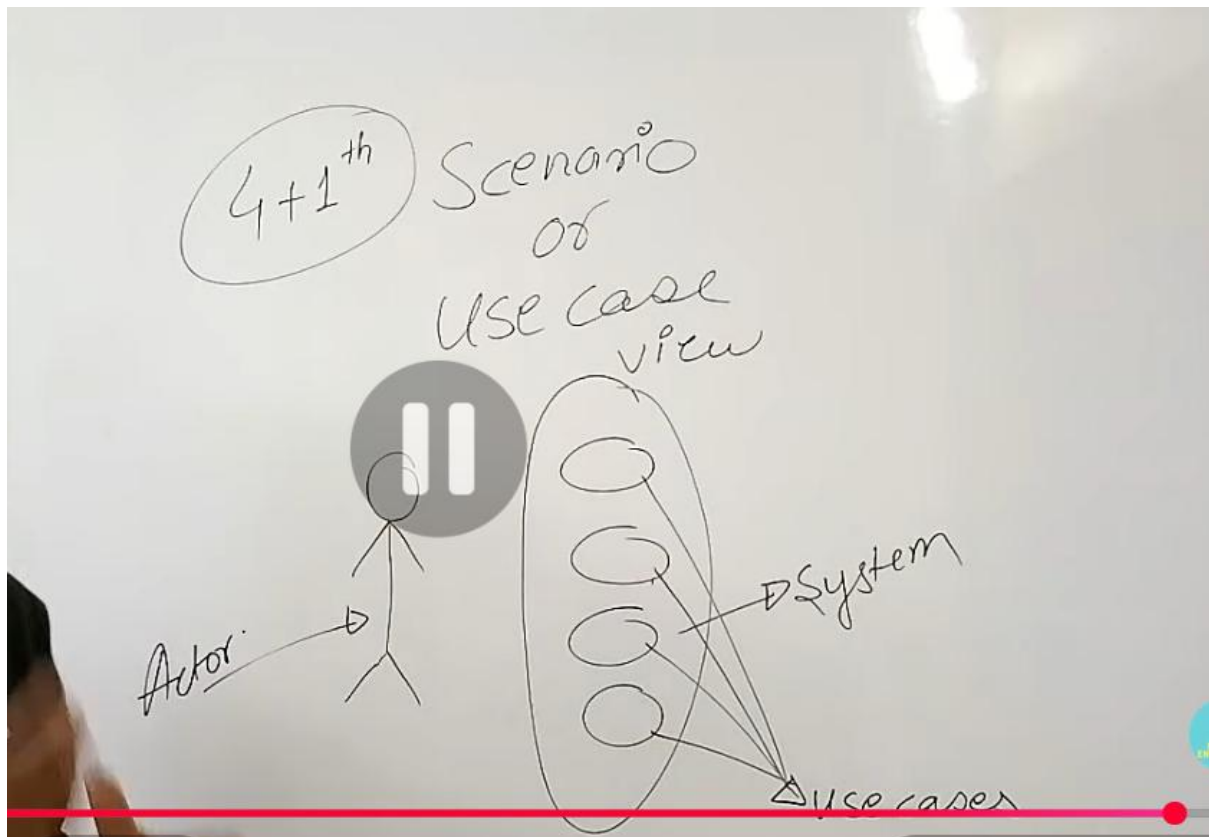
Instead of ONE big model:

👉 Split system into 5 views

4 views + 1 scenario = 4+1

◆ The 5 Views

View	Focus	Who uses it
Logical	What system does	Users
Development	Code structure	Developers
Process	Runtime behavior	Engineers
Physical	Deployment	System engineers
+1 Scenario	Real use cases	Everyone



● PART 3: WHY 4+1 is used

From your slides:

✓ 1. Handles complexity

Big systems → break into smaller views

✓ 2. Different stakeholders

Each person sees only what they need

✓ 3. Parallel development

All views are built at the same time

LOGICAL VIEW:

Logical view focuses on the functional part of the system . WHAT the system should provide in terms of services

● PART 4: THE 4 VIEWS (DETAILED)

Now the real understanding starts.

● 1. LOGICAL VIEW (What system DOES)

◆ Focus:

- Functionality
- Business logic

◆ Uses:

- Class diagrams
- Sequence diagrams
- State diagrams

◆ Example (ATM)

Classes:

- Account
- Customer
- Transaction

👉 This view answers:

"What are the main parts of the system?"



Logical view – what are the main parts of system

It uses class diagrams , sequence diagrams and state diagrams .

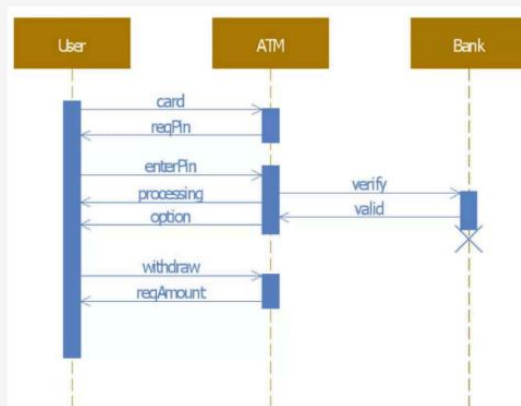
and sometimes sequence diagrams.

• Example:

- In an ATM system, classes like Account, Customer, and Transaction define the logical structure.

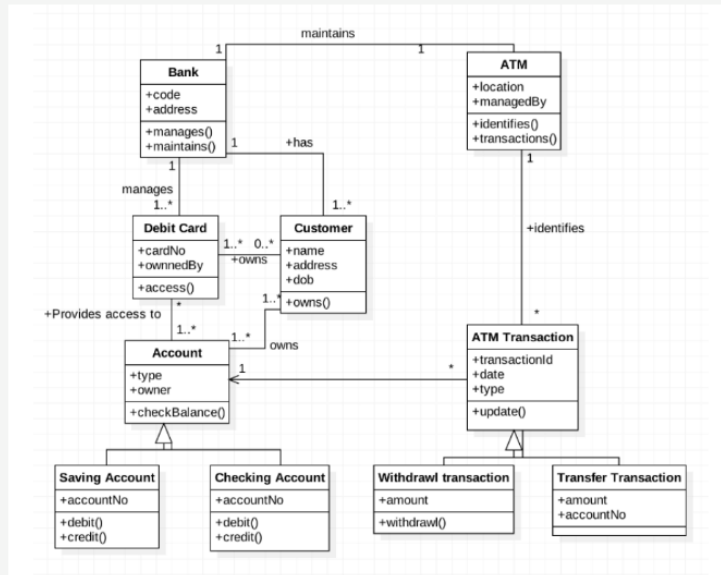
20

SEQUENCE DIAGRAM ATM



21

CLASS DIAGRAM ATM



2

PROCESS VIEW:

Process view – how system behaves at runtime . it focuses on non functional requirements like concurrency , performance . we see how processes are organized and communicate

2. PROCESS VIEW (How system RUNS)

◆ Focus:

- Runtime behavior
- Performance
- Concurrency

◆ Example:

Online ticket system:

- One process → login
- One process → payment
- One process → seat booking

👉 All run **together**

◆ Answers:

- How many processes?
- How they communicate?
- Speed / performance?

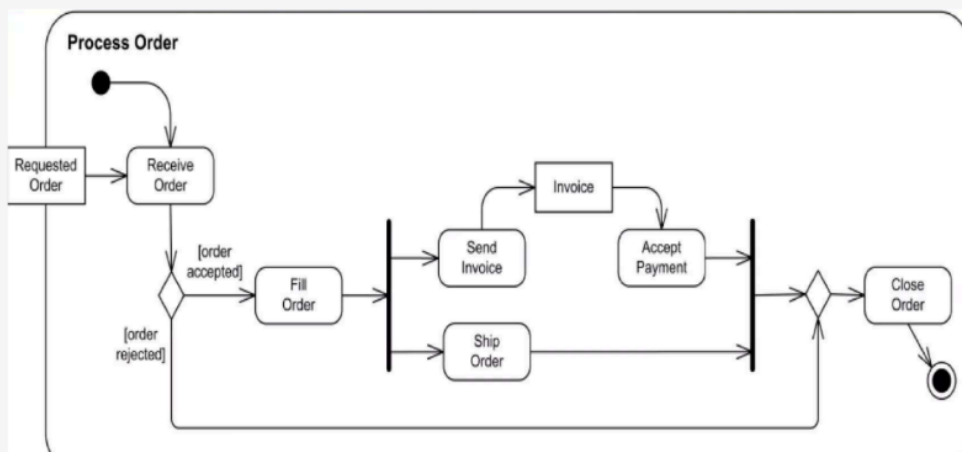


USES ACTIVITY DIAGRAM

- Example:
 - In an online ticketing system, different processes might handle user sessions, seat allocation, and payment verification – all concurrently.

23

PROCESS VIEW EXAMPLE



DEVELOPMENT VIEW:

Development view – focuses on code structure , components and modules. (how code is organized)

USES COMPONENT DIAGRAMS

3. DEVELOPMENT VIEW (How CODE is organized)

Also called:

Implementation View

◆ Focus:

- Code structure
- Modules
- Components

◆ Uses:

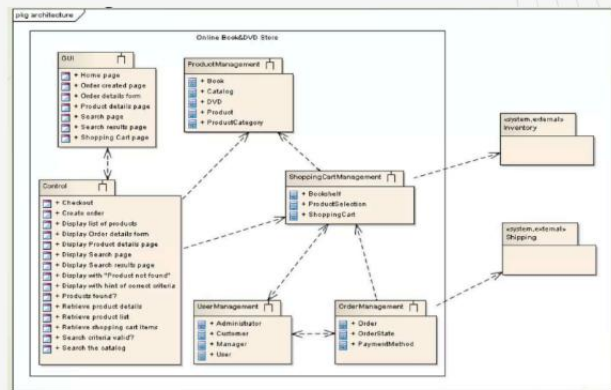
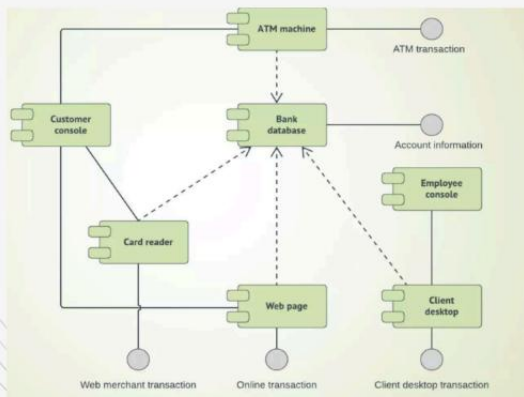
- Component diagrams
- Package diagrams

◆ Example:

Frontend Module
Backend Module
Database Module



COMPONENT & PACKAGE DIAGRAMS



PHYSICAL VIEW:

Physical view shows how software components map onto hardware nodes. i.e. where system runs eg hardware server.

4. PHYSICAL VIEW (Where system RUNS)

Also called:

👉 Deployment View

◆ Focus:

- Hardware
- Servers
- Network

◆ Example:

User → Web Server → App Server → Database

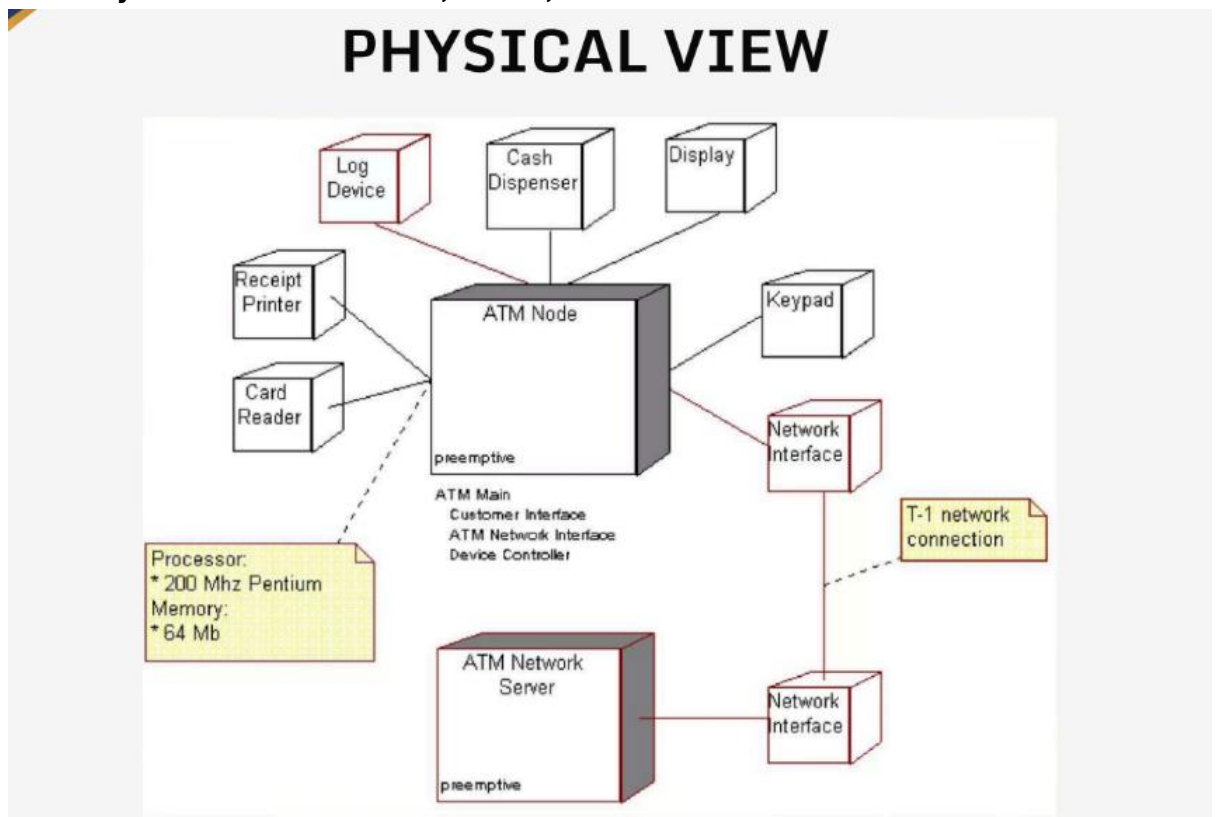


◆ Answers:

- Which machine runs what?
- System topology



- Where system runs ie hardware , server , environment .



🟡 +1 SCENARIO VIEW (MOST IMPORTANT)

◆ What it is:

- 👉 Real use cases (actual system usage)

Example:

- Withdraw money from ATM

◆ Why it's needed:

Because:

It CONNECTS all 4 views

◆ Why it's needed:

Because:

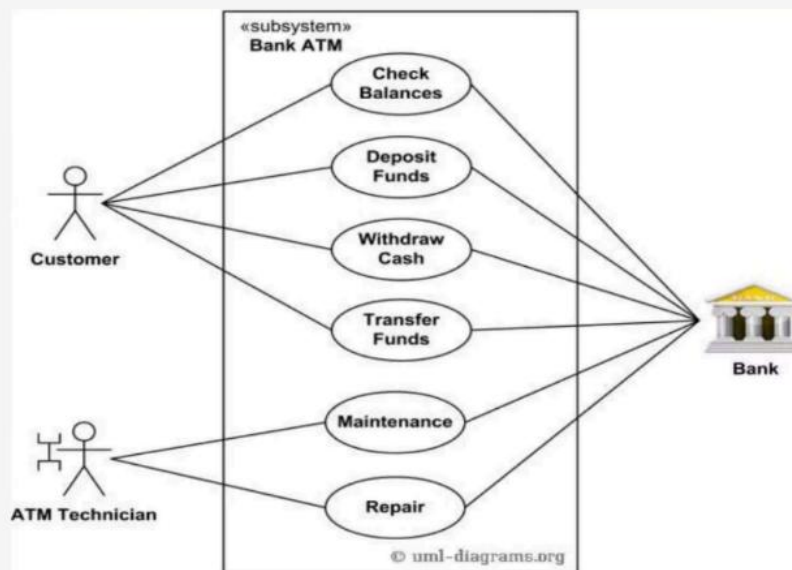
It CONNECTS all 4 views

◆ Example flow:

When user withdraws cash:

View	Role
Logical	Account, Transaction
Process	Request handling
Development	Code modules
Physical	 machine + server

SCENARIO VIEW (4+1 VIEW)



HOMOGENIZATION

In parallel development, different teams inevitably create design inconsistencies. Homogenization is the continuous process of reconciling these differences to form a unified, coherent model.

● PART 5: HOMOGENIZATION (VERY IMPORTANT EXAM TOPIC)

◆ Problem

Different teams create:

- Duplicate classes
- Conflicting names
- Wrong structure

◆ Solution:

👉 Homogenization = Clean and unify model

● Techniques (from slides)

1. Combining Classes

Problem:

- `Customer` vs `Client`

Solution:

👉 Merge them

2. Splitting Classes

Problem:

- One class doing too much

Solution:

👉 Break into smaller classes

Example:

Student → Student + Transcript



3. Eliminating Classes

Remove:

- Useless classes
- No role in system

4. Consistency Checking

Make sure:

- Sequence diagram ↔ Class diagram match

👉 Example:

If message exists → method must exist

5. Scenario Walkthrough

👉 Run real use case step by step

Check:

- Everything works logically



6. Event Tracing

For each message:

- Who sends?
- Who receives?
- Is connection valid?

7. Documentation Review

Check:

- Names are clear
- Everything defined

1. Combining Classes (Consolidation)

- **Goal:** Remove redundancy by merging "synonym" classes.
- **The Problem:** Different developers create classes that do the same thing (e.g., `PaymentProcessor` vs. `TransactionManager`).
- **The Action:** Identify semantic overlaps and merge their **attributes, operations, and relationships** into one class with the best name.
- **Pro-Tip:** Watch out for "Control Classes" with overlapping logic.

2. Consistency Checking (Static vs. Dynamic)

- **Goal:** Ensure the **Class Diagram** (structure) and **Sequence Diagrams** (behavior) match.
- **Key Checks:**
 1. Every **message** in a sequence diagram must be an **operation** in a class.
 2. Every interaction needs a valid **communication pathway** (association/link).
 3. Every class must appear in at least one scenario (no "orphan" classes).

3. Scenario Walk-Through (Validation)

- **Goal:** Step through use cases to validate the model.
- **The Process:** * Pick a critical use case.
 - Trace the sequence of messages.
 - **Verify:** Ensure the receiver class has the logic to handle the message and that no operation is left "unused."

4. Event Tracing (Interaction Audit)

- **Goal:** A deep-dive audit of the message flow.
- **The Audit List:**
 - Identify the **Sender** operation (initiator).
 - Identify the **Receiver** operation (handler).
 - Confirm an **Association** exists between them.
 - If using **State Diagrams**, ensure the event is listed there as well.

- If using **State Diagrams**, ensure the event is listed there as well.

5. Documentation Review (Quality Control)

- **Goal:** Final "polish" to ensure the model is a reliable reference.
- **Mandatory Checks:**
 - **Uniqueness:** No duplicate class names.
 - **Completeness:** All attributes and operations are fully defined.
 - **Standards:** Follow the project's formatting and naming conventions.

Exam Cheat Sheet (The "Big 5" keywords)

If you are asked about "Refining" or "Finalizing" a model, just remember these 5 headers:

1. **Consolidate** (Merge synonyms)
2. **Consistency** (Static = Dynamic)
3. **Walk-Through** (Test Use Cases)
4. **Trace** (Check Sender/Receiver/State)
5. **Document** (Clean up definitions)

● FINAL UNDERSTANDING (SUPER SIMPLE)

🔥 4+1 in ONE LINE

Break system into **different views** so every stakeholder understands it, and use **scenarios to verify everything works together**

🔥 Mental Map

```

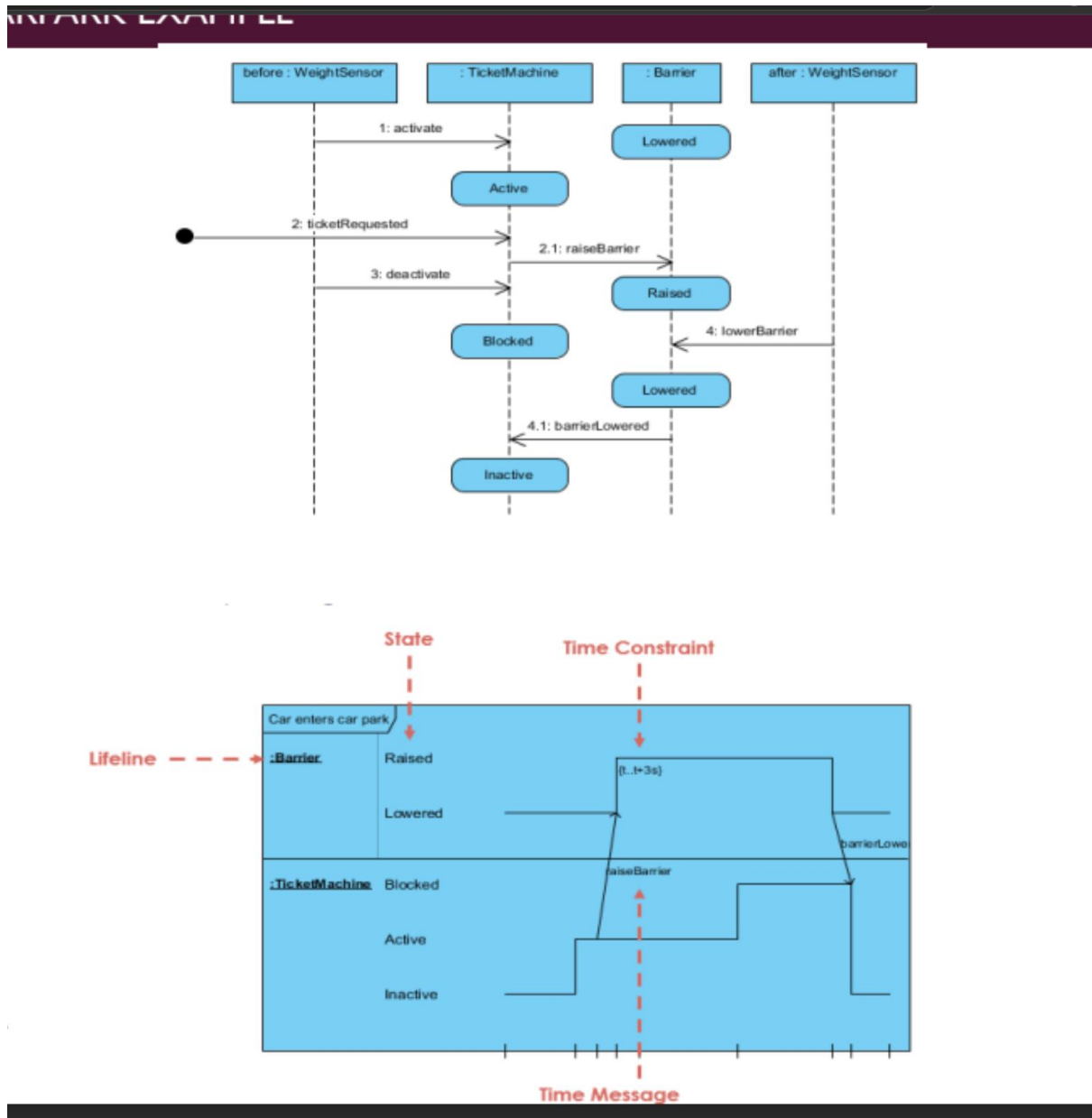
+1 Scenario (use cases)
  ↑
-----
| Logical | Process | Dev | Physical |
-----
  
```

🔥 Who uses what?

- User → Logical
- Developer → Development
- Engineer → Process
- System Engineer → Physical
- Everyone → Scenario



TIMING DIAGRAM



sends what message to whom. The timing diagram shows what state each object is in across time as a result.

Walking through the events:

t0 — activate (msg 1): The `before:WeightSensor` detects a car and sends `activate` to the `:TicketMachine`. On the timing diagram, you see the `:TicketMachine` step from Inactive → Active at this exact moment. The `:Barrier` is still Lowered — nobody told it to move yet.

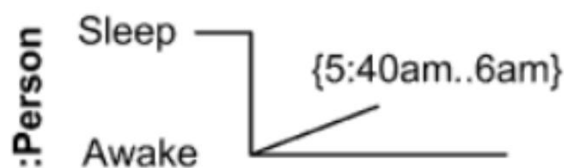
t1 — ticketRequested + raiseBarrier (msg 2 → 2.1): The customer requests a ticket. The `:TicketMachine` then fires `raiseBarrier` to `:Barrier`. On the timing diagram, the `:Barrier` line jumps from Lowered → Raised. This is also when the **time constraint** `[t..t+3s]` begins — the barrier can only stay up for a maximum of 3 seconds (a safety rule).

t2 — deactivate (msg 3): The `before:WeightSensor` sends `deactivate` (the car has moved forward past it). The `:TicketMachine` steps to Blocked — it won't accept another car until the barrier comes back down.

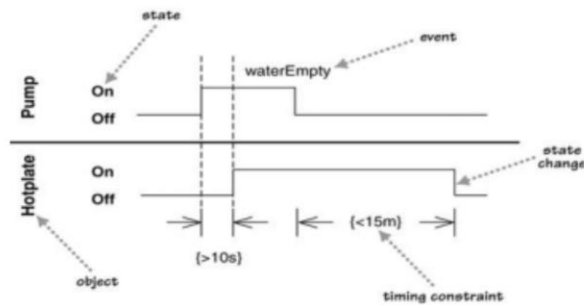
t3 — lowerBarrier + barrierLowered (msg 4 → 4.1): The `after:WeightSensor` detects the car has cleared the barrier, so it sends `lowerBarrier` to `:Barrier`. The barrier drops (Raised → Lowered). It then fires `barrierLowered` back to `:TicketMachine`, which finally moves from Blocked → Inactive, ready for the next car.

TIME CONSTRAINT

- **Time constraint** is an **interval constraint** that refers to a **time interval**. The time interval expression used to determine whether the constraint is satisfied.
- Typically this graphical association is a small line, e.g., between an occurrence specification and time interval.
- E.g., *Person should wake up between 5:40 am and 6 am*



- Let's take a simple scenario based on the pump and hotplate for a coffee pot. Let's imagine a rule that says that at least 10 seconds must pass between the pump coming on and the hotplate coming on. When the water reservoir becomes empty, the pump switches off, and the hotplate cannot stay on for more than 15 minutes more.



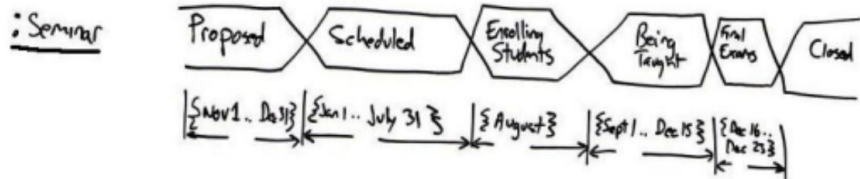
SDA

12

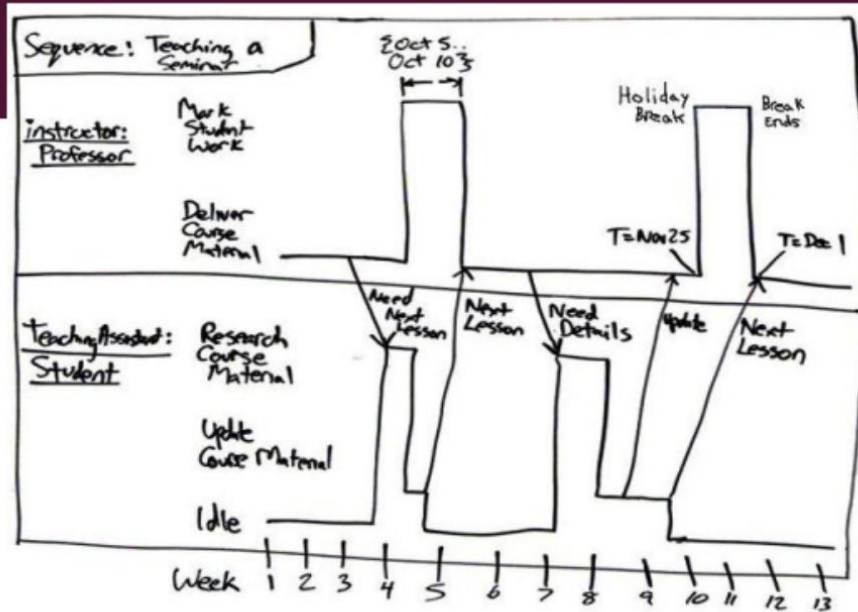
- There are two basic flavors of timing diagram,
 - Concise notation
 - Robust notation
- Showing states as lines – They are known as **State Lifeline diagram** or robust notations
- Showing states as areas - They are known as **Value Lifeline diagram** or concise notations

TIMING DIAGRAM-CONCISE NOTATION

- Life Cycle of Seminar**
- The critical states that the seminar exhibits
 - Proposed,
 - Scheduled,
 - Enrolling Students,
 - Being Taught,
 - Final Exams,
 - Closed



SDA



COMPONENT DIAGRAM

1. Context: Where We Are in UML

So far the course covered diagrams that either show structure (Use Case, Class) or behavior (Sequence, Activity, State, Collaboration). This lecture shifts to the **Implementation View** — how the system is physically built. Two diagrams fall here:

- **Component Diagram** — what software pieces exist and how they connect
- **Deployment Diagram** — where those pieces physically run (covered separately)

2. What is a Component?

A component is an **autonomous, self-contained unit** of a system. Think of it as a black box — it offers services to the outside world but hides how it works internally.

In software, a component can be any of: a source code file (login.java), a library (.dll / .jar), a database table, or an executable (.exe).

The critical property: a component **encapsulates functionality** and can be **reused or replaced independently**, as long as the interfaces match. You can swap one component for another without touching the rest of the system.

Exam indicator: If asked "what makes something a component?", the answer is: autonomous, self-contained, exposes interfaces, hides internals, independently replaceable.

Exam indicator: Component diagram = physical/implementation view. Class diagram = logical/conceptual view.

3. What is a Component Diagram?

A Component Diagram visualizes how a system is **divided into logical/physical pieces**, showing those pieces (components), their interfaces, and how they interact.

Key question it answers: *"How is my software system implemented in terms of components and their connections?"*

Three key characteristics to remember:

- Models high-level software components and their interfaces
- Shows dependencies between components
- Focuses on **independently deployable units** (you can deploy each component on its own)

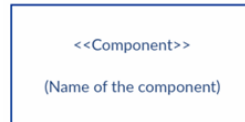
4. Elements of a Component Diagram

Element	What It Is
Component	Main building block — a rectangle with <code><<component>></code> stereotype
Interface	Defines operations a component provides or requires
Port	A connection point (small square on component boundary)
Connector	Link between components — either Assembly or Delegation

COMPONENT

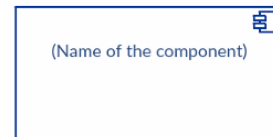
Notation 1:

Rectangle with the component stereotype (the text <<component>>). The component stereotype is usually used above the component name to avoid confusing the shape with a class icon.



Notation 2:

Rectangle with the component icon in the top right corner and the name of the component.



COMPONENT

Notation 3:

A rectangle with both the component icon and the stereotype label.



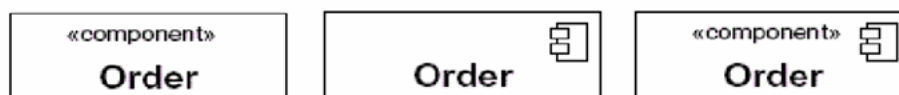
6. Component Stereotypes

These tell you *what kind of component* it is:

Stereotype	Meaning
<code><<application>></code>	Front-end app (GUI, web page)
<code><<database>></code>	Database or data source
<code><<library>></code>	A .dll or .jar library
<code><<executable>></code>	An .exe file
<code><<table>></code>	A table inside a database
<code><<web service>></code>	A web-based service interface

Exam indicator: These are just labels giving context. In a question about an e-commerce system, the web front-end would be `<<application>>`, the MySQL database would be `<<database>>`, etc.

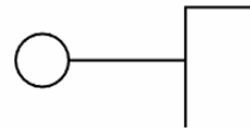
COMPONENT



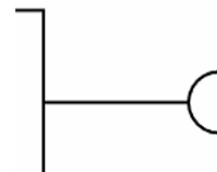
INTERFACES

Interfaces define how components talk to each other, what services one component offers and what services it expects.

Provided interfaces: A straight line from the component box with an attached circle. These symbols represent the interfaces where a component produces information used by the required interface of another component.



Required interfaces: A straight line from the component box with an attached half circle (also represented as a dashed arrow with an open arrow). These symbols represent the interfaces where a component requires information in order to perform its proper function.

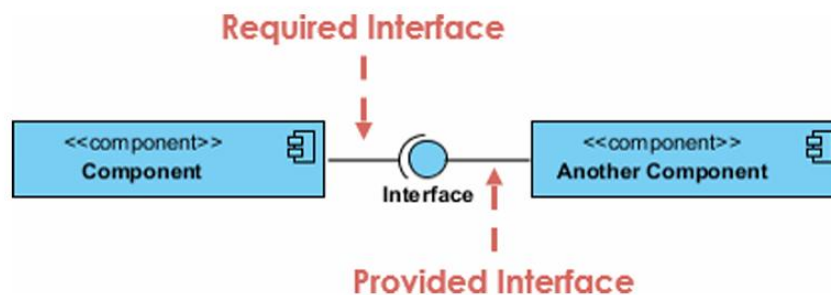


You can also list interfaces inside the component box in separate compartments, similar to how you list attributes and methods in a class. The compartments are labelled `«provided interfaces»` and `«required interfaces»`.

Exam indicator: Circle = provided (giving). Half-circle = required (needing). This is the most commonly tested notation detail.

INTERFACES

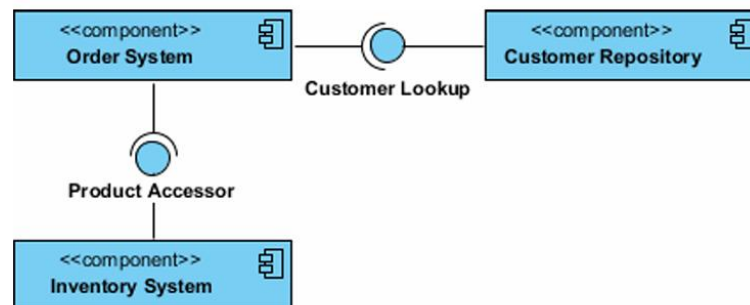
Interfaces define how components talk to each other, what services one component offers and what services it expects.





INTERFACES

Component Diagram Example - Using Interface (Order System)



Interpretation of the Order System Component Diagram

There are 3 components and 2 interfaces in this diagram.

Components:

- **Order System** — the central component. It manages orders and needs data from both the customer and inventory sides.
- **Customer Repository** — stores/provides customer data.
- **Inventory System** — stores/provides product data.

Interfaces:

Customer Lookup (between Order System and Customer Repository)

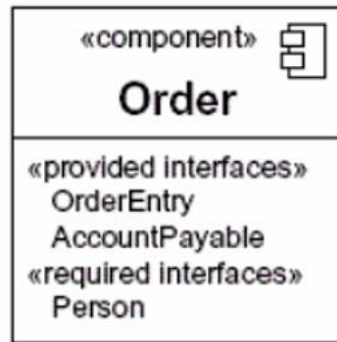
The circle (lollipop) is on the Customer Repository side, meaning Customer Repository **provides** the Customer Lookup interface. The Order System connects to it from the left, meaning Order System **requires** it. So: Order System asks "give me customer data" → Customer Repository fulfills it through the Customer Lookup interface.

Product Accessor (between Order System and Inventory System)

Same logic. The circle is between Order System (above) and Inventory System (below). Order System **requires** product information, and Inventory System **provides** it through the Product Accessor interface.

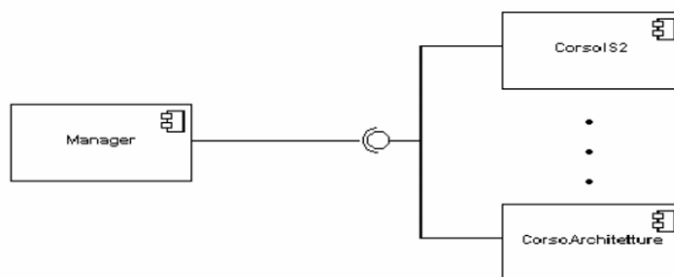
Reading it as a story:

When an order is being processed, the Order System needs to look up which customer placed the order — it does this via the Customer Lookup interface provided by the Customer Repository. It also needs to check product/stock information — it does this via the Product Accessor interface provided by the Inventory System.



INTERFACES

In a system context where there are multiple components that require or provide a particular interface, a notation abstraction can be used that combines by joining the interfaces.



CONNECTORS

There are two types of connectors:

1. Assembly Connector:

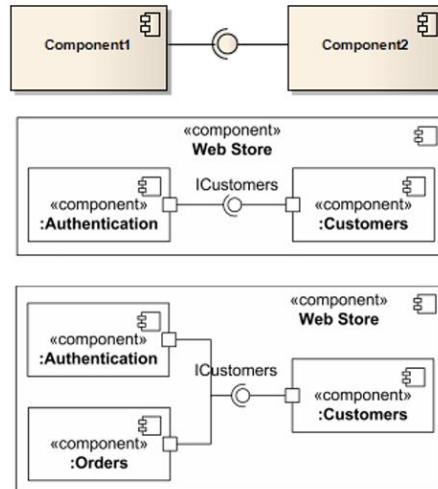
- Connects a required interface of one component to a provided interface of another.
- Think of it as “plugging” one component into another.

2. Delegation Connector:

- Used when a component passes responsibility to an internal component.
- Shows that internal parts realize or use the external interfaces.

ASSEMBLY CONNECTORS

Assembly connector between simple ports of Authentication and Customers components.



Assembly connector that assembles three parts.

PORTS

A port defines a specific interaction point between a component and its environment.

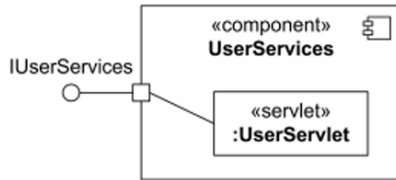
It's drawn as a small square on the component boundary.

Ports:

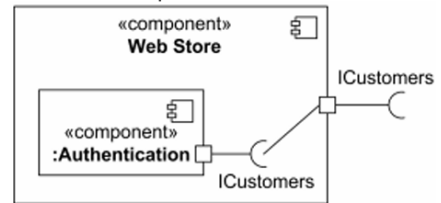
- Can handle uni- or bi-directional communication
- Can be named (e.g., searchPort)
- Can be linked with one or multiple interfaces

DELEGATION CONNECTORS

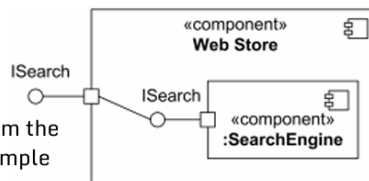
Delegation connector from the delegating port to the UserServlet part



Delegation connector from the simple port of Authentication component to the delegating port.



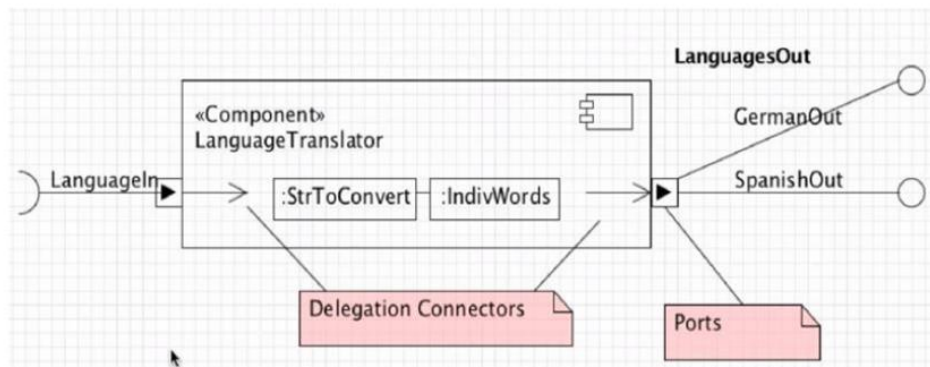
Delegation connector from the delegating port to the simple port of SearchEngine.



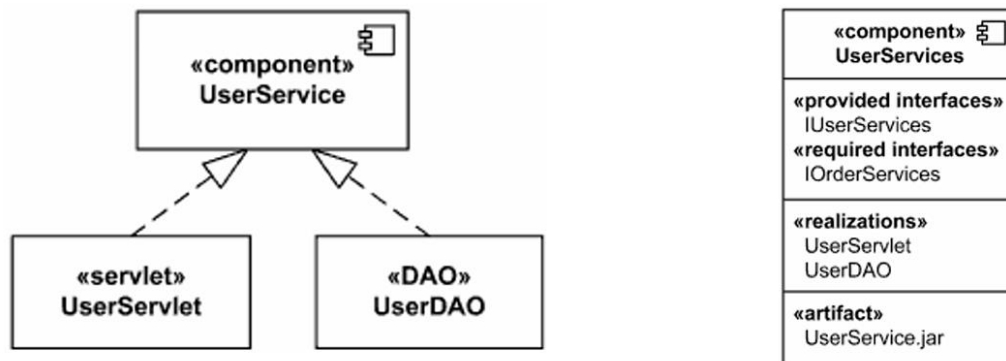
When you open up a component and show what's inside, that is the **Internal/White-Box View**.

the behavior promised by the component's external interfaces is physically implemented by the classes, components, or artifacts inside it

INTERNAL REALIZATION



INTERNAL REALIZATION



DEPENDENCY

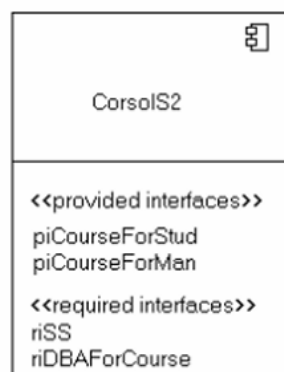
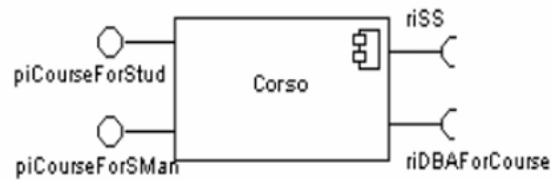
- A relationship in which one element requires another element for its full implementation
 - This means the dependent component (client) cannot function completely without the supplier component
- A dependency in which the client requires the presence of the supplier
- The client component needs the supplier component to be present in the system. If the supplier is missing, the client won't work properly
 - Shown as dashed arrow with a <<use>> keyword

• Key Properties:

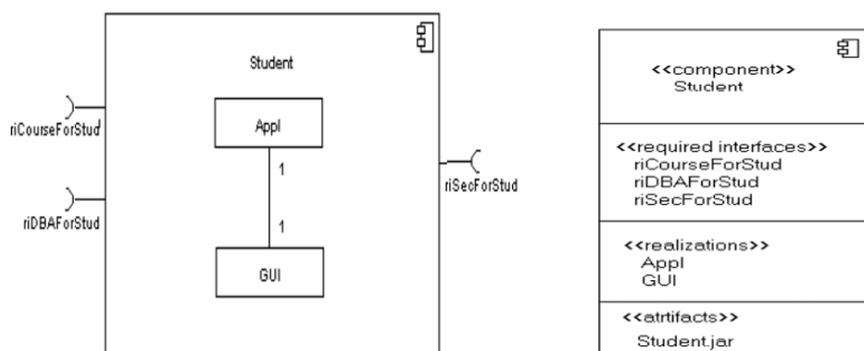
- **Directional:** The dependency has a clear direction—from the client (who needs something) to the supplier (who provides it).
- **Not a Direct Connection:** Unlike an assembly connector (ball-and-socket), a usage dependency does not specify the exact service or interface being used. It simply states that a dependency exists.



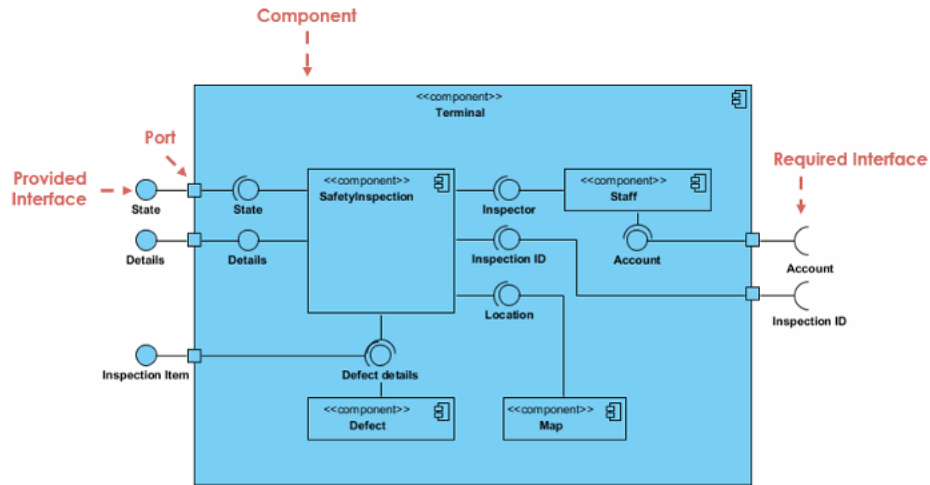
EXTERNAL VIEW



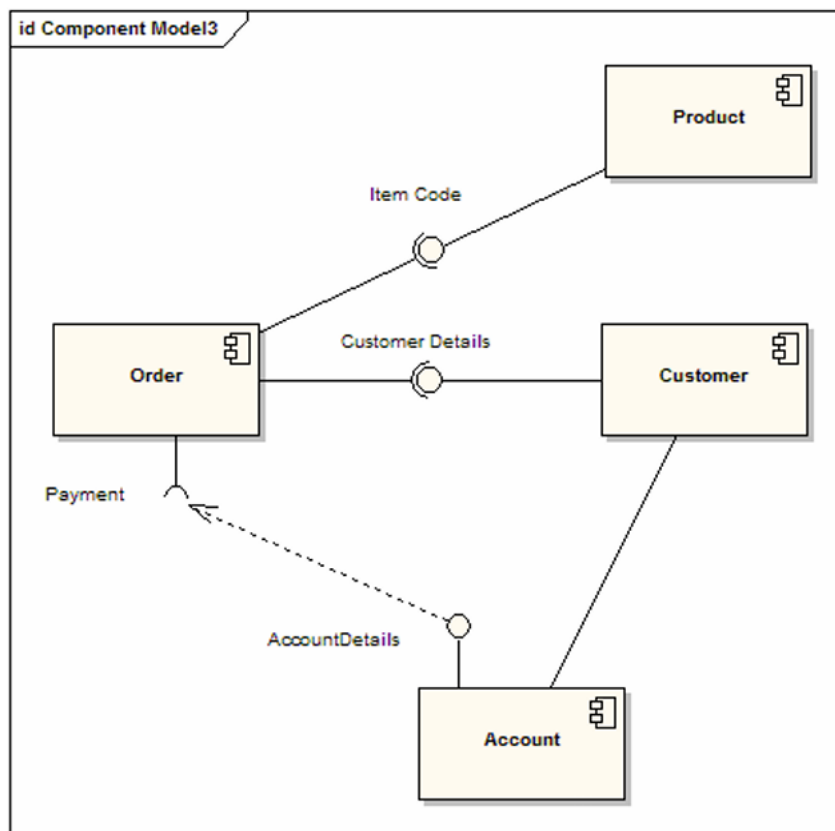
INTERNAL VIEW



EXAMPLE



EXAMPLE



The Dashed Arrow in This Diagram

The dashed arrow is labeled "Payment" and points from Account → Order.

What it means

This is a `<<use>>` dependency (usage dependency). It means:

Order depends on Account for payment processing. The Order component requires the Account component to be present in the system to handle payment — but the exact interface being used is not specified.

The Plain Line Between Customer and Account

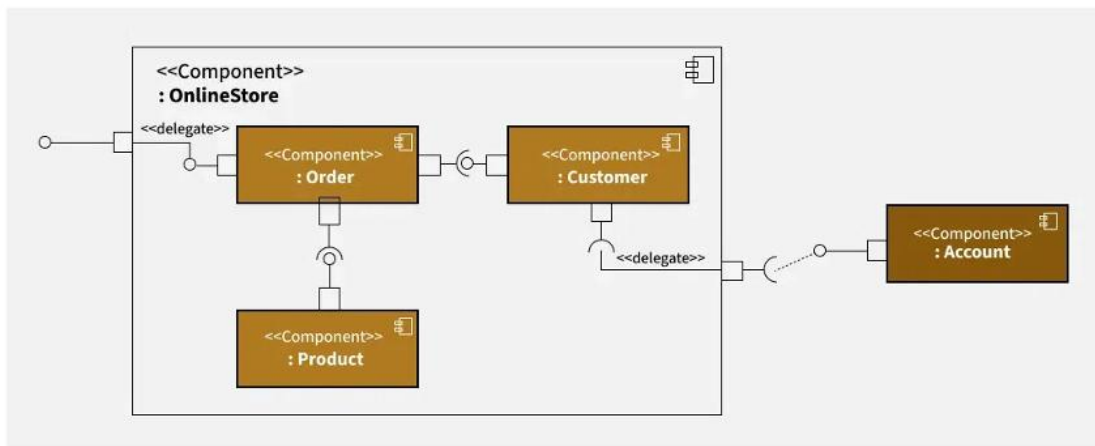
The line between Customer and Account has no ball, no half-circle, no dashed arrow — it's just a plain solid line.

What it means

This is simply an **association** — it means Customer and Account are **related/linked** to each other, similar to how you'd draw a relationship in a class diagram.

It's saying: "*A Customer has an Account*" — just a structural relationship between the two components, not a service/interface dependency.

EXAMPLE: ONLINE STORE



So what tells you the direction?

The `<<delegate>>` label tells you direction and intent — Customer is pushing its requirement *outward* toward Account. You don't need an arrowhead because the delegate label already communicates the flow.

Compare the two dashed line types

Dashed Line Type	Has Arrow?	Has Label?	Meaning
<code><<use>></code> dependency	YES, arrowhead	<code><<use>></code>	Component A generally depends on Component B
Delegation to external	NO arrowhead	<code><<delegate>></code>	Internal component passing requirement through boundary to outside

Bottom line

No arrow on the dashed line = it is **not** a use dependency. The `<<delegate>>` label is what identifies it instead. The circle on Account's side tells you Account is the *provider* in this connection.

Why it is mentioned/labeled `<<delegate>>`

The `<<delegate>>` label is placed explicitly to tell the reader:

"This is not an assembly connector between two peers. This is an internal component forwarding its responsibility to the boundary."

It distinguishes delegation from assembly so there is no confusion about what kind of connection it is.

One line summary

Delegation connector exists because **internal components cannot directly reach outside their parent boundary** — they must delegate that responsibility outward through a port, and the `<<delegate>>` label makes this explicit.

That gateway IS the delegation connector

The delegation connector + port combination says:

"I (Customer) need something from outside. I'm going to pass that requirement to my parent boundary (OnlineStore), and the parent will handle the external connection."

So the flow is:

```
Customer (needs Account data)
  → delegates outward via port on OnlineStore boundary
    → reaches external Account component
```

You are correct

If the connection was a **simple dependency inside the boundary** (between two peer components), it would be:

- Dashed line **with arrowhead**
- Labeled `<<use>>`
- **No** `<<delegate>>` label

Direction of Arrow in General Case

The arrow always points from **client** → **supplier**.

Meaning: the one who **needs** something points TO the one who **provides** it.

```
Client -----<<use>>-----> Supplier
(needs it)                      (provides it)
```



Real Example from this diagram if it were a dependency

If Customer simply depended on Account as a peer (no boundary issue):

```
Customer ----<<use>>-----> Account
```

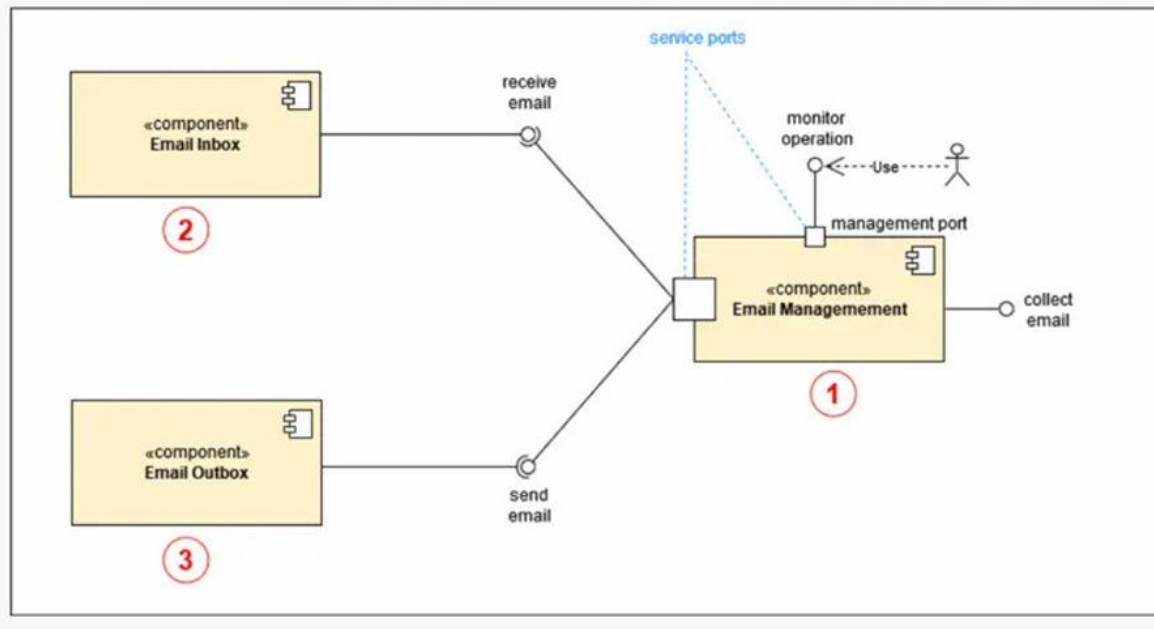
Customer is the client (needs account data), Account is the supplier (provides it). Arrow goes Customer → Account.



One line memory trick

`<<use>>` = "I need you" → arrow points at who you need. `<<delegate>>` = "pass it through the wall" → no arrow, boundary handles it.

EXAMPLE: EMAIL SYSTEM



Service Port — What it is and Why it's Here

The "management port" (small square on the top boundary of Email Management) labeled in blue as "service ports" is a specifically named port.

A **service port** is just a port that is designated for **external administrative/management access** — it is a named, dedicated entry point on the component boundary for a specific type of interaction (management/monitoring in this case).

Why it's shown here specifically:

- The "monitor operation" interface is accessible through this management port
- It separates management-level access from normal operational access (receive/send email)
- It makes clear that monitoring happens through a dedicated controlled entry point, not through the same port as regular email flow

service port (named port) Dedicated boundary entry point for management access

General Rule of Thumb for Any Description

- | | |
|-----------------------------|--|
| 1. Nouns doing things | = Components |
| 2. "inside / consists of" | = White-box / parent boundary |
| 3. Named services passed | = Named interfaces (assembly connectors) |
| 4. "provides X" | = Lollipop (provided interface) |
| 5. "requires / needs X" | = Half circle or required interface |
| 6. Vague "uses / talks to" | = <<use>> dashed arrow with arrowhead |
| 7. Internal → External need | = Delegation connector through boundary port |

Follow these 7 rules on any description and you will correctly build the component diagram every time.

Step 1 — Identify the Components

Read the description and ask: "what are the major software units/pieces mentioned?"

From the text:

- "sales till" → **Till**
- "message queue component" → **Message Queue**
- "sales server... transaction processor" → **Transaction Processor** (inside Sales Server)
- "accounting driver" → **Accounting Driver** (inside Sales Server)
- "accounting system component" → **Accounting System**

So you get: Till, Message Queue, Sales Server (containing Transaction Processor + Accounting Driver), and Accounting System.

Indicator: Every noun that does something independently = likely a component.

Indicator: Every noun that does something independently = likely a component.

Step 2 — Identify Boundaries (Who is inside what)

The text says: *"The server is broken down into two major components — transaction processor and accounting driver."*

This tells you **Transaction Processor** and **Accounting Driver** are **INSIDE** **Sales Server** — so Sales Server is a composite/parent component with internal white-box view.

Everything else (Till, Message Queue, Accounting System) is outside/peer level.

Indicator: Phrases like "broken down into", "consists of", "contains" = parent-child boundary relationship.

Breaking it down honestly

Sentence 1:

"Transaction processor realizes the sales message interface, and the accounting driver"

This means Transaction Processor realizes TWO things:

- The sales message interface
- The accounting driver

"Realizes" here means Transaction Processor **implements/uses** both.

Sentence 3:

"Accounting driver **provides** this information to transaction processor"

The word **provides** is explicitly written. Accounting Driver → provides billing info → to Transaction Processor.

So you are right — it IS written. The word "provides" is directly in the text for Accounting Driver → Transaction Processor.

For the Sales Message Interface

"Transaction processor realizes the sales message interface"

"Realizes" = implements/provides it. So Transaction Processor is the **provider** of sales message interface. That is why lollipop is on Transaction Processor side.

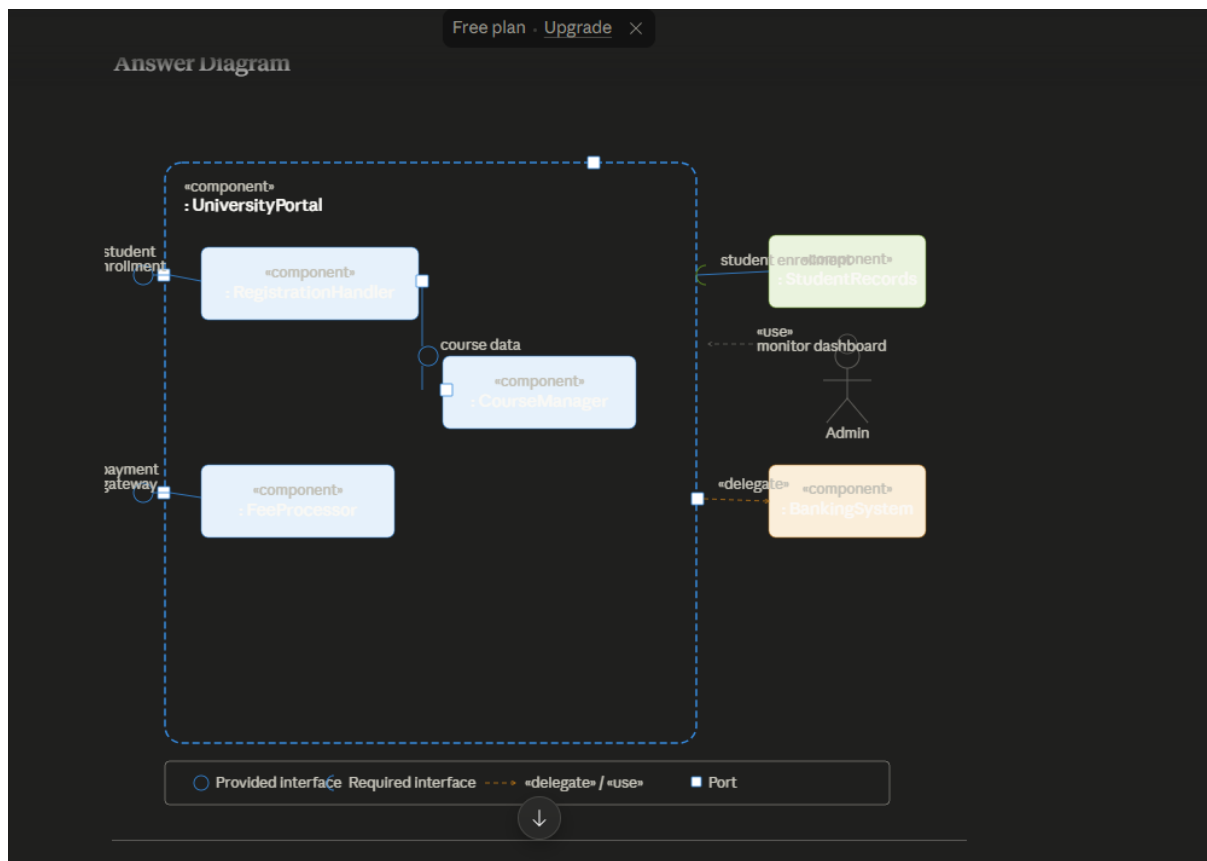
The Rule

Word in text	What it maps to
"realizes"	Provider — draw lollipop on that component
"provides X to Y"	Provider — draw lollipop, Y is the requirer
"talks to / needs / requires"	Requirer — dashed arrow toward provider
"uses"	<code><<use>></code> dependency — dashed arrow with arrowhead

Exam-Style Question

A **university portal system** manages student registrations, course management, and fee payments. The portal is the main system and contains two internal components: a **Registration Handler** and a **Fee Processor**. The Registration Handler realizes the **student enrollment** interface and also requires course data from an internal **Course Manager** component also inside the portal. The Fee Processor realizes the **payment gateway** interface and talks to an external **Banking System** for transaction verification. The portal provides the **student enrollment** and **payment gateway** interfaces to the outside world through its boundary ports. An external **Student Records** component requires the student enrollment interface from the portal. An external **Admin** actor uses the portal's **monitor dashboard** interface to oversee operations. The Course Manager provides course data to Registration Handler via the **course data** interface.

Now attempt the diagram first, then I will show you the answer.



What the diagram covers

Every concept from the lecture is present:

- **3 internal sub-components** inside UniversityPortal (white-box view)
- **Assembly connector** with named interface — course data between RegistrationHandler and CourseManager
- **Provided interfaces** (lollipops) on the portal boundary — student enrollment and payment gateway
- **Required interface** (half circle) on StudentRecords connecting to the portal's provided interface
- **Boundary ports** (small squares) on the portal wall
- **<<delegate>>** connector — FeeProcessor delegating payment requirement outward to external BankingSystem
- **<<use>>** dashed arrow — Admin actor using monitor dashboard interface
- **External components** — StudentRecords, BankingSystem, Admin actor all outside the boundary

DEPLOYMENT DIAGRAM

- Two major UML implementation diagrams:
 - a. **Component Diagram:** Focuses on software structure
 - b. **Deployment Diagram:** Focuses on hardware & runtime structure.

Implementation Diagrams (Quick Recap)

UML has two implementation diagrams. Component Diagrams focus on software structure — how code modules are organized and communicate. Deployment Diagrams focus on the physical/hardware side — where those components actually run.

What is a Deployment Diagram?

A Deployment Diagram shows the physical architecture of a system. It tells you which software (artifacts) runs on which hardware (nodes), and how those machines communicate with each other. While component diagrams answer "how do the pieces fit together logically?", deployment diagrams answer "where do these pieces actually live at runtime?"

Deployment diagrams are used to show the physical layout of hardware and software, visualize the runtime structure of a system, identify communication paths (TCP/IP, HTTP, RMI, sockets), and help with system configuration, scaling, and maintenance. They bridge the design model to real infrastructure and are heavily used by system architects and DevOps teams.

It tells you which software (artifacts) runs on which hardware (nodes), and how those machines communicate with each other

Elements of a Deployment Diagram

1. Nodes

Nodes represent hardware machines or software execution environments. They are drawn as 3D boxes. There are two types:

Hardware Nodes — actual physical machines, stereotyped as `<<device>>`. Examples:

`<<device>> Web Server`, `<<device>> Database Server`, `<<device>> Client Workstation`,
`<<device>> Mobile Device`.

Execution Environment Nodes — software platforms running on top of hardware,

stereotyped as `<<executionEnvironment>>`. Examples: `<<executionEnvironment>> Apache Tomcat`, `<<executionEnvironment>> Docker Container`, `<<executionEnvironment>> .NET Runtime`.

A node can contain other nodes. For example, a physical Web Server node (device) might contain an Apache Tomcat node (executionEnvironment) running inside it.

2. Connections

Connections are lines drawn between nodes to represent communication paths. They are solid lines and can be labeled with the protocol being used, such as TCP/IP, HTTP, or RMI.

3. Artifacts

Artifacts are the physical files that actually get deployed onto nodes. They are the real-world manifestation of components. They appear as rectangles labeled with the `<<artifact>>` stereotype and a document icon.

Examples of artifacts: `.jar`, `.war`, `.ear` files in Java; `.exe`, `.dll` files in Windows; configuration files like `config.xml`; HTML files like `index.html`; database scripts.

Artifacts sit inside nodes to show which machine they are deployed on. A dashed arrow labeled `<<deploy>>` can show that an artifact is being deployed to a node.

2. Connections

Connections are lines drawn between nodes to represent communication paths. They are solid lines and can be labeled with the protocol being used, such as TCP/IP, HTTP, or RMI.

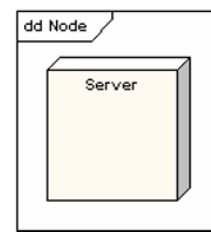
3. Artifacts

Artifacts are the physical files that actually get deployed onto nodes. They are the real-world manifestation of components. They appear as rectangles labeled with the «artifact» stereotype and a document icon.

Examples of artifacts: `.jar`, `.war`, `.ear` files in Java; `.exe`, `.dll` files in Windows; configuration files like `config.xml`; HTML files like `index.html`; database scripts.

Artifacts sit inside nodes to show which machine they are deployed on. A dashed arrow labeled «deploy» can show that an artifact is being deployed to a node.

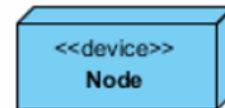
NODES



- One can think of nodes as the physical hosts or containers that execute components.
- A node may contain other nodes, for example, a Web Server Node might host multiple Application Containers.

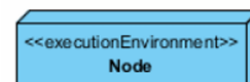
HARDWARE NODES

- These are the actual physical machines. We stereotype them as <<device>>:
 - <<device>> Web Server
 - <<device>> Database Server
 - <<device>> Client Workstation
 - <<device>> Mobile Device



EXECUTION ENVIRONMENT NODES

- Execution environments are software platforms that run on hardware:
 - <<executionEnvironment>> Apache Tomcat
 - <<executionEnvironment>> J2EE Container
 - <<executionEnvironment>> .NET Runtime
 - <<executionEnvironment>> Docker Container



COONNECTIONS

- Represent communication paths between nodes.
- Can be labeled with protocols (TCP/IP, HTTP, RMI, etc.).
- Often drawn as solid lines connecting 3D boxes.



ARTIFACT



- Artifacts are physical manifestations of components discussed in the last lecture.
- Represent physical files: compiled code, executables, configuration, databases.
- Shown as rectangles with «artifact» stereotype and a document icon.

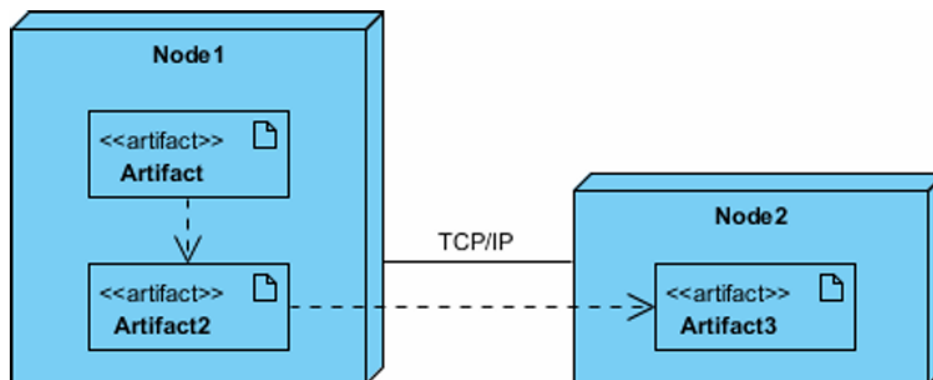
ARTIFACT



Artifacts are the ACTUAL files we deploy:

- .jar, .war, .ear files in Java
- .exe, .dll files in Windows
- Database script files
- Configuration files
- Examples:
 - «artifact» library.jar
 - «artifact» index.html
 - «artifact» config.xml

ARTIFACT



things.

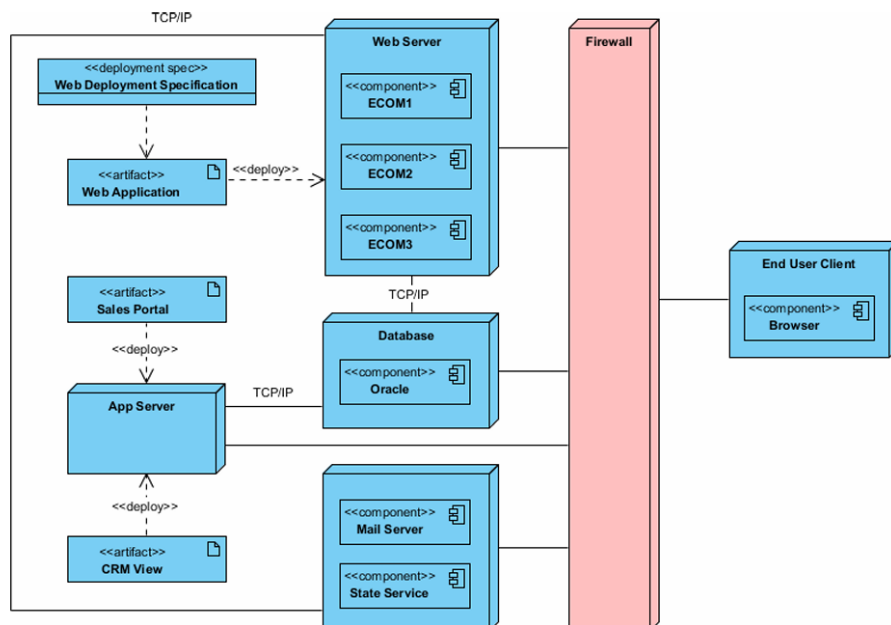
Dashed arrow going downward (Artifact → Artifact2) — this is a «manifest» or dependency relationship. It means Artifact2 is derived from or generated by Artifact. Think of it like: Artifact is the source (e.g., a component or spec), and Artifact2 is its physical manifestation (the actual deployed file).

Dashed arrow going right (Artifact2 → Artifact3 across nodes) — this represents a «deploy» or usage dependency across the network. It means Artifact2 on Node1 communicates with or depends on Artifact3 on Node2, and that communication happens over TCP/IP (shown by the solid line labeled TCP/IP between the nodes).

So in short, the solid line between the two nodes is the physical communication channel (the network connection), while the dashed arrow between artifacts shows which artifact is actually using or talking to the other artifact through that channel.

A simple way to remember: solid lines = physical connections between hardware, dashed arrows = logical dependencies between software artifacts.

DEPLOYMENT DIAGRAMS EXAMPLE



Left Side — Artifacts Being Deployed

There are three artifacts defined outside the servers, representing the software files that need to be deployed:

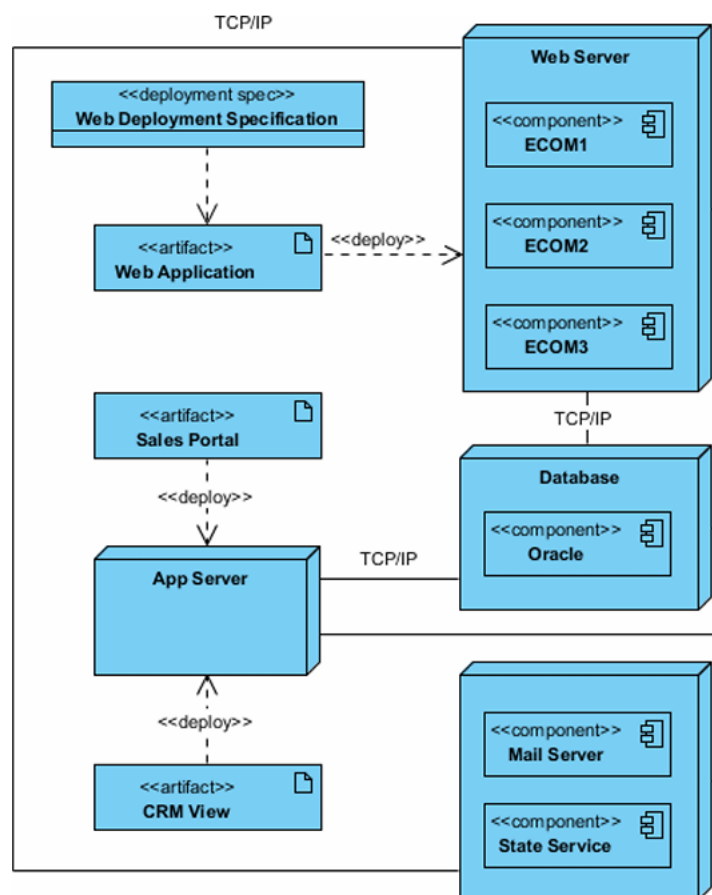
Web Deployment Specification (`<<deployment spec>>`) — this is a configuration/specification file (like a web.xml). A dashed arrow flows down from it to the Web Application artifact, meaning it governs or configures how the Web Application gets deployed.

Web Application (`<<artifact>>`) — the actual web app file (think a .war file). A `<<deploy>>` dashed arrow points from it toward the Web Server node, meaning this artifact gets deployed onto the Web Server.

Sales Portal (`<<artifact>>`) — another artifact. A `<<deploy>>` arrow points downward to the App Server, meaning the Sales Portal is deployed onto the App Server.

CRM View (`<<artifact>>`) — yet another artifact, also deployed onto the App Server via a `<<deploy>>` arrow pointing upward into it.

DEPLOYMENT DIAG



Nodes (Servers) — Center

Web Server — hosts three components: ECOM1, ECOM2, ECOM3. These are e-commerce application components running on the web server. The Web Application artifact is deployed here. The Web Server connects to the Database via TCP/IP.

Database — hosts Oracle as its component. It receives data requests from the Web Server and the App Server over TCP/IP.

App Server — the Sales Portal and CRM View artifacts are deployed here. It connects to the Database over TCP/IP and also communicates with the Mail Server and State Service below.

Mail Server + State Service node — sits below, containing two components: Mail Server (handles emails) and State Service (manages session/state). Connected to the App Server.

Firewall (Pink Vertical Bar)

The firewall sits between the internal server infrastructure and the outside world. Everything on the left of the firewall (Web Server, App Server, Database) is internal/protected. The End User Client is on the right, outside the firewall.

Right Side — End User Client

The End User Client node contains a Browser component. The user accesses the system through their browser, and their request passes through the firewall to reach the internal servers.

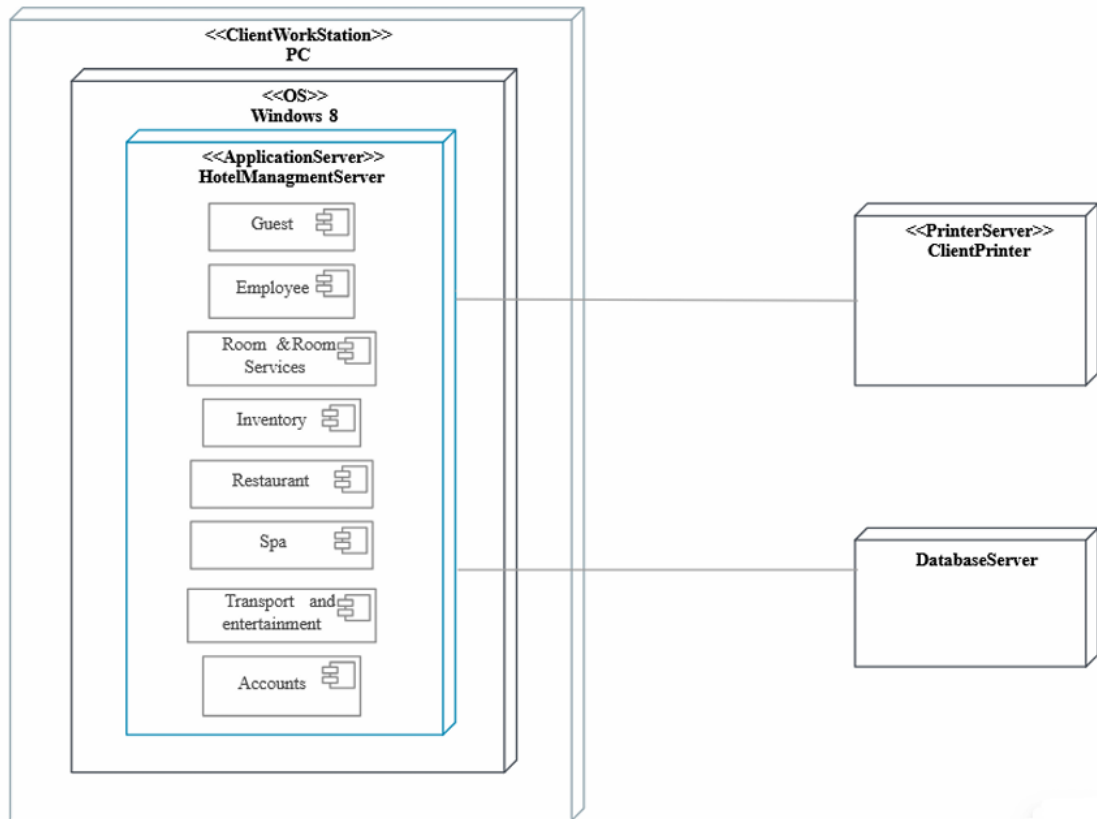
Flow Summary (How a request travels)

User (Browser) → through Firewall → hits Web Server (ECOM components handle request) → Web Server queries Database (Oracle) over TCP/IP → if business logic needed, App Server is involved (Sales Portal / CRM View) → App Server talks to Database and Mail/State services → response travels back through Firewall → back to Browser.

Key Relationships Recap

Relationship	What it means
Dashed arrow (spec → artifact)	Spec configures how artifact is deployed
«deploy» arrow	Artifact is deployed onto that node
Solid line with TCP/IP label	Physical network communication between nodes
Firewall bar	Security boundary between internal and external
Component inside node	Software running on that machine

HOTELMANAGEMENTSYSTEM



Deciphered deployment diagram's hardware and software node locations

Hotel Management System — Deployment Diagram Explained

The Nesting (This is the Key to Understanding)

This diagram uses **nodes inside nodes**, which is exactly what the slides mentioned. Read it from the outside in:

<code><<ClientWorkStation>> PC</code>	← HARDWARE NODE (outermost box)
<code><<OS>> Windows 8</code>	← EXECUTION ENVIRONMENT NODE
<code><<ApplicationServer>> HotelManagmentServer</code>	← EXECUTION ENVIRONMENT NODE
<code>Guest, Employee, Room & Room Services, Inventory, Restaurant, Spa, Transport, Accounts</code>	← COMPONENTS (software running inside)

Identifying Hardware vs Software Nodes

Hardware Node — `<<ClientWorkStation>> PC` This is the actual physical machine — a PC sitting at the hotel reception desk. It is the outermost 3D box. Stereotype is `<<ClientWorkStation>>` which falls under the `<<device>>` category taught in the slides.

Execution Environment Node — `<<OS>> Windows 8` This is the operating system running on that PC. It is a software platform, not physical hardware. It sits inside the PC box because the OS runs on that PC.

Execution Environment Node — `<<ApplicationServer>> HotelManagementServer` This is the application server software running on top of Windows 8. It is the innermost container. All the hotel system components run inside this server

External Nodes (Right Side)

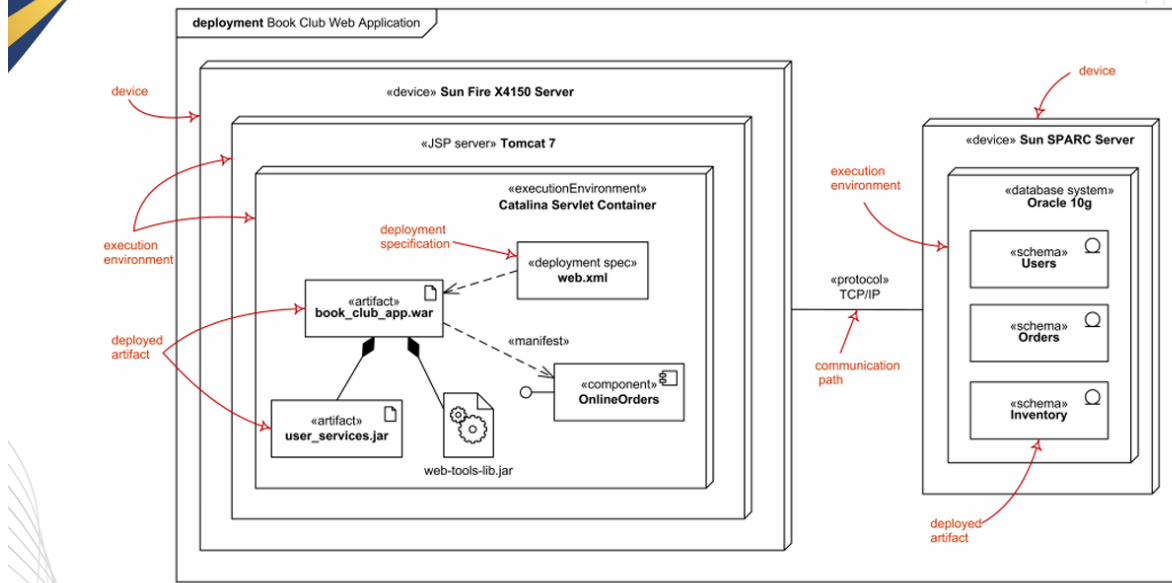
`<<PrinterServer>> ClientPrinter` — a separate physical printer server node. The PC connects to it so hotel staff can print receipts, invoices, etc.

`DatabaseServer` — a separate physical server storing all the hotel's data. The application server components connect to it to read/write data (guest records, bookings, accounts, etc.).

Summary Table

Element	Type	Role
<code><<ClientWorkStation>> PC</code>	Hardware Node	Physical machine at hotel desk
<code><<OS>> Windows 8</code>	Execution Environment	OS running on that PC
<code><<ApplicationServer>> HotelManagementServer</code>	Execution Environment	App server running on Windows 8
Guest, Employee, etc.	Components	Business logic modules
<code><<PrinterServer>> ClientPrinter</code>	Hardware Node	External printer machine
<code>DatabaseServer</code>	Hardware Node	External database machine
Horizontal lines	Connections	Network communication paths

DEPLOYMENT DIAGRAMS EXAMPLE



names the entire diagram. Everything inside it belongs to this system.

Left Side — Sun Fire X4150 Server

«device» Sun Fire X4150 Server — this is a **Hardware Node**. It is the actual physical server machine where the web application runs. Stereotype is **<<device>>** meaning real physical hardware.

Inside it sits:

«JSP server» Tomcat 7 — this is an **Execution Environment Node** running on top of the Sun Fire hardware. Tomcat 7 is the web server software. The stereotype here is a custom one (**«JSP server»**) describing what kind of server it is.

Inside Tomcat 7 sits:

«executionEnvironment» Catalina Servlet Container — another **Execution Environment Node**, nested inside Tomcat. Catalina is the actual servlet engine inside Tomcat that runs Java web applications. This shows the layering: physical machine → Tomcat → Catalina container.



Artifacts Inside Catalina Servlet Container

«artifact» `book_club_app.war` — this is the main deployed artifact. A `.war` file is a packaged Java web application. The red label on the left calls it a **deployed artifact**, meaning it has been physically deployed onto this node.

«artifact» `user_services.jar` — another artifact, a `.jar` library file providing user-related services. Also a deployed artifact inside the container.

`web-tools-lib.jar` — a third artifact (library), sitting outside the inner container but still within the Tomcat node. It is a supporting library used by the application.

«manifest» This means the `.war` artifact **physically contains/implements** the `OnlineOrders` component

The Manifest Relationship

A dashed arrow labeled «manifest» goes from `book_club_app.war` to the `OnlineOrders` component. This means the `.war` artifact **physically contains/implements** the `OnlineOrders` component. In other words, when you open `book_club_app.war`, the `OnlineOrders` component is what you find inside it.

«component» `OnlineOrders` — this is a software component (business logic) that handles online orders for the book club. It has a small circle (provided interface) shown attached to it.

Right Side — Sun SPARC Server

«device» `Sun SPARC Server` — another **Hardware Node**, the physical database server machine.

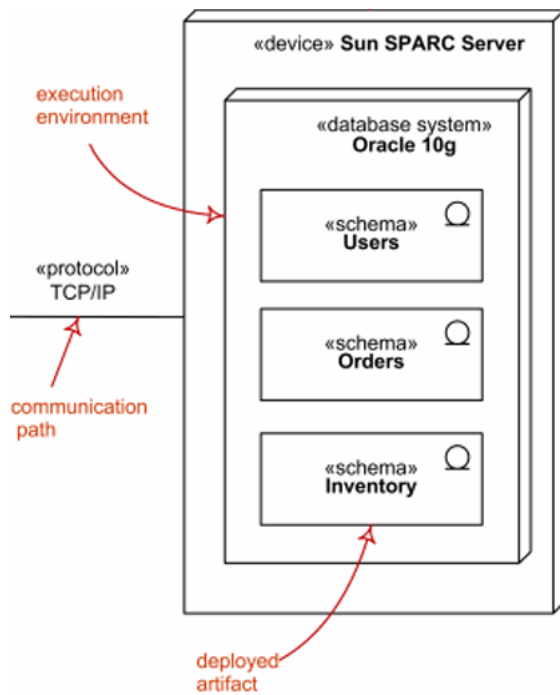
Inside it:

«database system» `Oracle 10g` — an **Execution Environment Node** (the red label says "execution environment"). Oracle 10g is the database management system running on the SPARC server.

Inside Oracle 10g are three **schemas** (deployed artifacts, as labeled in red):

- «schema» `Users` — stores user/member data
- «schema» `Orders` — stores order records
- «schema» `Inventory` — stores book inventory data

These schemas are labeled **deployed artifacts** on the right side, meaning they are the actual data structures deployed inside Oracle.



Communication Between the Two Servers

A solid line connects the Sun Fire server to the Sun SPARC server. It is labeled:

- `«protocol» TCP/IP` — the network protocol used
- **communication path** — confirming this is a network connection between the two physical machines

This means the OnlineOrders component running on the left server talks to the Oracle database on the right server over TCP/IP.

Looking at the diagram again carefully:

`web-tools-lib.jar` is connected to `book_club_app.war` and `user_services.jar` with **plain arrows (not dashed)**, and this represents a **«use» dependency** — meaning the `.war` and the `user_services.jar` both **use/depend on** the `web-tools-lib.jar` library to function.

Think of it this way:

- `web-tools-lib.jar` is a **shared utility library**
- Both `book_club_app.war` and `user_services.jar` need functions from it to run
- So both point to it (or it points to both) showing that dependency

The reason a **different arrow style** is used compared to the dashed manifest/deploy arrows is to distinguish the relationship type:

Arrow	Meaning
Dashed with «manifest»	This artifact physically contains/implements this component
Dashed with «deploy»	This artifact is being deployed onto this node
Solid/plain arrow	This artifact uses/depends on this other artifact

Q. A university has developed an online examination system. Draw a complete and detailed UML Deployment Diagram for the following scenario:

A student accesses the system through a **web browser** on their **personal laptop** running **Windows 11**.

The laptop connects over the internet to a **Linux-based Web Server** (running **Apache Tomcat 9** as the servlet container). The following artifacts are deployed on this server:

`exam_portal.war` (the main web app), `auth_service.jar` (handles login), and `web.xml` (deployment specification that configures how `exam_portal.war` is deployed).

The `exam_portal.war` manifests a component called **ExamEngine** which handles question delivery and timer logic.

Both `exam_portal.war` and `auth_service.jar` depend on a shared utility library called `commons-utils.jar`.

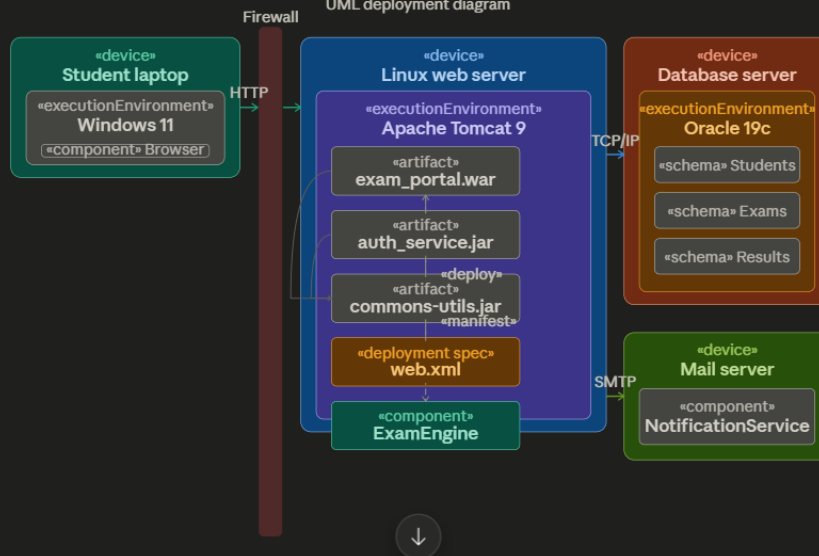
The Web Server communicates over **TCP/IP** with a **Database Server** (running **Oracle 19c**) which contains three schemas: `Students`, `Exams`, and `Results`.

The Web Server also communicates over **SMTP** with a **Mail Server** which contains a component called **NotificationService** that emails results to students.

A **Firewall** sits between the student's laptop and the internal server infrastructure.

Thought for 1s >

University online examination system UML deployment diagram



Legend

- device** — «device» — physical hardware machine
- exec env** — «executionEnvironment» — software runtime on hardware
- artifact** — Physical deployed file (.war, .jar, .xml etc.)
- deploy** — artifact deployed onto node
- manifest** — artifact implements component
- Solid arrow** — usage/dependency (shared library, bundle, communication path)
- Teal line** — HTTP communication path
- Blue line** — TCP/IP communication path
- Green line** — SMTP communication path

Here's the complete answer diagram. Let me walk you through what's shown:

Node hierarchy (outside → in): Student Laptop («device») → Windows 11 («executionEnvironment») → Browser component

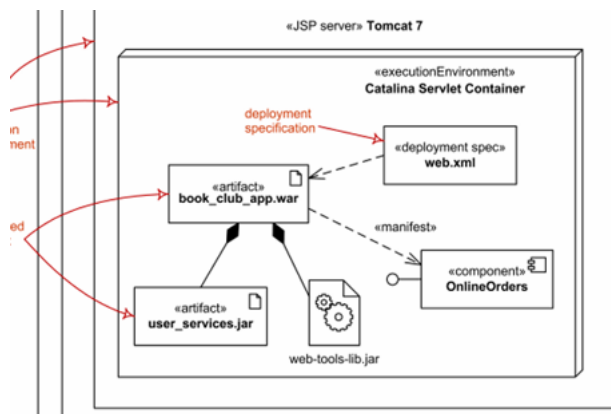
Linux Web Server («device») → Apache Tomcat 9 («executionEnvironment») → all artifacts and components inside

Database Server («device») → Oracle 19c («executionEnvironment») → three schemas

Mail Server («device») → NotificationService component

Key relationships shown:

- web.xml has a «deploy» dashed arrow to exam_portal.war (config governs deployment)
- exam_portal.war has a «manifest» dashed arrow to ExamEngine (war file implements that component)
- Both exam_portal.war and auth_service.jar have solid arrows pointing to commons-utils.jar (usage dependency — shared library)
- Firewall sits between student and internal servers
- Protocols labeled on every connection: HTTP, TCP/IP, SMTP



Do like this above arrows are wrong

State Chart Diagram

STATE CHART DIAGRAMS

State charts are excellent for showing how an object's behavior changes across multiple use case executions. However, they are not good for showing interactions between multiple objects. For that, you'd use a Sequence Diagram.

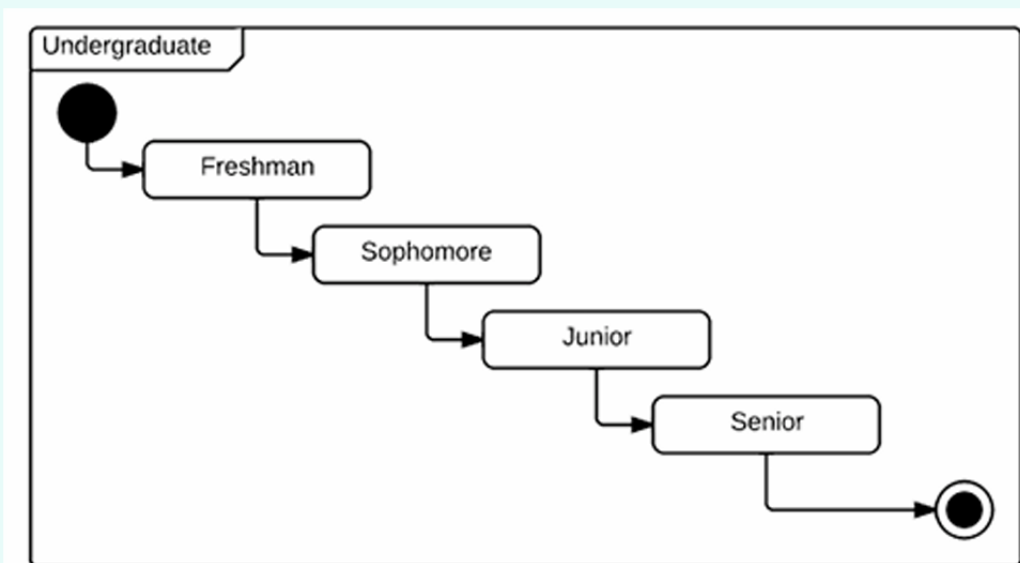
Examples to Understand the Concept

Library Book: Available → Reserved → Borrowed → Returned → Available (transitions triggered by user actions like reserving, borrowing, returning).

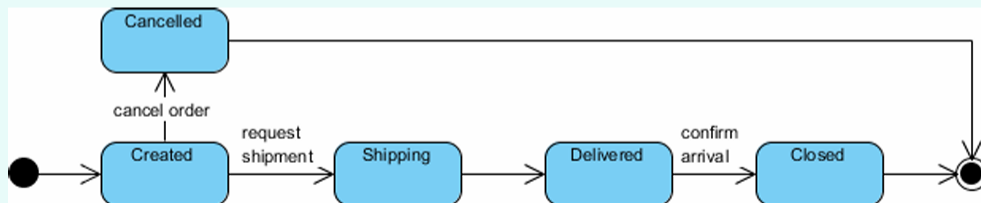
ATM Card: Inserted → Verified → Ejected or Blocked (transitions triggered by PIN entry, card removal, wrong PIN etc).

Order: Created → Shipping → Delivered → Closed (or Cancelled from Created).

STATE CHART DIAGRAMS



ORDER STATE CHART DIAGRAMS



WHY USE STATE CHART DIAGRAMS?

The power of state charts is that they model state-dependent behavior. This means an object can respond differently to the same event depending on what state it's in.

For example, the event `keyPress` will have a different effect if your keyboard is in the `"caps_locked"` state versus the `"default"` state. We usually apply this to objects, but it can be used for any element with behavior, like a whole system or a use case.

WHY USE STATE CHART DIAGRAMS?

state charts are often used hand-in-hand with Sequence Diagrams. A state chart describes all possible events for a single object; a sequence diagram describes the events for a single interaction across all involved objects.

STATE

A State is a condition or situation during the life of an object where it satisfies some condition, performs some activity, or waits for an event. Examples: an Invoice is Paid. A Car is Standing Still. Jim is Acting as a Salesman



State

Draw | T | A | Ask Copilot | 13 of 37

STATE

A state can have up to three compartments.

State with activities

entry/ entryActivity
do/ doActivity
exit/ exitActivity

Name

Internal Activities

Internal Transitions

1. The Name of the state (e.g., Idle, Processing).
2. The optional Activity compartment. This lists behaviors in response to events. UML has three standard events here:
 - entry: An action performed upon entering the state (e.g., entry / assignValue).
 - exit: An action performed upon exiting the state.
 - do: A long-running activity performed while inside the state (e.g., do / runSelfTest).
3. The optional Internal Transitions compartment. These are transitions that cause an action but do not result in a change of state.

INITIAL AND FINAL STATE

Two special states are crucial:

- The Initial State is where the object's life begins. It's a single, filled black circle.
- The Final State is where the object's life ends. It's a filled circle inside a larger circle. An object can have multiple final states.

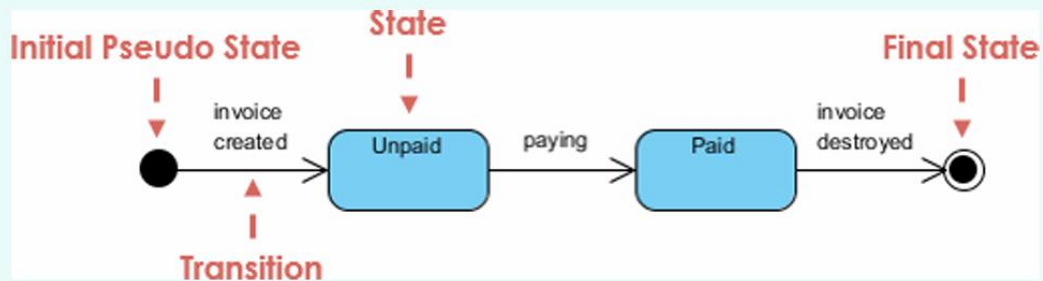


Initial State



Final state

INITIAL AND FINAL STATE



TRANSITION

A **transition** is a solid arrow from a source state to a target state.

It can be labeled with the following syntax:

Event [Guard] / Action

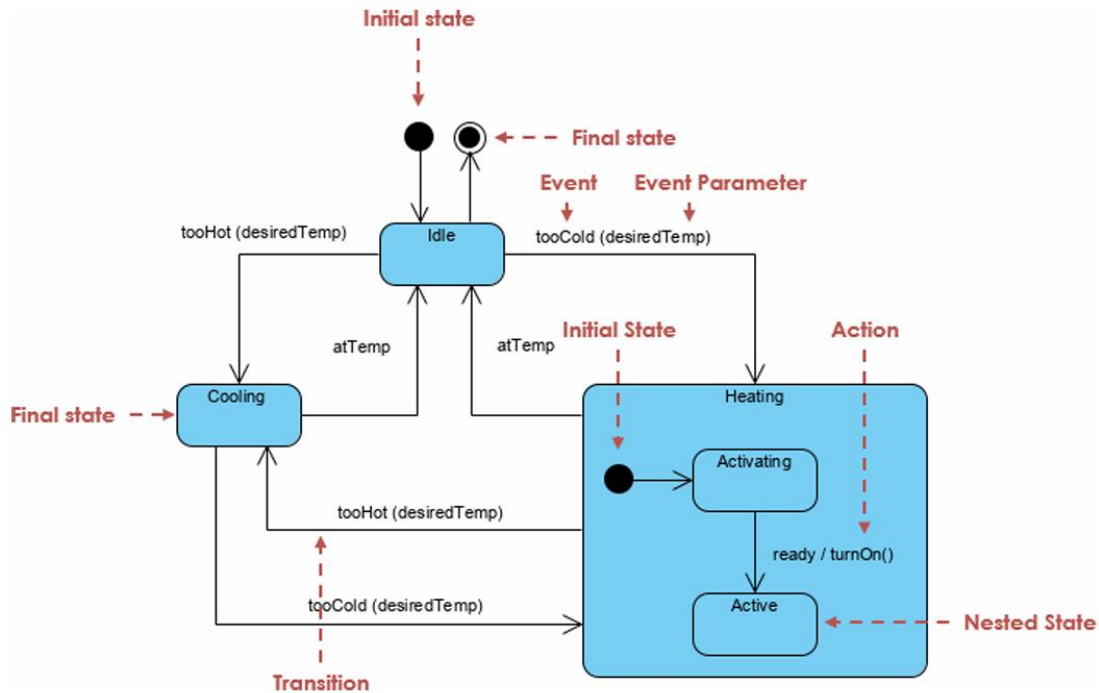


- Event: The happening that triggers the transition.
- Guard: A Boolean condition in square brackets [] that must be true for the transition to fire.
- Action: An executable, atomic computation that occurs during the transition.

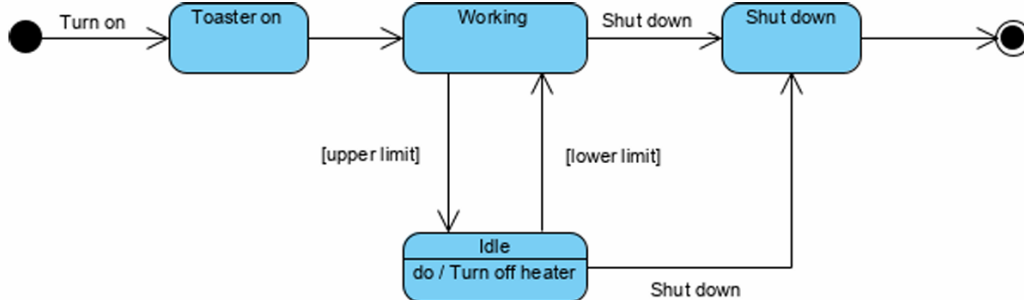


Turnon () – action

A STATE CHART DIAGRAM AT A GLANCE



EXAMPLE 2



5. Events

An event is an instant in time significant to the object that triggers a transition. UML defines four types:

Change Event — fires when a Boolean condition becomes true. Written as `when(condition)` or shown as a guard `[condition]`. Example: `[temperature > 100]`.

Signal Event — receipt of an explicit signal/message from another object. Example: receiving a notification.

Call Event — receipt of a call for an operation from another object. Example: a method call. Both Signal and Call events are messages.

Time Event — passage of a designated period of time. Written as `after(5 seconds)` or `when(date)`. Example: `after(15 sec.)` in the phone call diagram transitions from DialTone to Timeout.

6. Guard Condition

A guard is a Boolean expression in square brackets on a transition. The transition only fires if the guard evaluates to true. If the guard is false, the transition is ignored even if the event occurs.

Example: `withdraw(amount) [balance >= amount]` — the withdraw event is fired, but money only comes out if balance is sufficient.

Time guard example: `[t = 15sec]`

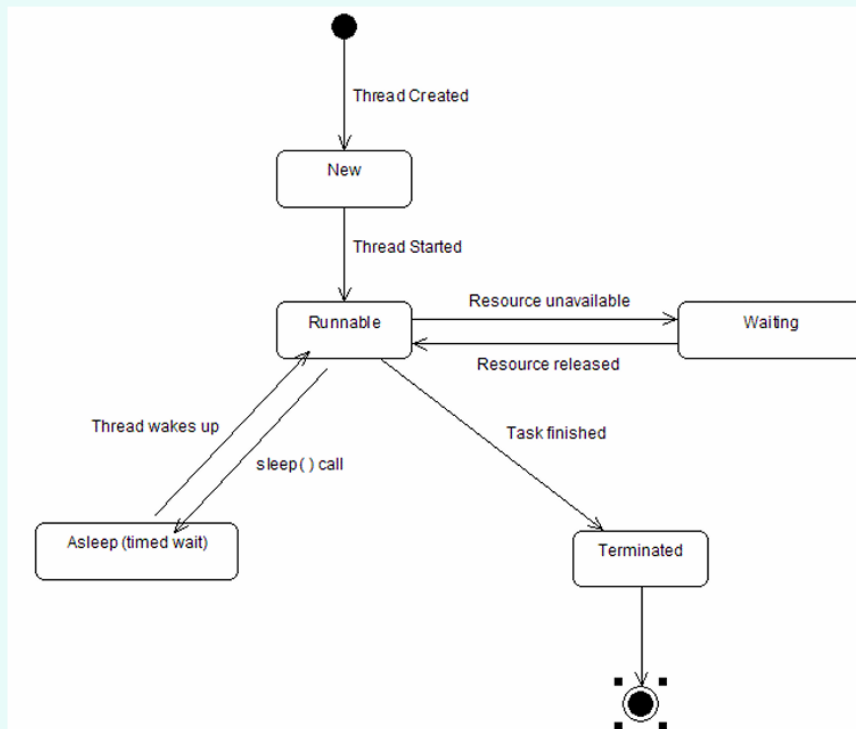
7. Internal Transition

An internal transition responds to an event with an action but does NOT change the state. The object stays in the same state. Entry and exit actions are NOT executed.

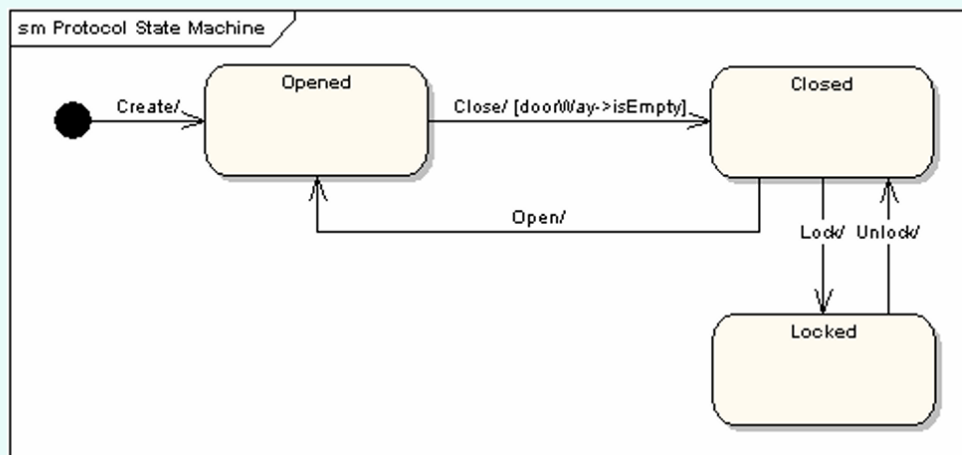
Example from keyboard: `ANY_KEY / send_lower_case_scan_code()` is an internal transition in the default state. The key press does something but the keyboard stays in default state.

Only `CAPS_LOCK` actually causes a state change to `caps_locked`.

EXAMPLE

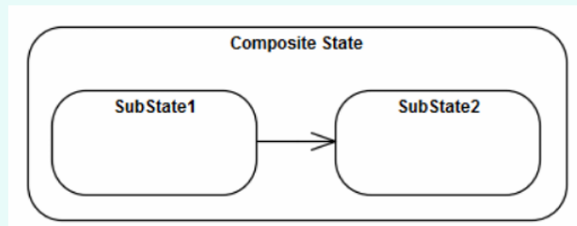


EXAMPLE

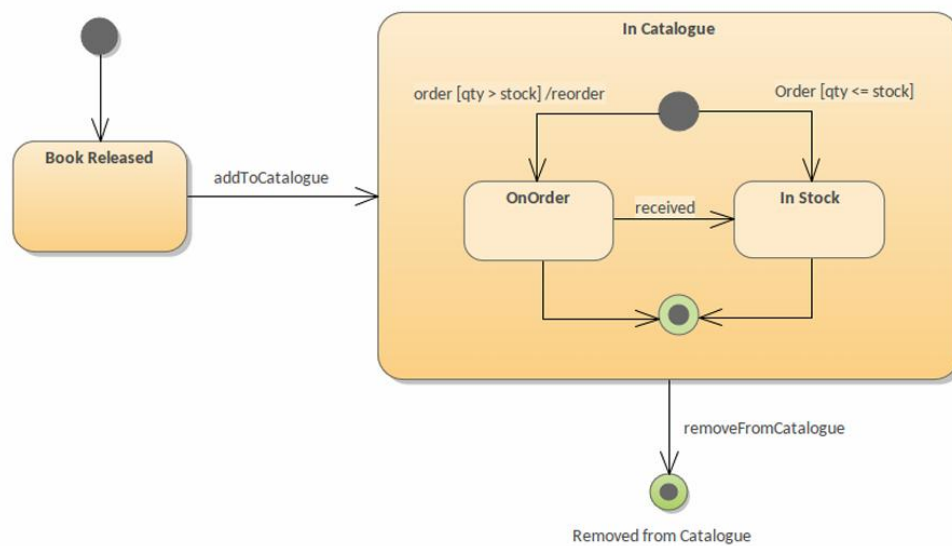


COMPOSITE STATE

States can be nested. A Composite State is a state that contains substates. This is great for managing complexity. The key rule is: a transition into a composite state enters its initial substate. A transition out of a composite state applies to all substates inside it.



COMPOSITE STATE



PHONE CALL: EXAMPLE

```
stateDiagram-v2
    [*] --> Idle
    Idle --> DialTone : lift receiver / get dial tone
    DialTone --> Dialing : dial digit(n)
    DialTone --> Timeout : after (15 sec.)
    Dialing --> Dialing : dial digit(n) [incomplete]
    Dialing --> Invalid : dial digit(n) [invalid]
    Dialing --> Connecting : dial digit(n) [valid] / connect
    Invalid --> DialTone : do / play dial tone
    Invalid --> Timeout : after (15 sec.)
    Connecting --> Busy : busy
    Connecting --> Ringing : connected
    Ringing --> Talking : callee answers / enable speech
    Talking --> Pinned : callee hangs up
    Talking --> Idle : caller hangs up / disconnect
```

The diagram illustrates the states and transitions of a phone call system. The states are represented by rounded rectangles, and the transitions are labeled with events and actions.

States:

- Idle**: Initial state. Action: lift receiver / get dial tone.
- DialTone**: State after dial tone is played. Action: do / play dial tone.
- Dialing**: State during dialing. Action: do / play message.
- Connecting**: State during connection. Action: do / play message.
- Ringing**: State during ringing. Action: do / play ringing tone.
- Talking**: State during conversation. Action: do / play message.
- Pinned**: State when the call is pinned. Action: do / play message.
- Busy**: State when the line is busy. Action: do / play busy tone.
- Invalid**: State when the dialing is invalid. Action: do / play message.
- Timeout**: State when the dialing times out. Action: do / play message.

Transitions:

- Idle** to **DialTone**: lift receiver / get dial tone.
- DialTone** to **Dialing**: dial digit(n).
- DialTone** to **Timeout**: after (15 sec.).
- Dialing** to **Dialing**: dial digit(n) [incomplete] (self-loop).
- Dialing** to **Invalid**: dial digit(n) [invalid].
- Dialing** to **Connecting**: dial digit(n) [valid] / connect.
- Invalid** to **DialTone**: do / play dial tone.
- Invalid** to **Timeout**: after (15 sec.).
- Connecting** to **Busy**: busy.
- Connecting** to **Ringing**: connected.
- Ringing** to **Talking**: callee answers / enable speech.
- Talking** to **Pinned**: callee hangs up.
- Talking** to **Idle**: caller hangs up / disconnect.

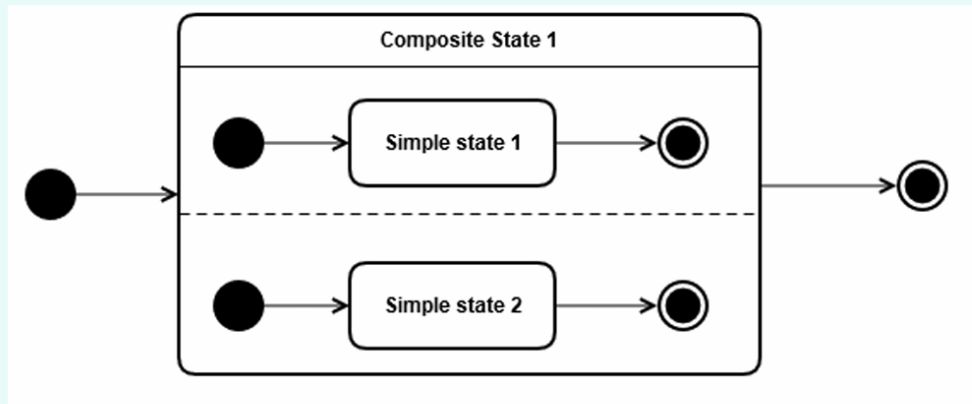
CONCURRENT STATE

You can model an object being in multiple states at the same time by using an orthogonal state—a composite state divided into multiple regions by a dashed line.

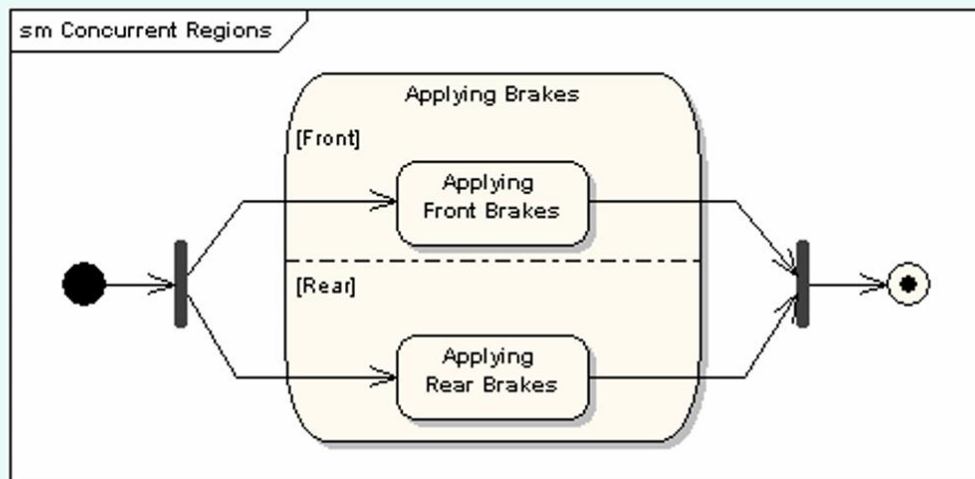
For example: the state "Applying Brakes" might have two concurrent regions: "Front Brakes" and "Rear Brakes," each with their own substates. When you enter "Applying Brakes," you enter both regions simultaneously. You can also show this explicitly using fork and join pseudostates.

For example: the state "Applying Brakes" might have two concurrent regions: "Front Brakes" and "Rear Brakes," each with their own substates. When you enter "Applying Brakes," you enter both regions simultaneously. You can also show this explicitly using fork and join pseudostates.

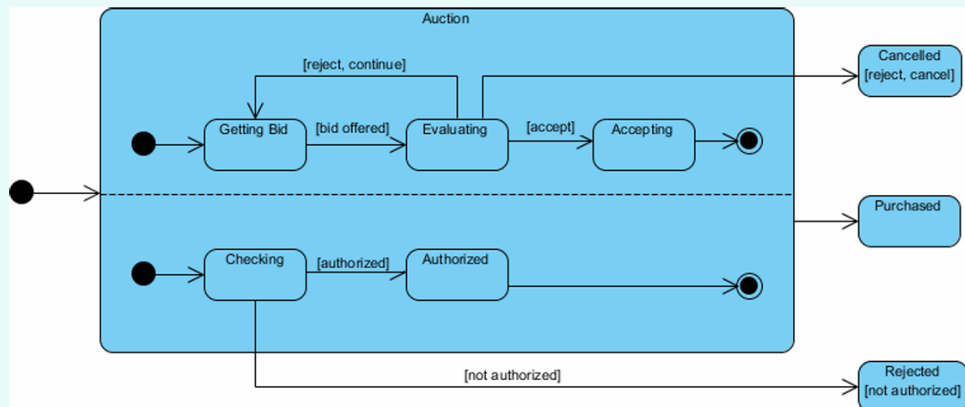
CONCURRENT STATE



CONCURRENT STATE



CONCURRENT STATE

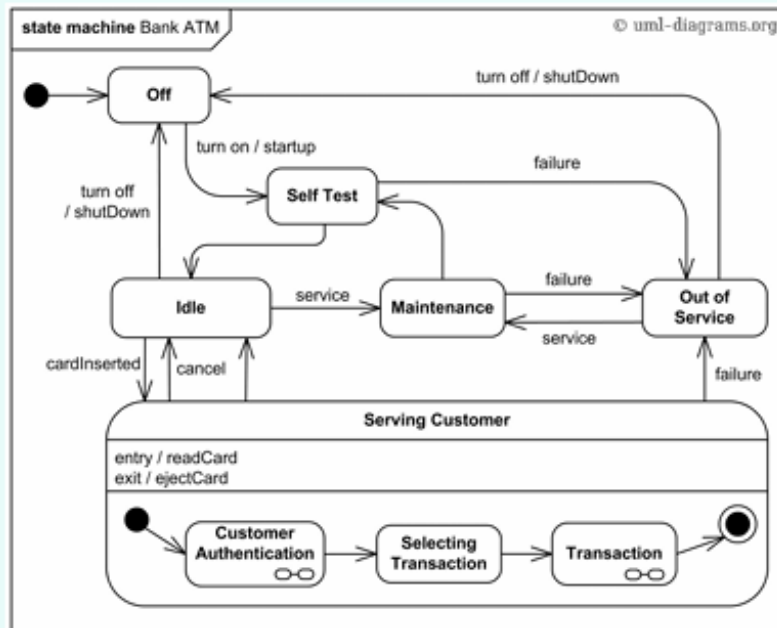


SCENARIO

ATM is initially turned off. After the power is turned on, ATM performs startup action and enters Self Test state. If the test fails, ATM goes into Out of Service state, otherwise there is triggerless transition to the Idle state. In this state ATM waits for customer interaction.

The ATM state changes from Idle to Serving Customer when the customer inserts banking or credit card in the ATM's card reader. On entering the Serving Customer state, the entry action readCard is performed. Note, that transition from Serving Customer state back to the Idle state could be triggered by cancel event as the customer could cancel transaction at any time.

SCENARIO



A very common exam question is "How do we identify states, events, and actions from a scenario?"

Use this simple rule:

Element	Usually identified by	Example	📄
State	Nouns / conditions / situations	Off, Idle, Self Test, Serving Customer	
Event (Trigger)	Verbs or things that happen	cardInserted, cancel, turnOn	
Action	Verbs (something the system does)	startup(), readCard(), ejectCard()	
Guard Condition	Boolean condition in brackets	[test fails], [qty > stock]	

Sentence 2

After the power is turned on, ATM performs startup action and enters Self Test state.

Break it:

power turned on

→ Event: `turnOn`

startup

→ Action: `startup()`

Self Test

→ State

Transition:

Off -- turnOn / startup --> Self Test



Sentence 7

On entering the Serving Customer state, readCard is performed.

readCard

→ Action

Because it happens when entering the state:

entry / readCard



inside the state box.

DESIGN SOLUTIONS



DESIGN PATTERNS

- **Definition:** A design pattern is a proven reusable design solution to a commonly occurring problem in software design.
- **Nature:** They are not finished designs but rather templates/descriptions for solving problems
- **Purpose:** Help designers reach the right design faster by leveraging proven solutions

What is a Design Pattern?

A proven, reusable solution to a commonly occurring problem in software design. It is **not** finished code — it's a template for solving a problem that can be applied in many situations.

Key Characteristics

- Standard solutions to common programming problems
- Techniques for making code more flexible
- High-level programming idioms
- Describe connections among program components



DESIGN PATTERNS & OOP

Design patterns extend OOP by:

- Providing higher-level abstractions
- Guiding large-scale design decisions
- Making your architecture easier to change
- Helping teams communicate (shared vocabulary: “use a Factory”, “apply Observer”, etc.)

Pattern Elements

Element	Description
Name	Meaningful identifier
Problem	When to apply the pattern
Solution	Elements, relationships, responsibilities
Consequences	Results and trade-offs

There are three types of design :

Creational

Structural

Behavioral

Creational — Control *how* objects are created. Encapsulate instantiation to make systems flexible and easy to extend.

Creational Patterns

Examples of Creational Patterns:

- Factory Method
- Abstract Factory
- Singleton
- Builder
- Prototype

Structural design patterns focus on how classes and objects are composed to form larger, flexible structures. They help ensure that system architecture is efficient, scalable, and easy to refactor by defining relationships that simplify complexity

Examples of Structural Patterns:

- Adapter
- Composite
- Proxy
- Decorator
- Facade
- Flyweight

BEHAVIORAL DESIGN PATTERNS

Behavioral design patterns define how objects interact and communicate, distributing responsibilities and enabling flexible control flows. They help make systems more modular, understandable, and adaptable to change.

BEHAVIORAL DESIGN PATTERNS

Examples of Behavioral Patterns:

- Strategy
- Observer
- State
- Command
- Iterator
- Template Method



DESCRIBING DESIGN PATTERNS



- Intent
 - Short description of the pattern & its purpose
- Also Known As
 - Any aliases this pattern is known by
- Motivation
 - Motivating scenario demonstrating pattern's use
- Applicability
 - Circumstances in which pattern applies
- Structure
 - Graphical representation of the pattern using modified UML notation



DESCRIBING DESIGN PATTERNS

- Participants
 - Participating classes and/or objects & their responsibilities
- Collaborations
 - How participants cooperate to carry out their responsibilities
- Consequences
 - The results of application, benefits, liabilities
- Implementation
 - Pitfalls, hints, techniques, plus language-dependent issues



DESCRIBING DESIGN PATTERNS

- Sample Code
 - Implementations in C++, Java, C#, Smalltalk, etc.
- Known Uses
 - Examples drawn from existing systems
- Related Patterns
 - Discussion of other patterns that relate to this one

Factory Pattern (Creational)

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate. Also known as the "Virtual Constructor" pattern. It is the most widely used pattern in software engineering.

Applicability — use when:

- A class cannot anticipate the type of objects it needs to create
- The application knows the superclass but not the concrete implementation at compile time
- A class wants its subclasses to specify which objects get created
- Responsibility for object creation needs to be localized to specific helper subclasses

Factory Pattern (3 Marks)

The Factory Pattern is a **Creational design pattern** that defines an interface for creating an object but lets subclasses decide which class to instantiate. Also known as the "**Virtual Constructor**" pattern, it is the most widely used pattern in software engineering.

It is used when a class cannot anticipate the type of objects it must create at compile time. Instead of calling `new` directly, the client asks a factory class which returns the appropriate object based on given information.

Example: A `ShapeFactory` receives a string "CIRCLE", "RECTANGLE", or "SQUARE" and returns the corresponding `Shape` object. The client never directly instantiates any shape — the factory handles all object creation.

EXAMPLE

- We have a `MediaPlayer` interface and a concrete class `AudioPlayer` implementing the `MediaPlayer` interface. `AudioPlayer` can play mp3 format audio files by default.
- We are having another interface `AdvancedMediaPlayer` and concrete classes implementing the `AdvancedMediaPlayer` interface. These classes can play vlc and mp4 format files.
- We want to make `AudioPlayer` to play other formats as well. To attain this, we have created an adapter class `MediaAdapter` which implements the `MediaPlayer` interface and uses `AdvancedMediaPlayer` objects to play the required format.
- `AudioPlayer` uses the adapter class `MediaAdapter` passing it the desired audio type without knowing the actual class which can play the desired format. `AdapterPatternDemo`, our demo class will use `AudioPlayer` class to play various formats.



INTENT

Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.



APPLICABILITY

1. Use the Adapter class when you want to use some existing class, but its interface isn't compatible with the rest of your code.
2. Use the pattern when you want to reuse several existing subclasses that lack some common functionality that can't be added to the superclass.



PROBLEM EXAMPLE

Imagine that you're creating a stock market monitoring app. The app downloads the stock data from multiple sources in XML format and then displays nice-looking charts and diagrams for the user.

At some point, you decide to improve the app by integrating a smart 3rd-party analytics library. But there's a catch: the analytics library only works with data in JSON format.



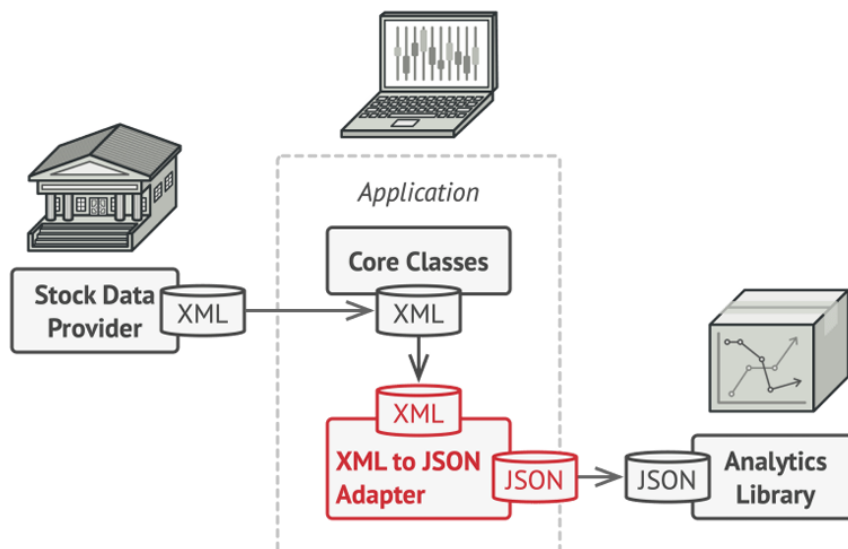
SOLUTION

You can create an adapter. This is a special object that converts the interface of one object so that another object can understand it.

Adapters can not only convert data into various formats but can also help objects with different interfaces collaborate. Here's how it works:

1. The adapter gets an interface, compatible with one of the existing objects.
2. Using this interface, the existing object can safely call the adapter's methods.
3. Upon receiving a call, the adapter passes the request to the second object, but in a format and order that the second object expects.

SOLUTION



UML Roles:

Role	Description
Client	Your existing system that makes requests
Target	The interface the client expects
Adapter	Implements Target; internally calls the Adaptee
Adaptee	The existing class with the incompatible interface

ADAPTER DESIGN PATTERN UML DIAGRAM

